

LLM4Log: A Systematic Review of Large Language Model-based Log Analysis

ZEYANG MA, Software Performance, Analysis, and Reliability (SPEAR) lab, Concordia University, Canada
JINQIU YANG, Department of Computer Science and Software Engineering, Concordia University, Canada
TSE-HSUN (PETER) CHEN, Software Performance, Analysis, and Reliability (SPEAR) lab, Concordia University, Canada

Software systems generate massive, evolving, semi-structured logs that are central to reliability engineering and AIOps, yet difficult to analyze at scale under drift and limited labels. Recent advances in pretrained Transformer models and instruction-tuned large language models (LLMs) have reshaped log analysis by enabling semantic generalization and cross-source evidence integration, but also introducing deployment risks such as context limits, latency and cost, privacy constraints, and hallucinations. This paper presents **LLM4Log**, a systematic review of LLM-based log analysis across the end-to-end pipeline, from upstream logging-statement generation and maintenance to log parsing/structuring and downstream tasks including anomaly detection, failure prediction, root cause analysis, and log summarization. Following a structured search and manual screening protocol, we completed literature collection in November 2025 and identified 145 unique papers across seven logging tasks. We organize the research area through a unified, task-driven taxonomy, summarize common design patterns (prompting/ICL, retrieval grounding, fine-tuning, tool/agent augmentation, and verification), and analyze evaluation practices, datasets, metrics, and reproducibility. Based on these cross-paper analyses, we summarize key lessons and open challenges for reliable real-world adoption. We emphasize robustness under drift and long-tail events, grounding and faithfulness for operator-facing outputs, and deployment-oriented designs with verifiable behavior.

CCS Concepts: • **Software and its engineering** → **Software maintenance tools**.

Additional Key Words and Phrases: Log analysis, Software logs, Large language models

1 Introduction

Software systems, from cloud services and microservices to networked and cyber-physical infrastructures, emit massive volumes of execution logs that record runtime states, events, and failures. Logs are a primary source of evidence for reliability engineering and operations: they support debugging, monitoring, incident triage, and postmortem analysis [36, 192, 219]. However, modern log streams are notoriously difficult to analyze at scale. They are *semi-structured* (mixing free-form text with identifiers and code-like tokens), *high-volume/high-velocity*, and *continuously evolving* due to software updates, configuration changes, and deployment diversity. Consequently, manual inspection and keyword search remain common yet brittle, while automated pipelines often suffer from format drift, long-tail behaviors, limited labeled data, and inconsistent logging quality [21, 46, 47, 87, 139].

Authors' Contact Information: Zeyang Ma, Software Performance, Analysis, and Reliability (SPEAR) lab, Concordia University, Montreal, Canada, m_zeyang@encs.concordia.ca; Jinqiu Yang, Department of Computer Science and Software Engineering, Concordia University, Montreal, Canada, jinqiu.yang@concordia.ca; Tse-Hsun (Peter) Chen, Software Performance, Analysis, and Reliability (SPEAR) lab, Concordia University, Montreal, Canada, peterc@encs.concordia.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/8-ART

<https://doi.org/10.1145/3842759>

To address these challenges, the research community has developed a rich body of work on automated log analysis, typically organized as a pipeline spanning logging practice, parsing/structuring, representation learning, anomaly/failure detection, failure prediction, diagnosis, and summarization [47]. In production settings, however, log analysis is constrained by operational realities. Incidents unfold under time pressure. Log streams are interleaved across services, hosts, and tenants. Failures manifest through long causal chains. The most diagnostic events are often rare, noisy, or buried in high-volume background messages [102, 192]. Moreover, engineers rarely rely on logs alone. Effective triage requires correlating logs with metrics/traces, historical incidents, runbooks, and code/configuration context [102, 192]. These characteristics make log analysis not only a *pattern recognition* problem, but also an *evidence-integration* and *human decision-support* problem. Log-centric LLM systems are therefore distinct from broader LLM4AIOps settings: logs combine natural-language-like descriptions with code-like identifiers, exhibit ambiguous template/value boundaries, evolve under version and deployment drift, and often carry the chronological symptom evidence that operators use to diagnose failures. The challenge is not merely to classify patterns, but to interpret noisy, long-tail runtime evidence in a form that remains useful for human decision making.

The classic log-analysis pipeline was largely shaped before instruction-tuned LLMs became widely available, when methods were either rule-based or relied on fixed representations and task-specific supervision. In recent years, there has been a rapid methodological transition: pretrained Transformer language models, including modern LLMs, are increasingly used as the backbone for log analysis. Unlike traditional approaches that heavily depend on templates, handcrafted features, or patterns inferred from previously observed log outputs, LLMs offer semantic generalization and can condition on heterogeneous evidence (e.g., logs together with tickets, traces/metrics, runbooks, configurations, and code/configuration context), which can be especially useful in modern operational workflows [20, 42, 68]. In log-centric settings, this matters because the same failure may surface through heterogeneous wording across components and versions, while the most useful diagnostic evidence is often sparse, context dependent, and embedded in long chains of runtime messages instead of a single fixed template.

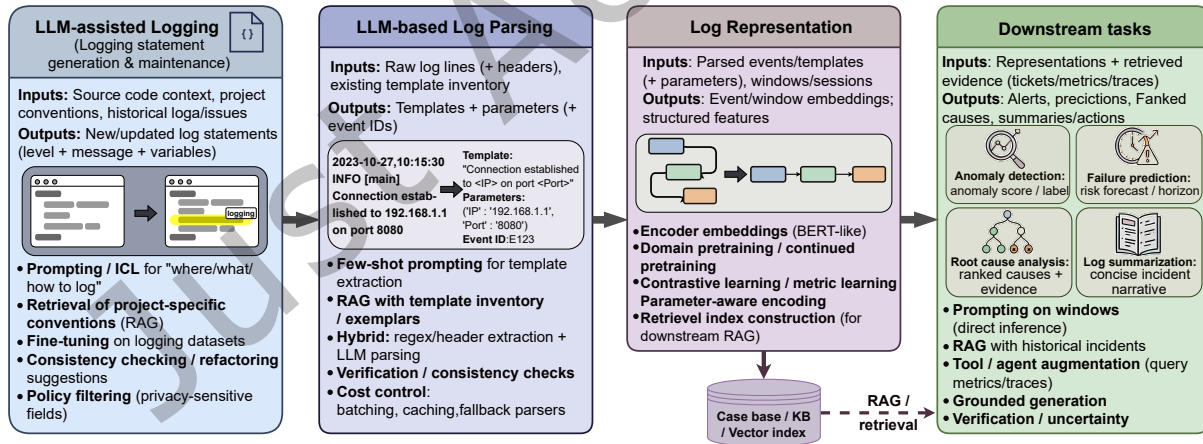


Fig. 1. An overview of LLM-based log analysis across the pipeline, from logging to downstream tasks. Note that some downstream studies operate directly on raw or lightly normalized log windows and therefore bypass explicit log parsing.

This shift is already visible across the *entire* log-analysis pipeline. Figure 1 provides an overview of how LLMs are applied across the log-analysis pipeline. Upstream, LLMs are used to **generate and maintain logging**

statements, helping developers decide where to log and what to record under project conventions. At the core of many pipelines, LLMs are applied to **log parsing**, converting raw messages into structured templates and parameters with improved robustness under drift, unseen formats, and long-tail events [6]. Between parsing and downstream inference, LLMs are also increasingly used for **log representation learning**, producing semantic embeddings or contextualized event representations for logs that improve robustness and transferability under drift. Downstream, LLMs are used for **anomaly detection** and **failure prediction**, but also increasingly for operator-facing tasks such as **root cause analysis** (RCA) and **log summarization**, where explanation quality and evidence integration are as important as prediction accuracy [20, 42]. Although parsing is a common intermediate stage, some downstream studies instead operate on raw or lightly normalized logs, so the pipeline should be read as a common analytical structure, not a mandatory sequence of steps. At the same time, LLM-based log analysis introduces new trade-offs and risks that are less prominent in classic pipelines: context window limits, latency/cost constraints in online settings, privacy and data-governance concerns, sensitivity to prompting and retrieval design, and hallucinations that can mislead incident response if outputs are not properly grounded and verified [20, 68].

Given the pace and breadth of this emerging area, practitioners and researchers need a consolidated, task-driven view of (i) *where* LLMs are applied in the log-analysis pipeline, (ii) *how* they are integrated (e.g., prompting/ICL, retrieval grounding, fine-tuning, tool/agent augmentation, and verification), (iii) *how* they are evaluated (datasets, metrics, baselines, and reproducibility), and (iv) *what* remains open for real-world deployment. While prior surveys have reviewed automated log analysis broadly [47] and recent work has summarized specific subareas such as LLM-based log parsing [6], a systematic, end-to-end review of the full LLM4Log landscape is timely.

In this paper, we present **LLM4Log**, a systematic review of *large-language-model-based log analysis*. We focus on software runtime logs as the central evidence source. For downstream tasks, studies are within the log-specific scope when runtime logs are the main evaluated input or are explicitly evaluated alongside another operational data source. When log use is optional, only results that explicitly involve runtime logs support log-specific findings. Using a task-driven search and screening protocol, we reviewed 162 task-paper records across seven log-analysis tasks, with literature collection completed in November 2025. Deduplicating repeated task assignments by paper identity yielded 145 unique papers published from 2020 to 2025. Building on this corpus, we organize the field through a unified taxonomy and cross-paper analyses.

Our goal is to offer a consolidated, end-to-end reference for how LLMs are applied to log analysis, what works in practice, and what remains open. To that end, we make the following contributions:

- **A systematic, task-driven mapping of LLM4Log research (2020–2025).** We provide a curated and categorized corpus, summarize the temporal and task distribution, and characterize where the community concentrates effort and where gaps remain.
- **A pipeline-level review of log-centric LLM methods.** We review and compare LLM-based approaches for (i) logging statement generation and maintenance, (ii) log parsing, and (iii) downstream tasks (anomaly detection, failure prediction, root cause analysis, and log summarization), highlighting reusable design patterns and recurring trade-offs across the log-analysis workflow.
- **Cross-cutting insights, evaluation caveats, and future directions for LLM4Log.** We distill lessons about evaluation practices, robustness under drift and long-tail events, grounding and faithfulness for operator-facing outputs, privacy/security constraints, deployment practicality, and reproducibility, and we summarize these observations in a dedicated cross-cutting section that outlines priorities for reliable real-world adoption.

Paper organization. The rest of this paper is organized as follows. Section 2 describes our systematic survey methodology. Section 3 provides LLM background and a taxonomy of adaptation/enhancement paradigms for log analysis. Section 4 reviews LLM-based logging statement generation and maintenance. Section 5 surveys

LLM-based log parsing. Section 6 covers downstream log analysis tasks, including anomaly detection, failure prediction, root cause analysis, and log summarization. Section 7 synthesizes recurring insights, open challenges, and future directions across tasks. Finally, Section 8 concludes the survey.

2 Survey Methodology

To build the literature corpus for this survey, we conducted a structured search for publications on *LLM-based log analysis*. We used a flagship-venue-first search strategy. We started from major software engineering venues to build a focused initial paper set. We recognize that this choice may under-represent some interdisciplinary or industry-facing work. We therefore complemented the initial venue-based search with backward and forward snowballing, and with digital-library and academic-search cross-checking to broaden coverage. The literature collection was completed in November 2025. The complete paper list and corpus metadata are available in our public repository: <https://github.com/zeyang919/LLM4Log>.

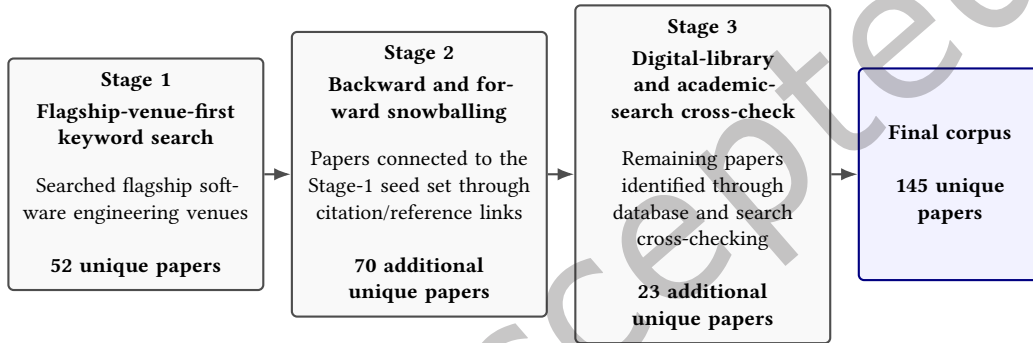


Fig. 2. Contribution of the three literature-collection stages to the final corpus of 145 unique papers. The values report the number of papers added at each stage.

2.1 Literature collection protocol

Stage 1: flagship-venue-first keyword search. We began with keyword-based searches in the following software engineering venues, which constituted our Stage-1 flagship-venue-first search set: ICSE, FSE, ASE, ISSTA, ICST, ICSME, ISSRE, ESEM, ICPC, TSE, TOSEM, EMSE, JSS, and IST as shown in Table 1. This stage yielded 52 unique papers, as summarized in Figure 2. Table 1 summarizes the venues covered in the Stage-1 search.

Table 1. Stage-1 search venues and year range.

Type	Venues	Years
Conference	ICSE, FSE, ASE, ISSTA, ICST, ICSME, ISSRE, ESEM, ICPC	2020–2025
Journal	TSE, TOSEM, EMSE, JSS, IST	2020–2025

We used combinations of *log-related* and *LLM-related* keywords. The log-related keywords include terms such as “log”, “system log”, “runtime log”, “log analysis”, “log parsing”, “log anomaly detection”, “failure prediction”, “root cause analysis”, “incident diagnosis”, “log summarization”, the LLM-related keywords include terms such as “large

language model”, “LLM”, “Transformer”, “BERT”, “GPT”, “ChatGPT”, “in-context learning”, “prompting”, “retrieval-augmented generation”, and “agent”. We iteratively refined the keyword set as we encountered task-specific terminology and community-preferred phrasing (e.g., operational diagnosis and troubleshooting terms).

Stage 2: snowballing to extend coverage. From the seed set obtained in Stage 1, we performed backward snowballing by inspecting references and forward snowballing by checking citations to discover additional relevant papers that may not be easily retrieved via keyword queries, such as papers using non-standard task names or papers whose titles and abstracts emphasize systems and operations without using “log.” We also tracked community resources that aggregate log-analysis and LLM-for-operations literature, such as benchmarks, dataset/toolkit papers, and curated lists, as auxiliary entry points. This stage contributed 70 additional unique papers (Figure 2).

Stage 3: digital-library cross-check. To reduce omission risk and broaden coverage beyond the Stage-1 venue set, we cross-checked our corpus using major digital libraries and preprint servers (i.e., IEEE Xplore, ACM Digital Library, SpringerLink, Elsevier/ScienceDirect, Wiley Online Library, arXiv, and OpenReview), and used Google Scholar for additional coverage checks and citation tracing. This stage contributed the remaining 23 unique papers. Figure 2 summarizes how the three stages contributed to the final corpus of 145 unique papers.

Scope control and manual screening. Keyword searches inevitably return false positives due to term ambiguity and task overlap. For example, some papers include both “root cause analysis” and “LLM” keywords, but evaluate only metrics or traces, while some anomaly-detection papers use network flows, sensor telemetry, or general text without runtime logs. Similarly, the term “log” can refer to non-runtime logs (e.g., query logs, change logs, process mining event logs) or even mathematical “log” usage. Candidate papers were manually screened by examining their titles, abstracts, and relevant method and evaluation sections. Inclusion was decided at the paper level using the criteria below. We consolidated repeated occurrences of the same paper across task assignments and collection stages by title and stable identifier. For downstream tasks, we distinguish three cases and apply them consistently across all task sections. Studies that explicitly evaluate software or system runtime logs as the main input are within the log-specific scope. Multimodal studies are also within scope when runtime logs are explicitly evaluated together with at least one other operational data source, such as metrics, traces, code, tickets, or topology. When logs are optional or used only in some experimental settings, only results that explicitly involve runtime logs are used to support log-specific findings. If those results cannot be separated, the study may be discussed as a related comparison, but it is not used as evidence about log analysis.

Runtime logs in this review include application, system, security/audit, shell, build, test, and console event records produced during software or system operation. Host audit and provenance records qualify when they record process actions or system calls. Metrics, distributed traces, network packets or flows, sensor telemetry, tickets, and incident summaries do not qualify by themselves. A derived artifact qualifies only when the study documents that it was constructed directly from runtime logs.

Inclusion and exclusion criteria. A publication contributes log-specific evidence if it satisfies all of the following: (1) it targets one or more tasks represented in our taxonomy (e.g., logging, parsing/representation, anomaly/failure detection, failure prediction, diagnosis/RCA, or summarization). (2) It uses a Transformer-based pretrained language model (encoder-only, encoder-decoder, or decoder-only) via prompting/ICL, retrieval grounding, fine-tuning/post-training, tool augmentation/agents, or representation learning. (3) It reports an empirical evaluation, ablation/analysis, or a concrete case study in which logging statements are the primary technical output or runtime logs are an explicit input to the method or benchmark, or it presents a concrete technical design directly focused on logging. Concrete vision or design records are described as such and are not used as empirical evidence. Related comparisons are identified in the discussion and are not treated as evidence for log-specific findings. We exclude work whose task is outside our taxonomy.

Data extraction protocol. For each paper, we extracted structured information to support later taxonomy construction and cross-paper comparison. When a paper covered several tasks, we created a task-paper record

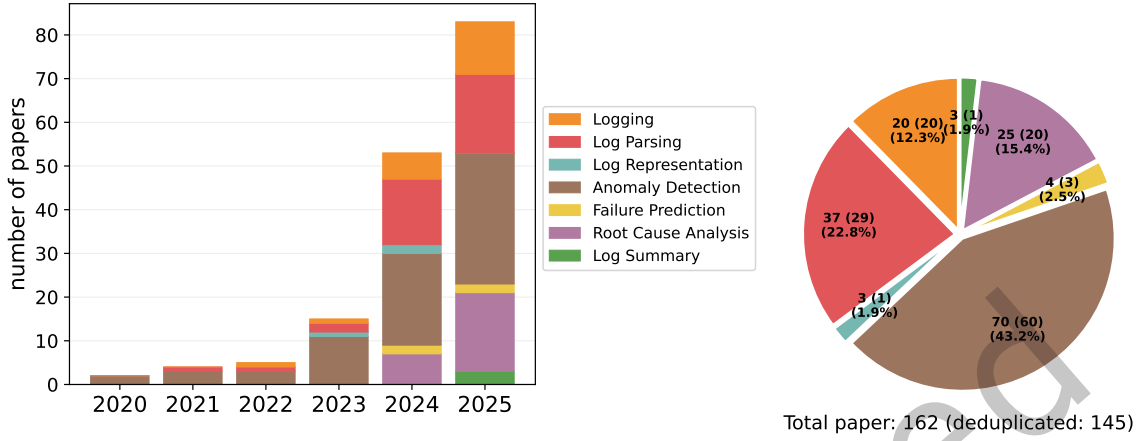


Fig. 3. Temporal and task distribution of the 162 task-paper records. Labels use $n(u)$, where n is the number of task-paper records and u is the number of papers assigned only to that task. The figure also reports the deduplicated total of 145 unique papers.

for each assignment and extracted fields relevant to that task. Across tasks, we recorded: (i) target task(s), (ii) dataset type and whether logs were used alone or with auxiliary artifacts (e.g., metrics/traces/tickets/runbooks/-code/config), and (iii) evaluation setup. In addition, for each task, we explicitly extracted task-relevant technical details, such as: *baseline LLM(s)*, *LLM enhancement/adaptation paradigm(s)* (e.g., ICL, retrieval grounding/RAG, fine-tuning, tool/agent augmentation, verification), and *evaluation metric(s)*. The complete paper list and extracted metadata are available in our public repository.

2.2 Corpus summary and categorization.

Based on our task-driven screening, we reviewed 162 task-paper records spanning seven log-analysis tasks, published between 2020 and 2025. The literature search was completed in November 2025. Deduplicating these records by paper identity yields 145 unique papers. He et al. [47] reported 158 log-analysis papers from 1997 to 2020 under a different collection protocol. Although the totals are not directly comparable, the 145 papers collected here over 2020–2025 show the rapid growth of LLM-based log analysis. Figure 3 shows the temporal trend and task composition of the full reviewed set.

The task distribution (Fig. 3, right) is highly skewed. Anomaly Detection and Log Parsing together account for roughly *two-thirds* of all task-paper records, reflecting the early use of LLMs to identify suspicious behavior in noisy, drifting logs and to structure raw logs for later analysis. At the same time, the corpus also shows a clear expansion toward more downstream, operator-facing tasks. In particular, Root Cause Analysis forms a sizable slice and grows rapidly in the most recent years, indicating a shift from “detecting something is wrong” toward “explaining why and what to do,” often requiring richer reasoning and evidence integration.

Temporally, the corpus spans 2020–2025 and exhibits a pronounced surge in recent years (Fig. 3, left). Only a small number of papers appeared before 2023, followed by a sharp jump in 2024 and an even larger volume in 2025. Beyond the overall growth, the per-task evolution is also informative: anomaly detection appears earliest and grows steadily, while several downstream tasks emerge later and accelerate quickly (e.g., RCA and log summarization are largely concentrated in the most recent period). This pattern suggests that the community first adopted LLMs where they could be plugged into existing pipelines (detection/parsing) and then progressively pushed them toward higher-level operational assistance as models and prompting/retrieval techniques matured.

Finally, the venue landscape indicates both broad dissemination and strong SE anchoring. As summarized in Table 2, a noticeable fraction of the corpus appears as preprints (reflecting the fast-moving nature of LLM research), while many peer-reviewed papers are mainly published in software engineering venues. Interestingly, the venue concentration differs by task: papers on anomaly detection are much more widely distributed across research communities and venues (e.g., systems, security, networking, data mining, and AI), whereas topics closer to software engineering practice (e.g., logging and some parsing work) are more likely to appear in SE-focused outlets. This cross-venue spread reinforces that LLM4Log is inherently interdisciplinary: it inherits operational problems from SE while drawing methods and evaluation norms from multiple adjacent fields.

Table 2. Top venues by number of papers in our LLM4Log corpus.

Venue	#Papers
arXiv	29
ICSE	11
ASE	7
ISSRE	7
FSE	6
CIKM	5
ICLR	5
EMSE	4
TOSEM	4
ICPC	3

Note: We report the top 10 venues in the paper. The full list is available in our online repository.

2.3 Relation to existing surveys.

In recent years, several surveys and reviews have summarized progress in automated log analysis and related LLM-based operational analytics. He et al. [47] offered a broad survey of automated log analysis for reliability engineering, summarizing established pipelines spanning logging practice, compression, parsing, anomaly detection, failure prediction, and diagnosis, and largely reflecting the pre-instruction-tuned-LLM era. As LLMs entered log analytics, more focused surveys emerged. Beck et al. [6] concentrated on *system log parsing with large language models*, emphasizing how LLMs convert raw logs into structured templates and fields and how such parsers are evaluated. Akhtar et al. [1] reviewed *LLM-based event log analysis* in a broader sense, motivated primarily by the needs of security professionals and covering event logs from multiple computing contexts, including system, application, and network logs. Their survey mainly summarizes what LLM techniques have been applied to event logs, while our work uses software runtime logs as the organizing lens for task structure, evaluation practice, and open challenges.

Several adjacent surveys also provide useful comparison points without being log-centric in the same way as our work. Remil et al. [136] reviewed AIOps solutions for incident management across tasks, data sources, and technical approaches. Zhou and Fokaefs [216] surveyed AI assistants for the incident lifecycle in microservice environments, considering observability data such as logs, traces, and metrics. Wang and Qi [174] surveyed root cause analysis in (micro)services across logs, traces, metrics, and multimodal data. From the operational analytics side, Zhang et al. [200] discussed AIOps in the LLM era at a broader reliability-engineering level, where logs are one type of operational data among metrics, traces, alerts, incidents, and other signals.

Overall, our survey complements these efforts by using software and system runtime logs as the organizing evidence across the end-to-end workflow. For downstream tasks, we apply the same three-case policy: studies are

Table 3. Positioning of this survey relative to representative prior surveys.

Survey	Focus	LLM	Difference from our survey
He et al. [47]	Broad log analysis before the recent LLM wave	No	Updates the field for the LLM era with a dedicated log-centric focus
Beck et al. [6]	LLM-based system log parsing	Yes	Extends beyond parsing to the full log-analysis workflow
Akhtar et al. [1]	LLM-based event-log analysis across security and computing contexts	Yes	Mainly summarizes what LLM techniques exist for event-log analysis. Our survey focuses on how LLM4Log is structured, evaluated, and where open challenges remain
Remil et al. [136]	AIOps incident-management review	No	Broader incident-management scope without a dedicated LLM-based log-centric analysis
Zhou and Fokaefs [216]	AI assistants for incident lifecycle in microservices	No	Incident-lifecycle support across logs, traces, and metrics outside a log-centric workflow
Wang and Qi [174]	RCA survey in (micro)services	No	Focuses on one downstream task and broader observability signals, not the full LLM4Log workflow
Zhang et al. [200]	LLM-enabled AIOps and reliability engineering	Yes	Covers broader LLM4AIOps workflows across logs, metrics, traces, alerts, and incidents. Our survey focuses on how LLM methods are adapted and evaluated when runtime logs are the central evidence source

within the log-specific scope when runtime logs are the main evaluated input or an explicit part of multimodal evidence. When log use is optional, only separable results that explicitly involve runtime logs support log-specific findings. Studies without separable log-specific results may be discussed as related comparisons, but they do not support findings about log analysis. As summarized in Table 3, the survey spans logging, parsing, and downstream analysis while examining how LLM-based methods interact with deployment constraints and evidence quality.

2.4 Threats to validity.

Our survey collection methodology faces several limitations. First, the flagship-venue-first search strategy improves precision and anchors the survey in core software engineering venues, but it may still miss some interdisciplinary, industry-facing, or rapidly emerging work despite the subsequent snowballing and digital-library cross-checking. Second, the search process depends on keyword design, so relevant papers that use uncommon terminology may be harder to retrieve in the initial stages. Third, deciding whether runtime logs are explicitly evaluated inputs still requires manual judgment because terms such as “log,” “event,” “trace,” and “incident record” are not used consistently across papers. We reduce this ambiguity by applying the definition above and reporting evaluated input types. Because our retrieval strategy was centered on log-related terms, the related comparisons provide context but not a complete review of non-log or modality-agnostic LLM4AIOps research.

Cross-paper evaluation remains a threat to validity. The studies covered in this survey differ in their datasets, label definitions, model families, prompts, decoding settings, and evaluation metrics. In addition, many papers do not report enough prompt or inference details to support exact reproduction. For LLM-based methods, results may also vary across runs because of stochastic decoding, model updates, retrieval choices, or context selection. These differences make it difficult to compare reported performance numbers directly across papers. For this reason, our main conclusions focus on recurring design patterns, common evaluation gaps, and practical constraints in LLM4Log research. We treat claims about the best-performing method as exploratory.

3 LLM Background & Taxonomy for Log Analysis

LLMs have become a practical tool for log-centric software operations because logs are semi-structured, drift over time, preserve execution chronology, and are typically interpreted alongside heterogeneous artifacts (tickets, dashboards, traces, configurations, and code) [40, 72, 103, 108]. This section explains why logs form a distinct setting for LLM-based analysis, how major model families map to log-analysis workloads, and how adaptation paradigms interact with operational constraints such as drift, grounding, privacy, latency, and cost.

3.1 Why LLMs are a Good Fit for Log Tasks

Logs as semi-structured text that bridges natural language and code. A key reason LLMs fit log analysis is that log messages sit between natural language and code. A typical log line mixes free-form descriptions (symptoms, actions, causal hints) with code-like structure and identifiers (function and class names, stack frames, paths, IPs, config keys, request IDs, error codes) [62, 74]. This hybrid format aligns with two core LLM strengths: natural-language understanding and code-aware pattern modeling [24, 108]. At the same time, log analysis is not just generic text understanding: the boundary between stable templates and variable values is often ambiguous, and the same token may be a symptom, parameter, component name, or incident clue depending on context. As a result, LLMs can capture intent and semantics that are often discarded when logs are reduced to templates/event IDs, and can still remain robust when token vocabularies evolve (e.g., new components, identifiers, or new error strings) [64, 103].

Latent semantics beyond templates and surface patterns. Although many classic pipelines reduce logs to templates/event IDs, real log messages encode operational meaning: what operation was attempted, what failed, which component is implicated, how severe it is, and what the system was doing right before/after [72, 74]. These signals are frequently expressed implicitly via linguistic cues (error idioms, temporal markers, causal connectors, negation), code-like cues (exception types, API names, call sites, module prefixes), and temporal ordering across neighboring events [40, 103]. This makes logs different from many standalone text artifacts: they often carry a symptom chronology that operators use to reconstruct incidents. LLMs can exploit these cues to support interpretation, explanation, and cross-source integration, especially when purely symbolic representations are brittle or incomplete [42, 68, 98].

Robustness under drift, partial observability, and long-tail behaviors. Logs evolve with versions, configuration changes, feature flags, and deployment environments. Meanwhile, rare events are common in operations and often carry disproportionate diagnostic value, even when they appear too infrequently for stable supervised learning [64, 103]. Compared with fixed-vocabulary or rule-based parser-dependent pipelines, LLM-based representations and prompting enable soft matching and semantic generalization, which helps when (i) rule-based parsers fail or drift, (ii) labeled anomalies/failures are scarce, or (iii) the same underlying issue is described with heterogeneous phrasing across services and versions [4, 103, 188].

Unifying heterogeneous evidence into a single reasoning space. In production triage, the relevant evidence rarely lives in logs alone [20, 42]. Engineers correlate logs with traces/metrics, tickets, runbooks, configurations, and sometimes code diffs [20, 42]. LLMs can condition on mixed inputs (log snippets, retrieved historical cases, metric summaries, runbook steps, code/config fragments) and produce operator-oriented outputs such as hypotheses, ranked causes, incident narratives, and action candidates [42, 68, 98]. This ability to work across heterogeneous artifacts matches the reality of AIOps workflows, but a log-centric view remains necessary because logs often provide the chronological symptom evidence around which other signals are interpreted [42, 68].

Human-facing outputs and decision support. Many operational tasks require a narrative (what happened, what changed, why likely) and not just a score/label [42, 68]. LLMs can highlight evidence, translate technical fragments into concise explanations, and generate structured artifacts for handoff (e.g., timelines, suspected

components, extracted entities) [20, 98, 100]. However, free-form generation increases the risk of hallucinated rationales or overconfident claims [20, 68]. In practice, LLMs are most reliable when outputs are constrained and augmented (e.g., by quoting evidence spans, using structured extraction, or tying claims to retrieved/tool-produced facts) rather than relying on unconstrained storytelling [42, 98, 100].

3.2 Evolution of LLMs: Architectures and Training Stages

Transformer backbones for long-context log modeling. Most LLM-based log-analysis methods build on Transformer architectures [170], whose self-attention mechanism supports scalable contextual modeling across long input windows. This matters for logs because useful cues can be far apart (e.g., an early warning followed by a later error burst), and multi-line context often changes the meaning of individual events [170].

Encoder-only models: masked language modeling and contextual embeddings. Encoder-only pretraining with masked language modeling (MLM), popularized by BERT [24] and its variants (e.g., RoBERTa [95], ELECTRA [17]), learns bidirectional contextual representations. Compared with decoder-only chat-style LLMs, encoder-only models typically have fewer parameters and lower inference cost, and some log-analysis papers therefore refer to them as *pretrained language models (PLMs)*, not LLMs. However, since they are still substantially larger than traditional log models and have a large body of log-centric research, we treat them as part of the broader LLM family in this survey [130]. In log analytics, encoder-only LMs are most useful as efficient representation and scoring backbones [24, 40]. Their pretraining objectives can support semi- or unsupervised anomaly signals, while their embeddings support retrieval, clustering, and lightweight downstream heads for anomaly/failure classification when some supervision exists [17, 24, 40, 69, 135].

Encoder-decoder models: conditional generation and structured transformation. Encoder-decoder Transformers generalize from representation learning to conditional generation: an encoder ingests the input sequence and a decoder produces an output sequence [160, 170]. Representative families include T5 [134] and BART [78], which are typically trained with denoising or text-to-text objectives [78, 134, 171]. For logs, this paradigm is most relevant when the task transforms one textual form into another: incident summarization, noisy-message normalization, structured extraction, or template-like rewriting conditioned on surrounding context [78, 134, 142].

Decoder-only models: autoregressive pretraining and general-purpose assistance. Decoder-only autoregressive LMs dominate interactive settings because they are flexible under prompting and can be scaled to strong instruction-following behavior [11, 66]. Representative examples include the GPT family [11] and open-weight decoder models such as LLaMA-style backbones [167]. For log analysis, they are attractive when workflows require open-ended reasoning, hypothesis generation, retrieval-augmented triage, remediation suggestions, or multi-turn operator interaction, especially when labeled data is scarce [120, 180]. The trade-off is higher inference cost and latency, which motivates selective escalation and hybrid designs discussed later [149].

Key capabilities that matter for log tasks. Across architectures, several capabilities are particularly relevant to log analysis [10, 79, 116, 120, 181, 194]: **(1) semantic normalization and abstraction**—mapping heterogeneous phrasings to consistent meanings and grouping variants of the same issue. **(2) contextual disambiguation**—interpreting the same token differently depending on surrounding components, timestamps, and execution context (crucial under drift). **(3) evidence integration over long contexts**—integrating signals across many lines and multiple sources (logs, traces, metrics, tickets), typically mediated by windowing or retrieval in practice. **(4) controllable outputs**—producing operator-facing artifacts (ranked lists, extracted entities, timelines) instead of only free-form text. **(5) reasoning and decomposition**—breaking complex tasks into steps (form hypotheses, retrieve context, verify, refine), which underpins agentic log analysis pipelines. **(6) software**

and code awareness—understanding stack traces, API names, configuration keys, and common failure modes, enabling more actionable interpretation than purely surface-level text matching [13].

3.3 LLM Capability and Specialization Taxonomy for Log Analytics

LLMs used in log analysis can be grouped by specialization level, which often determines where they fit in the log pipeline [10].

General-purpose LLMs are trained broadly on large-scale, diverse text corpora and excel at natural-language interpretation, summarization, and interactive assistance [11, 116]. They are useful for prompt-based log understanding, RCA narratives, and incident reporting, but usually need retrieved context to handle organization-specific terminology, component names, and operational conventions [43, 79, 116, 120].

Code-oriented LLMs are trained or fine-tuned heavily on code and technical corpora, improving performance on stack traces, error messages, configuration files, and program-context reasoning [13, 33, 178]. They are a better fit when log interpretation depends on linking symptoms to code/config, call stacks, APIs, or remediation commands [13].

Domain-tailored LLMs adapt a backbone to a specific operational domain (e.g., AIOps for cloud platforms, network diagnostics, ICS security) via continued pretraining, instruction tuning, or explicit knowledge integration [43, 120]. They are most valuable when logs contain specialized vocabularies, recurring incident patterns, and internal operational procedures that general-purpose models cannot infer reliably [43].

3.4 Adaptation and Enhancement Paradigms

Off-the-shelf LLMs are often not trained *specifically* for log analysis: they rarely see an organization’s proprietary log vocabulary, evolving component taxonomy, or operational ground-truth data (incidents, runbooks, and on-call decisions) [10, 43]. Therefore, LLM-based log analysis approaches typically improve reliability by controlling *what* the model sees (context and retrieved evidence) and *how* it decides (prompting, reasoning scaffolds, tuning, tools, and verification) [79, 120, 181, 194]. Below, we summarize the most common paradigms in terms of the log-specific trade-offs they address: label scarcity, drift, grounding, controllability, inference cost, and deployment practicality [10].

Prompting and in-context learning (ICL), including retrieval grounding. ICL treats the LLM as a training-free learner: prompts provide task instructions, output constraints, and a small set of demonstrations [11, 91, 113]. In log settings, effective ICL is rarely a single “classify this log” instruction. It typically includes: (i) **input normalization** (variable masking, deduplication, component/time grouping), (ii) **bounded evidence** (top-*k* suspicious lines/windows rather than full streams), (iii) **schema-constrained outputs** (labels, ranked candidates, JSON fields), and (iv) **system-specific exemplars** (a few representative normal/failure patterns from the same environment) [91, 113].

A common extension is **retrieval-augmented generation (RAG)**, which retrieves relevant context (e.g., similar historical windows/incidents, runbook passages, known signatures, or indexed summaries) and injects it into the prompt as additional demonstrations or evidence [79]. In many log-analysis pipelines, RAG can be viewed as an *ICL enabler* rather than a separate paradigm: retrieval supplies high-quality in-context examples and system knowledge at inference time, without changing model weights [79]. This “retrieve → prompt → decide” pattern improves both *grounding* (anchoring outputs to concrete evidence) and *adaptation* (bringing system-specific knowledge under drift) [43, 79].

ICL remains attractive for rapid deployment and heterogeneous tasks, but it is sensitive to prompt quality, context selection, and distribution shift. As a result, many practical approaches combine RAG-augmented ICL with lightweight filtering (to reduce prompt noise) and downstream verification (to prevent overconfident

hallucinations) [91, 113]. **Reasoning scaffolds: CoT, ToT, and structured decomposition.** Many log tasks (especially diagnosis and remediation) benefit from explicit decomposition rather than one-shot generation [181, 217]. **Chain-of-Thought (CoT)** prompting encourages stepwise reasoning (e.g., identify key symptoms → map to components → hypothesize causes → justify with evidence) [181]. In log settings, CoT is most useful when intermediate artifacts are *checkable* and *grounded* (e.g., extracted entities, cited log lines, matched templates), not when it produces purely speculative narratives [176, 181]. **Tree-of-Thought (ToT)** generalizes CoT by exploring multiple candidate hypotheses/paths, scoring or pruning them, and selecting the best [176, 195]. This matches operational diagnosis where multiple plausible causes exist. ToT-style search can be implemented via self-consistency sampling, explicit branching (e.g., per service/component), and re-ranking using retrieved evidence or tool outputs [176, 195]. The practical trade-off is cost: ToT multiplies calls/tokens, so it is often reserved for high-severity complex incidents, and paired with retrieval and/or a smaller model to narrow the search space [195]. More generally, many approaches adopt **structured decomposition** without explicitly naming CoT/ToT by splitting the workflow into stages such as symptom extraction, candidate generation, evidence retrieval, and final decision [194, 217].

Fine-tuning and post-training for log tasks. When data is available, training often yields better stability than pure prompting [43]. It is useful to distinguish: (i) **continued pretraining** (domain-adaptive pretraining on large unlabeled log corpora) to improve coverage of log vocabulary and style, (ii) **supervised fine-tuning (SFT)** on labeled log tasks, (iii) **parameter-efficient tuning (PEFT)** (e.g., LoRA/adapters) to reduce cost and enable more frequent updates under drift, and (iv) **post-training/alignment** (instruction tuning and preference optimization) to improve instruction following, format adherence, and safety/harmlessness behaviors [120, 133]. In log analysis, post-training is especially relevant when the model must (a) reliably follow output schemas, (b) avoid leaking sensitive data in summaries, and (c) produce calibrated, non-overconfident explanations (including the ability to abstain) [120]. A common pattern is **hybrid training**: fine-tune a smaller encoder/LM for scoring or classification while using a larger aligned model for explanation and interactive assistance [10, 116]. Another pattern is **task-to-instruction transformation**, where multiple log tasks are cast into a unified instruction format so one model can support classification, extraction, and generation consistently [120].

Tool augmentation and agentic workflows. Agentic log-analysis approaches extend in-context retrieval by letting an LLM iteratively call external tools to gather, filter, and validate evidence [111, 140, 194]. Typical tools include log search/index queries, metric/trace dashboards, ticket/KB/runbook lookup, config/code retrieval, and lightweight parsers or summarizers that pre-compress high-volume inputs [111]. This paradigm is a natural fit for real-world log analysis because a single prompt window is rarely sufficient: relevant signals may be scattered across long time spans, multiple components, and heterogeneous repositories [111, 194].

Agentic workflows commonly follow a loop of: (i) **planning** (what evidence to obtain next and which hypotheses to test), (ii) **acting** (tool calls for retrieval, slicing, aggregation, or correlation), (iii) **observing** (interpreting tool outputs and updating the working context), and (iv) **deciding** (producing a final prediction/explanation/-summary with evidence pointers) [194]. The benefits are stronger grounding and broader coverage. The costs are latency, more failure modes (tool errors, cascading mistakes), and higher security risks (e.g., prompt injection via retrieved documents) [111, 140]. Accordingly, practical deployments often add guardrails such as allow-listed tools, bounded iterations, schema validation, and conservative fallback behaviors [111, 148].

Verification, calibration, and controllability. Because log analysis is high-stakes and noisy, many approaches add explicit reliability layers: **self-consistency** (sample multiple reasoning paths and aggregate), **cross-checking** (verify claims against retrieved lines/windows or tool outputs), **schema validation** (force typed/JSON outputs and reject invalid responses), **evidence citation** (require each claim to point to a log span or retrieved document), and **selective generation** (generate only after candidate filtering by a smaller model) [63, 65, 105, 176]. These

paradigms reappear throughout the task sections. Section 7 later compares their trade-offs in terms of supervision needs, inference cost, robustness under drift, controllability, and operational fit.

3.5 Practical Concerns: Privacy, Security, Compliance, and Efficiency

Logs may contain sensitive identifiers (user IDs, IPs, hostnames), infrastructure topology, security indicators, and occasionally personal data [31, 70]. As a result, deploying LLMs for log analysis is often governed as much by **privacy/security constraints** as by model accuracy, while **latency and cost** remain first-class operational requirements [31, 34]. These concerns are widely recognized in the literature, but rigorous production-scale validation remains limited and uneven across tasks.

Commercial LLMs vs. self-hosted LLMs. Commercial LLMs are typically consumed via *remote APIs*, meaning prompts (and any retrieved context inserted into them) must be transmitted to an external model provider [31, 34]. This increases **data exposure risk** and reduces end-to-end controllability: even with contractual terms and vendor assurances, organizations may have limited visibility into provider-side handling (e.g., retention, secondary processing, or cross-region routing) and must treat the prompt as a potentially sensitive payload [12, 31]. In contrast, self-hosted (open-source) models keep inference *within the organization's trust boundary*, making data handling more controllable: access can be restricted via internal IAM, prompts/outputs can be logged or disabled under local policy, and compliance requirements (e.g., residency) can be enforced operationally [34]. However, self-hosting shifts responsibility to the operator: the main risks become the local security posture, including securing model-serving endpoints, hardening the retrieval stack, and protecting stored prompts, retrieved documents, and generated outputs [34].

Tool-augmented and retrieval-based workflows amplify these issues regardless of deployment. Untrusted retrieved text can introduce prompt-injection style instructions, and generated summaries/explanations can inadvertently echo secrets or sensitive identifiers if not constrained [12, 38].

Common mitigations in log analysis pipelines. Typical mitigations include: (i) **minimization** (send only the smallest relevant window and avoid full dumps), (ii) **redaction/masking** (identifiers, secrets, tokens), (iii) **bounded retrieval disclosure** (retrieve on-prem, but pass only small snippets/fields), (iv) **RBAC and auditing** (who can query what and how prompts/outputs are safely logged), and (v) **policy-constrained outputs** (no secret echoing, structured extraction, and citation requirements) [31, 34, 70]. These controls interact with utility: aggressive redaction may remove key signals, so many approaches combine masking with structured extraction and retrieval to preserve diagnostic content while limiting exposure [31, 34].

Cost drivers and practical optimizations. The main cost drivers are context length (tokens), number of model calls (multi-pass/agent workflows), and retrieval/embedding computation [23, 126]. Common optimizations include windowing and deduplication, template compaction, hierarchical summarization, caching embeddings and intermediate summaries, batching, and **selective escalation** (invoke a large model only after a cheaper filter flags candidates) [111]. For self-hosted models, quantization and efficient serving reduce inference cost. For API models, prompt compression and retrieval-first designs reduce token budgets [23, 126].

Hybrid designs as a practical deployment pattern. A frequent design is **small model for scoring/filtering** (ranking suspicious windows, candidate component selection, coarse labeling) plus **large model for explanation and summarization** [10, 116]. This keeps expensive reasoning focused on a small subset of high-value context, reduces latency, and yields operator-facing artifacts (summaries, rationales, action candidates) without incurring LLM costs on every log line [111, 126]. In practice, such hybrids also simplify governance: only the most capable (and often most sensitive/costly) components are exposed to the most sensitive or costly data [31, 34].

4 Logging Statement Generation and Maintenance

Logging statement generation studies how to automatically insert and update log statements in source code so that systems emit informative runtime records for debugging and monitoring. In practice, insufficient logging can hinder incident triage and reduce observability, whereas excessive or inconsistent logging introduces noise, performance overhead, and scalability concerns. Hence, one active line of research is automated logging statement generation, which aims to recommend or synthesize high-quality log statements so that developers can obtain effective logs with less manual effort.

4.1 Task Definition

Logging statement generation is the task of inserting and updating log statements into source code so that, during execution, the system emits textual records (logs) describing internal states and runtime events. Given a code context (e.g., a method, a basic block, or a file), automated logging must decide **where** to place logs, **what** to record, and **how** to phrase messages in a readable and maintainable way. A log statement is a hybrid artifact that combines a code-level logging API call (e.g., `logger.info(...)`) with a concise natural-language message and selected program variables. Effective generation, therefore, requires aligning program behavior with project-specific conventions, such as naming, verbosity, and severity-level usage, typically implemented through standard logging frameworks. This makes automated logging not only a code-generation problem, but also an observability-design problem shaped by developer intent, code context, message readability, and runtime diagnostic value [36, 87, 139, 219].

```

1 public User getUser(String userId) {
2     logger.info("Fetching user with id={}", userId);
3     try {
4         User user = userRepository.find(userId);
5         if (user == null) {
6             logger.warn("User not found: id={}", userId);
7         }
8         return user;
9     } catch (DatabaseException ex) {
10        logger.error("Failed to fetch user: id={}, error={}", userId, ex.getMessage());
11        throw ex;
12    }
13 }

```

Listing 1. A simplified example of single-line logging in Java

Listing 1 illustrates a simplified Java method instrumented with three log statements, and Listing 2 shows the corresponding runtime logs. The method logs an INFO message before accessing the repository, emits a WARN when the requested user is missing, and records an ERROR upon a database exception. Each statement pairs a concise event description with a small set of diagnostic variables (e.g., the user identifier and exception message), enabling developers to reconstruct key control-flow decisions and failure context from the log stream.

```

1 INFO 2025-05-12 10:15:03 UserService - Fetching user with id=U1024
2 WARN 2025-05-12 10:15:03 UserService - User not found: id=U1024
3 ERROR 2025-05-12 10:15:04 UserService - Failed to fetch user: id=U1024, error=connection timeout

```

Listing 2. Example single-line runtime logs emitted by Listing 1

In practice, logs are commonly placed at execution milestones, boundary interactions with external components, and exception-handling paths, where recorded context is most useful for diagnosis [36, 219]. Most systems rely on standard logging frameworks that provide unified APIs and configurable formatting. In Java, widely used options include `java.util.logging` [117], `Log4j 2` [3], `SLF4J` [132], and `Logback` [131], while Python typically uses the built-in logging module [128], sometimes complemented by `structlog` [141] or `loguru` [22]. Accordingly, automated logging techniques generally assume these APIs and aim to generate statements that can be directly inserted and compiled in real projects.

4.2 Challenges

Although logging is essential for observability, producing high-quality logs in practice is non-trivial. Prior automated approaches, typically based on heuristics, static analysis, and traditional machine-learning models, have therefore faced persistent limitations. Existing studies consistently suggest that logging usefulness is largely determined by three intertwined decisions: where to log, what to log, and how to keep logs effective over time [36, 84, 214, 219].

4.2.1 Where-to-Log: Placement and Coverage. A core difficulty of logging is deciding **where** a statement should be inserted. Logs placed too sparsely may miss critical runtime evidence, reducing diagnosability, whereas overly dense logging introduces noise, log flooding, and runtime overhead [8, 36, 84, 219]. In real systems, diagnostically important events often occur along complex, low-frequency control-flow paths (e.g., rare branches, retries, and exception chains), making it difficult to identify insertion points that reliably capture true execution states [36, 83, 219].

Traditional automated techniques often suggest logging-statement placement using local syntactic cues (e.g., method entry/exit, catch blocks, API calls) or handcrafted rules over AST patterns [108, 185, 187, 219]. Such signals are inexpensive and scalable, but they are weak proxies for diagnostic value: they may over-instrument common patterns while missing semantically important events that span multiple statements or methods [82, 162]. Static-analysis-based methods can improve precision by tracking control flow and exceptional paths, yet they still struggle to decide which program points are worth logging from a human-debugging perspective, and they often produce redundant locations when applied at scale [25, 36, 83]. Moreover, placement decisions made in isolation are difficult to reconcile globally: a log that appears useful within one method may be inconsistent or redundant when viewed across the system, especially in large-scale or distributed applications [36, 83, 215, 219].

4.2.2 What-to-Log: Level, Message, and Variables. Even if a location is chosen, determining **what** to record remains challenging. First, log levels encode severity and expected frequency, but these levels are often ambiguous in real projects. The same event may be viewed as INFO in one module and WARN in another, leading to inconsistency and unreliable filtering during incident response [49, 86, 219]. Second, messages must convey clear event semantics, yet developers frequently struggle to express intent concisely, avoid vague wording, and align with project terminology [87, 219]. Third, selecting which variables to include is subtle: missing key variables reduces diagnostic value, whereas logging too many values increases verbosity [25, 83, 219]. These issues are exacerbated by the need to maintain a consistent style across contributors, components, and versions.

Prior approaches typically decompose this decision into separate subtasks, such as level prediction and variable selection, using local features (identifiers, types, nearby API calls, shallow data-flow signals) and then fill messages with templates [82, 86, 219]. This decomposition is convenient for modeling, but it often yields incoherent outcomes, for example, a predicted level that does not match the message phrasing or variables that are only weakly related to the event being described. Template- and pattern-based message generation further struggles to express nuanced intent (e.g., distinguishing a transient network hiccup from a persistent configuration error), producing rigid or generic strings that provide limited diagnostic cues. Overall, the fundamental difficulty is that level, variables, and message are semantically coupled decisions, whereas many traditional techniques treat them as largely independent.

4.2.3 How-to-Log: Quality, Maintainability, and Evolution. Logging quality is not only about writing a single statement correctly, but also about keeping logs useful throughout software evolution. As code and requirements change, logs can become stale, misleading, or incorrect (e.g., referring to outdated conditions, variables, or execution phases), yet they are rarely updated with the same rigor as functional code [59, 175, 214]. In addition, practical logging must respect non-functional constraints, such as performance overhead, storage budget, and

Table 4. Summary of LLM-based automated logging studies organized by task category.

Paper	Task	Paradigm	Input	Output	Context/Technique	Base Model(s)	Comparisons	Metric(s)
Full Logging Generation (General Purpose: Location, Message, Variables)								
LANCE [108]	Full Gen	Fine-tuning	Java Method	Log Stmt	-	T5	-	Acc, BLEU
FastLog [185]	Full Gen	Fine-tuning	Method + AST	Insert Point	AST Locator	PLBART	-	Acc, BLEU
LEONID [109]	Multi-Log	Fine-tuning	Full Method	0-N Logs	-	T5	-	BLEU, METEOR
ELogger [37]	Block Gen	Fine-tuning	Code Block	Single Log	Block Features	Enc-Dec	-	-
UniLog [187]	Full Gen	ICL	Method	Loc+Level+Msg	Warm-up Demos	Codex, GPT-3	-	PA, LA, MA
SCLogger [83]	Full Gen	ICL + CoT	Method	Log Stmts	Call Graph (2-hop)	GPT-4, LLaMA2	GPT-3.5	PA, AOD, F1
PDLogger [25]	Full Gen	ICL + CoT	Method + Slice	Logs	Backward Slicing	o3-mini, DeepSeek	LLaMA3-70B	PA, L-ACC
AUCAD [199]	Align Gen	Fine-tuning	Method + Issue	Logs	Issue Alignment	LLaMA3.1	Magocoder	PA, LA, VF1
Zhong et al. [215]	Full Gen	Fine-tuning	Java Method	Logs	Compact Model FT	LLaMA-8B, Mistral	GPT-4o, Claude	PA, LA, F1
Specialized Logging Generation (Test Code, File-level, Configuration)								
File-Level [138]	File Gen	ICL	Python File	Multi Logs	Pipeline Stages	GPT-4o-mini	-	L-ACC, AOD
Test-Code [150]	Test Log	ICL	Test Method	Assert/Log	Test Intent	GPT-3.5, CodeLlama	GPT-4o, Llama3	Acc, BLEU
ConfLogger [146]	Config Log	ICL + Rules	Config Code	Logs	Config Analysis	GPT-4o	-	LA, AOD
Benchmarks, Empirical Studies, and Visions								
LogBench [82]	Study	ICL / Zero	Method	Level+Msg	-	GPT-3.5/4	InCoder, StarCoder	BLEU, ROUGE
AL-Bench [162]	Benchmark	-	Java Code	Logs	Benchmark Suite	-	-	ALD, DEA
Auto-Logging [8]	Vision	-	-	-	AI Instrumentation	-	-	-
Specific Log Tasks (Level Prediction, Defect Detection, Repair, Quality)								
Heng et al. [49]	Level Pred	FT / Zero	Snippet	Level	-	CodeLlama-13B	BERT, RoBERTa	Acc, AUC
OmnLLP [119]	Level Pred	RAG + ICL	Log + Neighbors	Level	Cluster Retrieval	CodeXEmbed	-	ARI, AUC
LogUpdater [214]	Log Repair	Agent	Defect Log	Fixed Log	Defect Taxonomy	CodeT5+, GPT-4o	Claude3.5	BLEU, ROUGE
Defects4Log [175]	Detection	ICL + CoT	Method + Log	Defect Type	Defect Patterns	DeepSeek-R1	GPT-4o	Precision, Recall
LOGIMPROVER [84]	Quality	-	Codebase	Improved Logs	Proactive Analysis	-	-	-

privacy/security policies, which further complicate the amount and type of information that can be safely recorded [47, 215].

Traditional tooling provides limited support for maintaining logs over time. Simple consistency checks (e.g., unused variables, formatting rules) can catch surface-level issues, but they do not reliably detect semantic drift, in which the code behavior changes while the log message remains plausible [59, 214]. Likewise, mining- or rule-based detectors often rely on project-specific patterns and are brittle in the face of refactoring, API evolution, or style changes [84, 214, 219]. As a result, projects accumulate redundant, inconsistent, or outdated logs, gradually reducing the overall utility of the logging infrastructure and increasing maintenance costs [84, 214].

4.3 LLM-based Approaches for Logging Statement Generation

Table 4 groups recent LLM-based logging studies by task category, while the discussion below emphasizes the methodological paradigms behind them. Across these paradigms, the key idea is to exploit LLMs as joint models over code and natural language, so that the three core decisions, where to log, what level to use, and how to phrase the message and variables, are no longer made by hand-crafted rules or local classifiers. These logging

decisions emerge from learned semantic patterns and in-context reasoning. Below, we organize prior work by how it uses LLMs and highlight the mechanisms that make each paradigm effective.

4.3.1 Fine-tuned End-to-End Generators. Early work treats logging as a sequence-to-sequence learning problem and fine-tunes encoder-decoder models to transform raw code into logged code. LANCE [108] fine-tunes a T5 model on pairs of Java methods with and without logging, so that the model learns to rewrite an input method into an output method containing an injected log statement. During decoding, the model jointly decides where to insert the statement, which level to use, which variables to log, and how to phrase the message. This joint modeling is precisely where LLMs outperform handcrafted AST patterns: instead of matching rigid templates, the fine-tuned model learns the distribution of logging patterns from large corpora and can adapt to unusual constructs, such as nested exceptions or multi-call sequences, without additional rules.

As the first end-to-end LLM-based automated logging tool, LANCE delivers an innovative proof of concept. Yet, it suffers from three key limitations: (1) rewriting the entire method may inadvertently alter non-logging code, (2) full-method input and output incurs substantial latency compared to local insertion, and (3) it always injects exactly one log statement, thus failing to decide when logging is unnecessary or when multiple logs are required. Subsequent approaches explicitly refine the use of fine-tuning to mitigate these issues. FastLog [185] decouples location from content: a learned module first predicts fine-grained insertion points, and the LLM then generates only the log statement to be inserted. This changes the learning problem from “rewrite the whole method” to “predict a small edit” and thus preserves the original code while reducing decoding cost. LEONID [109] extends LANCE by teaching the model to predict not just the content of a single log, but also whether a method needs logging at all and how many logs to insert. This turns the fixed one-log-per-method assumption into a learned, method-level decision, closer to how developers actually log. ELogger [37] pushes the same idea to block level: it first classifies whether a block warrants logging and only then invokes the generator, reducing over-instrumentation by using the LLM where it matters most. AUCAD [199] further strengthens this fine-tuning line by automatically constructing aligned datasets from log-related issues. The model is trained not just on arbitrary method/log pairs, but on positive and negative examples mined from real bug reports, which sharpens its notion of “good” versus “bad” logging behavior.

Overall, these fine-tuned generators demonstrate that LLMs can implicitly encode common logging idioms and jointly model location, level, message, and variables. Their main trade-off is training and deployment cost: models are tied to specific architectures and datasets, motivating lighter-weight paradigms such as in-context learning.

4.3.2 Prompt-based In-Context Learning. With stronger LLMs, an alternative to task-specific fine-tuning is prompt-based in-context learning (ICL). Compared with fine-tuned models, ICL-based approaches greatly reduce training and infrastructure costs and make it easier to deploy or swap models in practice. UniLog [187] is the first general-purpose framework to systematically exploit ICL for logging. It feeds the target method, along with a small set of (code, log) examples from the same project, and asks a Codex-like model [13] to output the location, level, and message. Mechanistically, UniLog leverages an emergent ability of LLMs: given a handful of exemplars, the model can infer a project’s logging style (e.g., how verbose to be, which variables to mention) and apply that style to new methods. A warm-up strategy stabilizes this behavior by first exposing the model to generic logging patterns before switching to project-specific examples, reducing variance across invocations.

ICL-based methods shift the burden from training to prompting: instead of adjusting millions of parameters, practitioners adjust the prompt. This makes automated logging much easier to deploy or migrate between models, and it directly leverages LLM strengths in style transfer and pattern imitation. File-level ML logging [138] prompts GPT-4o-mini with Python pipeline files and a few annotated examples, letting the model infer where to place logs at pipeline boundaries. Test-code logging [150] supplies test methods and exemplar test logs so that the model learns to describe test intent and failure conditions. ConfLogger [146] combines static analysis to find

configuration-sensitive operations with ICL prompts that show how such operations are typically logged, enabling the LLM to specialize its suggestions for configuration diagnosability.

4.3.3 Execution-Aware Reasoning with CoT and Static Context. While plain ICL uses only the immediate method body, real logging decisions often depend on broader execution context and on a chain of reasoning (e.g., “this call propagates an error from a lower-level component, so it should be logged at ERROR”). To expose this context and reasoning to the LLM, several works enrich prompts with chain-of-thought (CoT) style structure and execution-aware static information.

SCLogger [83] augments the input with two-hop callers and callees on the call graph and organizes the prompt in a CoT fashion. Instead of giving the LLM only the target method, SCLogger explicitly injects the surrounding static context: which functions call this method, what it calls, and how logging is used there. This design exploits a key LLM capability: when given a broader static context, the model can infer variable roles (e.g., which ID identifies a request versus a user), propagate logging conventions across the call chain, and better align with project-specific patterns without extra training. The CoT-style prompt further nudges the model to “think through” why a location is important before emitting a final log statement.

PDLogger [25] makes the execution link even tighter by performing backward slicing from predicted log locations. Data and control dependencies along the slice are turned into explicit prompt tokens, and variables flowing into the logging site are enumerated with their syntactic and semantic roles. In effect, PDLogger transforms implicit runtime behavior (which values influence this point, under which conditions) into an explicit context for the LLM. This allows the model to simulate execution more accurately and to generate multiple log statements per method that are consistent with the underlying control and data flow, instead of relying solely on local patterns.

These CoT- and execution-aware prompts thus serve as a bridge between classical program analysis and LLM reasoning: static analyses compute precise dependencies, and the LLM uses them to perform high-level semantic reasoning and message generation.

4.3.4 Retrieval-Augmented Logging and Subtasks. Some prior research focuses on specific subtasks (e.g., level prediction, message reconstruction) and combines LLMs with retrieval-augmented generation.

For log levels, Heng et al. [49] benchmark a range of models in zero-shot, few-shot, and fine-tuned settings, showing that local code context suffices in many cases but that borderline severity levels remain difficult. OmniLLP [119] takes a retrieval-augmented view: it clusters log statements by semantic similarity and developer ownership, then retrieves representative neighbors to include in the prompt when predicting the level for a new log. Here, retrieval plays to LLM strengths in analogical reasoning: instead of deciding “in the abstract” whether a statement is INFO or WARN, the model compares the target context against a handful of concrete, project-specific examples and chooses the level that best aligns with those patterns. This improves level consistency and disambiguation for borderline cases, effectively grounding the LLM in real project practices.

LogBench [82] investigates the message and level subtasks via a fill-in-the-blank formulation. It constructs paired datasets of original methods (LogBench-O) and transformed-but-functionally-equivalent methods (LogBench-T), removes existing log statements, and asks LLMs to reconstruct the level and message at fixed locations. This approach isolates “what-to-log” from “where-to-log” and assesses whether models rely on brittle syntax or genuine semantic understanding: if performance holds on LogBench-T, where surface syntax has changed, the model is likely relying on deeper semantic cues. By comparing a variety of commercial and open-source LLMs under this setup, LogBench clarifies which architectures and prompting strategies are better at reconstructing realistic log content.

4.3.5 Agentic Pipelines for Log Maintenance. Finally, some work goes beyond generating new logs by using LLM agents in multi-step maintenance pipelines. LogUpdater [214] first uses a classifier (trained on synthesized defective logs) to detect and categorize logging defects, and then invokes an LLM such as CodeT5+ or GPT-4o to

repair the flagged statements. The approach decomposes the task: a lightweight detector narrows down suspicious logs, and the LLM focuses on high-level semantic repair (e.g., aligning message text with code behavior and fixing mismatched variables or temporal descriptions). By separating “where is something wrong?” from “how should it be rewritten?”, LogUpdater reduces the risk of unconstrained LLM edits and leverages the model’s strength in natural-language revision instead of raw anomaly detection.

Defects4Log [175] complements this pipeline view by providing a benchmark and taxonomy for log defects and systematically evaluating LLMs’ ability to classify and reason about them. Together, these efforts illustrate a broader pattern: instead of using LLMs as single-shot generators, logging research increasingly wraps them in agentic workflows—detect, classify, explain, repair—so that each step can be checked, constrained, or retried, and logs can evolve along with the codebase.

In summary, LLM-based approaches for logging differ not only in which tasks they tackle, but also in how they expose code, context, and prior examples to the model. Fine-tuned generators encode logging patterns into model parameters and jointly model location, level, message, and variables, which helps with where-to-log and what-to-log decisions but increases training cost and reduces portability. ICL methods exploit style imitation and adapt quickly to project conventions, but are sensitive to prompt and example quality. CoT, static context, and slicing ground placement and variable choices in control/data-flow evidence, improving semantic grounding at the cost of additional system complexity. Retrieval-augmented subtasks improve consistency for levels and messages through project-specific neighbors, but depend on retrieval quality. Agentic pipelines support how-to-log and maintenance by decomposing detection, classification, explanation, and repair, but introduce more moving parts and validation needs. These mechanisms collectively explain why LLMs can address long-standing challenges in where-to-log, what-to-log, and how-to-maintain-logs that were difficult to handle with hand-crafted rules or traditional ML alone.

Across studies, the key shift introduced by LLMs is not raw generation capability but the ability to treat logging intent as a semantic inference problem instead of a syntactic insertion task. The most effective approaches consistently constrain and inform LLMs using execution-aware context, project-specific exemplars, or multi-step control, instead of relying on unconstrained generation. At the same time, LLMs do not eliminate the ambiguity of logging decisions but relocate difficulty from rule design to context selection, validation, and oversight, pointing toward hybrid, iterative maintenance workflows as a common design pattern.

4.4 Performance Evaluation Metrics

For automated logging, evaluation has gradually shifted from “exact-match-centric” reporting to a more nuanced, component-wise view. Early end-to-end generators often borrowed machine translation-style metrics (e.g., n-gram overlap such as BLEU [122]) because the task was framed as producing a single reference-like statement for each context. However, as LLM-based methods moved to prompt-based generation and multi-log settings, two issues became hard to ignore: (1) logging is inherently under-specified (multiple insertion points or phrasings can be equally reasonable), and (2) surface-form metrics are brittle (semantically correct messages can score poorly due to paraphrasing). As a result, more recent evaluations increasingly combine strict metrics (for comparability) with softer, semantics-aware ones (for faithfulness), such as embedding-based similarity metrics like BERTScore [204].

Below, we summarize the most widely adopted metrics for each aspect and highlight what they capture (and what they miss).

For **logging-statement position**, the de facto metric is **Position Accuracy (PA)**: whether the predicted insertion point matches the reference point at an agreed granularity (often statement/block instead of exact line). PA is easy to interpret and facilitates comparability across papers, so it remains a useful headline number. However, PA becomes less informative once methods are allowed to output multiple logs or “no log” decisions, because the problem is no longer a single-label prediction. In such cases, it is more appropriate to treat placement as a

set prediction problem and report **precision/recall/F1** over predicted logging-statement position. This directly exposes two practically relevant failure modes: over-instrumentation (low precision) and under-instrumentation (low recall).

For **log level**, **Level Accuracy (LA)** (or L-ACC) is the simplest and most commonly reported metric. Yet log levels are not purely categorical in practice: they have an ordinal meaning (e.g., predicting WARN when the reference is ERROR is typically less harmful than predicting DEBUG). Therefore, modern evaluations often complement LA with an ordinal-distance metric, such as **Average Ordinal Distance (AOD)**, to quantify the extent to which predictions deviate along a severity scale. In addition, because log levels can be imbalanced (e.g., INFO dominates), macro-F1 or per-class reporting is often more diagnostic than a single LA number when the goal is to understand behavior on rare but critical levels.

For **log message**, exact-match metrics such as **Message Accuracy (MA)** are increasingly treated as a “strict sanity check” instead of the main quality signal, since natural language admits paraphrases. To maintain backward compatibility, many studies still report overlap-based metrics such as **BLEU/ROUGE**, but these metrics mainly reflect surface similarity and can penalize semantically faithful rephrasing [122]. For LLM-generated messages, it is now common (and more defensible) to additionally report a semantics-aware metric such as **BERTScore**, which compares contextual embedding similarity instead of exact token matches [204]. Finally, edit-distance-style measures (token/character Levenshtein distance) are useful when the paper’s objective includes developer fixing effort: they approximate how much manual editing is needed to correct a generated message.

For **variables**, the field has largely converged on viewing variable logging as a set selection problem. Accordingly, **Variable Precision/Recall/F1** is often more informative than a single “all-or-nothing” accuracy. These metrics separate “missing important variables” (low recall) from “logging too many irrelevant variables” (low precision), and they also interact naturally with hallucination checks (e.g., penalizing variables that do not exist in scope). In addition, if a method jointly generates message and variables, it is worth reporting variable metrics separately from message metrics to avoid conflating semantic description quality with instrumentation completeness. Overall, automated-logging results are not fully comparable across papers because studies differ in dataset construction, insertion granularity, task decomposition (full generation versus subtask prediction), whether multiple valid logs/messages are accepted, programming language and project context, and metric choice.

5 Log Parsing

Modern software systems generate massive volumes of execution logs to support debugging, monitoring, and reliability engineering. However, the scale and velocity of log streams make manual inspection impractical: operators must quickly locate relevant events, correlate symptoms across components, and identify anomalous behaviors under strict time constraints. This motivates automated log analysis, which treats logs as machine-processable data instead of free-form text. A prerequisite for most downstream log analytics is log parsing, the process of converting raw log lines into structured event records [6, 47]. By normalizing heterogeneous messages into reusable templates and extracting runtime parameters, log parsing enables efficient indexing, aggregation, statistical modeling, and cross-run comparison, and thus directly impacts the effectiveness of tasks such as anomaly detection, failure prediction, and root cause analysis. Parsing is also where log-specific LLM challenges become especially visible: the parser must decide ambiguous template/value boundaries, preserve stable event identities under drift, cover long-tail templates, and still operate under streaming-scale constraints.

We review LLM-based log parsing via reusable paradigms and the approaches that make them practical at scale, highlighting how they address the limitations of traditional parsers (format drift, long-tail events) and what new trade-offs they introduce (e.g., context budgets and coordination overhead).

Log messages					
2015-10-18	18:01:51	INFO	main	org.apache.hadoop.http.HttpServer2	Jetty bound to port 62267
2015-10-18	18:01:51	INFO	main	org.apache.hadoop.http.HttpServer2	Jetty bound to port 62258
2015-10-18	18:01:52	INFO	main	org.apache.hadoop.yarn.WebApps	Web app /mapreduce started at 62267
2015-10-18	18:01:53	INFO	main	org.apache.hadoop.app.RMContainerRequestor	nodeBlacklistingEnabled: true
2015-10-18	18:01:53	INFO	IPC Server	org.apache.hadoop.ipc.Server	IPC Server listener on 62270 : starting

Parsed logs					
Date	Time	Level	Process	Component	Log Template
2015-10-18	18:01:52	INFO	main	org.apache.hadoop.http.HttpServer2	Jetty bound to port <*>
2015-10-18	18:01:52	INFO	main	org.apache.hadoop.http.HttpServer2	Jetty bound to port <*>
2015-10-18	18:01:52	INFO	main	org.apache.hadoop.yarn.WebApps	Web app <*> started at <*>
2015-10-18	18:01:53	INFO	main	org.apache.hadoop.app.RMContainerRequestor	nodeBlacklistingEnabled:<*>
2015-10-18	18:01:53	INFO	IPC Server	org.apache.hadoop.ipc.Server	IPC Server listener on <*>: starting

↓ Log Parsing

Fig. 4. An example log parsing result from Hadoop.

5.1 Task Definition

Log parsing is the task that transforms each raw log line into a structured representation that separates **invariant** event semantics from **dynamic** runtime values. Concretely, given a raw log line x , a parser typically outputs (i) a set of **header fields** (e.g., timestamp/date, severity level, process/thread, component), (ii) a **log template** t where dynamic spans are replaced by placeholders (e.g., <*>), and (iii) a **parameter list** v that records the extracted runtime values. At the stream level, especially in online settings where logs must be parsed on-the-fly as they arrive, parsers often maintain a **template inventory** (or event dictionary) and assign an **event ID** to each unique template so that the raw stream can be converted into an event sequence for downstream mining [46, 164, 177].

Figure 4 illustrates this objective with Hadoop logs. For example, raw messages such as “Jetty bound to port 62267” and “Jetty bound to port 62258” differ only in the concrete port number, yet they represent the same underlying event. A correct parser normalizes them to the same template “Jetty bound to port <*>” and extracts the port number as a parameter. Similarly, the message “Web app /mapreduce started at 62267” is parsed into the template “Web app <*> started at <*>”, separating stable keywords (e.g., “Web app”, “started at”) from variable tokens (e.g., the path and port).

In practice, log parsing is rarely a single monolithic step. Most pipelines implement it as a two-stage process. First, a **preprocessing** stage extracts standard fields and isolates the message body. This stage is typically implemented using regular expressions or log format specifications because these header fields are usually emitted in a framework-controlled and relatively stable format (e.g., the timestamp/level/thread/component layout is determined by logging libraries and configuration, and thus changes slowly and predictably). Second, a **message parsing** stage focuses on the remaining free-text-like body and performs template–parameter separation. Unlike headers, the message body is largely developer-written and varies widely across components and versions. It mixes natural-language-like phrases with project-specific tokens and runtime values, and it evolves as code changes. As a result, a single set of regular expressions is rarely sufficient to robustly normalize message bodies without either over-generalizing templates or exploding them into many variants. Across these implementations, the core goal remains the same: to produce stable templates that support consistent grouping while accurately extracting the parameters needed for diagnosis.

5.2 Challenges

Although log parsing has been studied extensively, traditional (non-LLM) parsers face persistent difficulties because logs are semi-structured artifacts produced by evolving software. These difficulties arise from both the *data* (highly variable message styles and continuous drift) and the *operational requirements* (low latency, high throughput, and stable event identities in online pipelines). Below, we summarize four recurring challenges that limit the robustness of traditional parsers.

5.2.1 Structural variability and format drift. Logs exhibit substantial structural variability across systems, components, and logging libraries, and even within the same system, different modules may adopt different writing styles, delimiters, and token conventions. Many traditional parsers therefore embed implicit assumptions about message structure (e.g., token positions, separators, fixed vocabularies, or stable prefix patterns) to make parsing tractable [21, 46]. Such assumptions can work well for the formats they were designed around, but they degrade when encountering unseen layouts, rare formatting choices, or cross-component inconsistencies.

Crucially, this variability is not static: log formats drift over time as code evolves, dependencies are upgraded, configuration changes alter logging layouts, and developers revise wording for readability or diagnostics [64, 104, 198]. Under drift, an initially accurate template inventory can become stale, causing template fragmentation (the same event being split into multiple templates) or erroneous merges (different events being grouped together). Rule-based parsers then require continuous maintenance, while data-driven parsers trained on historical logs can suffer when the distribution of templates or vocabulary shifts.

5.2.2 Ambiguous template–variable boundaries. Even with a stable structure, deciding which tokens are *invariant* versus *variable* is inherently ambiguous. Runtime values have various forms (IDs, UUIDs, paths, IP addresses, error codes), and some values may appear constant within a single deployment but should be treated as variables across runs (e.g., stable ports or hostnames in a fixed environment). Conversely, some numeric tokens or symbols are semantically meaningful constants (e.g., HTTP status classes or well-known error codes) that should remain in templates.

As a result, common heuristics such as “all digits are variables” or delimiter-based splitting can lead to either over-general templates (too many placeholders, losing discriminative semantics) or over-specific templates (template explosion). Both outcomes reduce downstream utility: overgeneralization blurs distinct events, whereas over-specificity fragments event types, harming aggregation and anomaly detection.

5.2.3 Long-tail patterns and coverage gaps. Real log streams often follow a heavy-tailed distribution: a small number of frequent templates dominate the volume. In contrast, rare templates may be crucial for diagnosis (e.g., exception paths and failure-handling branches). Traditional parsers that rely on frequency cues or stable clustering can underperform on the long tail. Rare patterns may be misclassified into frequent templates (degrading precision) or split into many singletons (degrading recall and inventory usefulness), resulting in poor coverage precisely where operators most need structured signals.

This long-tail effect also interacts with drift and boundary ambiguity: rare events provide fewer repetitive examples to stabilize grouping, making them more vulnerable to mis-parsing when wording changes or when variable boundaries are subtle.

5.2.4 Operational constraints and stability requirements. Operational log parsing is commonly performed at a large scale, where millions of log lines must be processed with low latency, high throughput, and predictable resource usage. Traditional parsers are attractive in production usages because they provide deterministic execution and low per-line cost, but achieving *both* high throughput and high accuracy becomes difficult as structural variability and drift increase.

Moreover, many downstream pipelines assume that event identities remain stable across time windows (e.g., for alerting, dashboards, and anomaly detectors trained on historical event sequences) [47, 64]. Template churn caused by drift, boundary ambiguity, or long-tail fragmentation can propagate instability into downstream analytics, degrading both monitoring reliability and cross-run comparability.

5.3 LLM-based Approaches for Log Parsing

Table 5 shows that LLM-based log parsing has expanded from early capability probes into several reusable methodological families. Across these families, the core idea is to trade brittle, log-specific heuristics (e.g., delimiter rules and fixed similarity thresholds) for semantic generalization: LLMs can better infer which spans belong to the stable event skeleton versus instance-specific parameters, even when surface forms vary across environments and versions. We distinguish the main learning/adaptation paradigm from orthogonal reuse and efficiency mechanisms: a parser may be supervised, prompt-based, or retrieval/refinement-oriented, while still using caches, grouping, scheduling, template memories, or tool-based decomposition to reduce cost and stabilize online behavior.

5.3.1 Benchmarks and empirical analyses as the starting point. Early work primarily asked whether general-purpose LLMs can parse logs at all and how sensitive they are to prompting choices. Le & Zhang [73] provides a representative capability analysis of GPT-3.5 (ChatGPT) under zero-/few-shot prompts, showing that seemingly small prompt-format differences can materially change parsing quality. As the field matured, benchmark-style efforts (e.g., LogEval [20]) began to standardize evaluation and make it clearer *which aspect* improves (e.g., grouping versus precise template extraction), thereby enabling more comparable method design.

A key takeaway from follow-up empirical analyses is that *example selection* and *variable handling* repeatedly emerge as high-leverage factors that affect both accuracy and cost (e.g., LLMDPP [173], VISTA [97], and comparative studies [4]). In other words, the “benchmark line” of work did not merely measure performance. It also identified recurring design knobs that later approaches considered.

5.3.2 Supervised and log-specific model adaptation: training for structure and controllability. One family treats log parsing as a supervised learning problem and leverages modern LLMs to learn invariant skeletons and variable boundaries from data, often with greater tolerance for noisy token patterns than purely heuristic rules. A useful way to view this family is *where supervision is placed*: some methods supervise *token-level decisions* (constant vs. variable), while others supervise *template generation* directly. LogPPT [72] is a label-efficient supervised parser. It fine-tunes a RoBERTa model on a small labeled dataset to perform few-shot token-level parameter labeling, reducing reliance on handcrafted parsing rules while remaining grounded in curated examples. Complementarily, LLMParser [103] represents lightweight supervised adaptation via few-shot fine-tuning, enabling a compact open-source LLM to directly generate templates with high accuracy when limited labels are available.

In parallel, log-specific pretraining and domain adaptation aim to internalize “log language” from large corpora, narrowing the gap between natural text and log tokens. PreLog [74] illustrates this direction by pretraining a log-focused model for log analytics tasks (including parsing), providing a stronger initialization than generic LLMs for downstream log understanding. Overall, supervised paradigms tend to provide more controllable output formats and predictable behavior (often with lower inference-time cost than per-line prompting), but they trade off flexibility and maintenance: they rely on curated labels or domain corpora, and evolving templates may still require re-training or re-alignment to sustain accuracy under drift.

5.3.3 Prompt-based and adaptive ICL parsing: structured generation without parameter updates. A second family avoids parameter updates and instead casts log parsing as a *structured prompting* problem: given a raw message, the model is instructed to emit a normalized template (and optionally the extracted parameters) under a constrained output format. This direction is attractive for rapid deployment because it can work in a zero-shot or a few-shot

Table 5. Summary of LLM-based log parsing approaches. The “LLM Technique” column describes the main learning/adaptation paradigm, while “Reuse / Eff.” records orthogonal scalability, reuse, or system-level efficiency mechanisms.

Paper	Setting	Superv.	LLM Technique	Reuse / Eff.	Base Model(s)	Metric(s)
Benchmarks and empirical analyses						
Le & Zhang [73]	Online	Both	Zero-/few-shot prompting study (capability analysis)	–	GPT-3.5	PA, GA, ED
Astekin et al. [4]	Online	Both	Few-shot prompting comparison across LLMs	–	CodeLlama-7B	ED, PA, LCS
LogEval [20]	Offline	Sup.	Benchmark suite (multiple log-analysis tasks incl. parsing)	–	GPT-4	PA, ED
LogBase [197]	Offline	–	Benchmark dataset for semantic log parsing	–	–	–
LLMDPP [173]	Online	Sup.	DPP-based diverse sample selection for few-shot parsing	–	Flan-T5-small	GA, PA
VISTA [97]	Online	Sup.	Variable-aware ICL (efficiency/cost-oriented study)	Cache	GPT-3.5-Turbo	PA
Supervised and log-specific model adaptation						
GPT-2C [144]	Offline	Sup.	Fine-tuned GPT-style parser (Q&A formulation)	–	GPT-2	F1
LogStamp [164]	Online	Sup.	Sequence labeling for template/parameter tagging	–	BERT-base	RI
LogPPT [72]	Offline	Sup.	Prompt-based few-shot token labeling	–	RoBERTa	PA, GA, ED
LLMParser [103]	Offline	Sup.	Few-shot fine-tuning for template generation	–	LLaMA-7B	PA, GA
Mehrabi et al. [110]	Offline	Sup.	Compact fine-tuning (effectiveness study)	–	Mistral-7B	MLA, ED, F1
OWL [42]	Offline	Sup.	Log foundation model training (IT operations)	–	OWL	RI, F1
PreLog [74]	Online	Sup.	Log foundation model pretraining (log analytics)	–	PreLog-140M	GA, OA, ED, uPA
Le et al. [75]	Online	Sup.	Few-shot tuning of semantic LLMs + adaptive caching	Adap. cache	RoBERTa (125M)	GA, PA, FGA, FTA
LogLM [94]	Offline	Sup.	Instruction tuning for log tasks	–	LLaMA-2-7B	RI, F1
SuperLog [62]	Offline	Sup.	Continual pretraining + interpretable domain knowledge	–	LLaMA-2-7B	RI, F1
Prompt-based and adaptive ICL parsing						
DivLog [188]	Online	Sup.	Adaptive few-shot ICL with prompt enhancement	–	GPT-3 (Curie)	PA, PTA, RTA
Liu et al. [92]	Online	Unsup.	Prompt strategies (fixed demos) for online parsing/-analysis	–	GPT-3.5-Turbo	F1
Xu et al. [212]	Online	Unsup.	Zero-shot prompting for template extraction	Temp. DB	GPT-3.5	PA, GA, ED
LogGenius [196]	Online	Unsup.	Zero-shot prompting + synthetic log generation (assistance)	–	GPT-3.5-Turbo	PA
Zhou et al. [218]	Online	Unsup.	Few-shot prompting (with a BERT-based component in pipeline)	–	GPT-3.5	GA, MLA, ED
LLM-TD [169]	Offline	Unsup.	ICL-based template detection for security event logs	Temp. cand.	OpenChat	FGA
Lemur [207]	Online	Unsup.	Entropy-guided clustering + CoT template merging	Cluster	GPT-3.5	Template accuracy
LogRules [57]	Offline	Sup.	ICL enhanced with rules (case-/rule-based knowledge injection)	Rules	GPT-4o-mini + LLaMA-3-8B-Instruct	PA, GA, ED, FGA, FTA
Retrieval/refinement-oriented parsing and system-level efficiency layers						
LILAC [64]	Online	Sup.	ICL + hierarchical demo sampling + cache refinement	Adap. cache	GPT-3.5	PA, GA, FGA, FTA
LogBatcher [184]	Online	Unsup.	Demonstration-free parsing via retrieval (no labeled demos)	Cache	GPT-3.5-Turbo	GA, MLA, ED
LogParser-LLM [213]	Online	Unsup.	Grouping + multi-step prompting (RAG-style)	Grouping	GPT-3.5-Turbo	GA, PA, FGA, FTA, GGD, PGD
SelfLog [123]	Online	Both	Group-wise ICL with self-evolutionary merge tree	Tree	GPT-3.5	GA, PA, PTA, RTA
HELP [186]	Online	Both	Embedding clustering + LLM template generation	Cluster	text-embedding-3-small + Claude-3.5-Sonnet	GA, FGA, PA, FTA
LibreLog [104]	Offline	Unsup.	Retrieval-augmented parsing + self-reflection refinement	Temp. mem.	LLaMA-3-8B-Instruct	PA, GA
Parse-LLM [152]	Online	Unsup.	Agentic decomposition with callable tools (header separation, etc.)	Tools	GPT-4	GA, FGA, PA, FTA
AdaParser [182]	Online	Unsup.	Self-generated ICL + self-correction	Tree	GPT-3.5	GA, FGA, PA, FTA
Huang et al. [54]	Online	Unsup.	Unsupervised parser with retrieval (no labeled examples)	Temp. DB	GPT-3.5	GA, FGA, PA, FTA
EPAS [15]	Online	Unsup.	Asynchronous scheduling of LLM queries (tail-latency aware)	Async	Llama-3.1-70B-Instruct	GA, FGA, PA, FTA
SemanticLog [198]	Online	Unsup.	Variable-aware template generation for large-scale parsing	Cache	LLaMA-2-7B	GA, PA, FTA
Duan et al. [26]	Offline	Unsup.	LogBatcher-style grouping + LLM template correction	Cache	GPT-3.5-Turbo	GA, FGA, PA, FTA
InferLog [177]	Online	Unsup.	ICL-oriented prefix KV-cache reuse + auto-tuning	KV cache	Qwen2.5-14B-Instruct	PA, PTA, RTA, GA

manner without collecting training data. For example, Xu et al. [212] show that even a zero-shot LLM can extract templates online when paired with a lightweight template repository to reuse previously derived results. Beyond one-shot prompting, fixed-demo prompt strategies have been explored to stabilize parsing behavior in streaming settings (e.g., Liu et al. [92]), and some pipelines further leverage synthetic or auxiliary LLMs to improve downstream parsing quality (e.g., LogGenius [196]).

Within prompting, adaptive ICL refines the paradigm by selecting demonstrations on the fly instead of relying on a static prompt. DivLog [188] is representative: it dynamically selects relevant labeled examples for each incoming log, turning parsing into a contextual compare-and-contrast process that helps the model separate invariants from variables for the specific message at hand. Empirical follow-ups suggest that *diversity* in the selected examples and explicit treatment of variables are recurring levers for robustness and cost control (e.g., LLMDPP [173] and VISTA [97]).

More broadly, prompt-based parsing effectively converts “rules in code” into “rules in the prompt”, which improves iteration speed and portability across models, but shifts the main technical challenge to *controllability and normalization*: the approach must guard against over-generalization, inconsistent placeholders, and format violations, which motivates explicit output constraints, prompt templates tailored to log syntax, and restricted decision spaces such as candidate/template detection prompts (e.g., LLM-TD [169]) or rule-augmented prompting (e.g., LogRules [57]). In effect, adaptive ICL shifts the core engineering problem from “design a single good prompt” to “design a good *retriever/sampler* for prompts”, which improves coverage but introduces a new dependency on selection quality and context-budget constraints.

5.3.4 Retrieval/refinement-oriented unsupervised parsing: reducing labels via self-generated context and iterative correction. A third family targets the no-label regime by replacing curated demonstrations with retrieval and refinement loops. Instead of supplying ground-truth templates as exemplars, these methods retrieve representative logs (or groups of logs) as contextual anchors and let the LLM infer templates from the retrieved context. LogBatcher [184] is representative: it performs demonstration-free parsing by retrieving or grouping similar logs and querying the LLM only on representative instances, thereby reducing both labeling and query volume. LibreLog [104] pushes this direction further with open-source models: it combines retrieval-augmented parsing with iterative self-reflection and a template memory, using refinement to compensate for missing labels while improving efficiency and privacy. Related work employs the same approach via self-generated ICL and self-correction mechanisms (e.g., AdaParser [182]) or via unsupervised template repositories that accumulate reusable templates over time [54].

A recurring observation is that “unsupervised” here is rarely “single-shot”. Effectiveness hinges on the quality of upstream grouping/retrieval: if retrieved logs mix multiple templates, the model may output an averaged, over-general template. If retrieval is too narrow, variables may be frozen as constants. Consequently, many approaches couple retrieval with explicit *grouping and correction* strategies, such as embedding-driven clustering before template generation (e.g., HELP [186]), entropy- or structure-guided grouping followed by template merging/refinement (e.g., Lemur [207] and entropy-based correction pipelines [26]), and memory-based reuse to converge toward stable inventories over long streams [54, 104]. In effect, these parsers trade manual labels for approach-level iteration: retrieval/grouping proposes candidate structure, while refinement/memory stabilizes templates across time and workload shifts.

5.3.5 System-level efficiency layers: grouping, caching, scheduling, and agentic decomposition. Across the paradigms above, many approaches add system-level layers that make LLM parsing feasible in online or high-volume settings. These layers are orthogonal to the main learning paradigm: a parser may use prompting, ICL, or retrieval/refinement while also caching templates, grouping similar messages, scheduling LLM calls, or using tools to decompose the workflow.

A common pattern is to explicitly *reuse* past results to amortize LLM queries and stabilize outputs. LILAC [64] exemplifies this engineering pattern: it combines accuracy-oriented ICL (via carefully selected demonstrations) with an adaptive parsing cache that stores and refines templates over time, reducing repeated calls for recurring patterns while improving consistency under streaming workloads. Some approaches attach a template repository to zero-shot prompting so previously derived templates can be looked up and reused [212], while unsupervised parsers can rely on a template database to eliminate labeled demonstrations but still exploit reuse across the stream (e.g., Huang et al. [54]). Other approaches reduce open-ended generation by narrowing the search space, e.g., by prompting the model to identify multiple templates from a batch or from candidate structures (LLM-TD [169]), or by coupling semantic modeling with adaptive caching for robustness under drift [75].

Grouping-based approaches parse only a small set of representative logs and propagate templates to the rest. LogParser-LLM [213] is representative of this family, using grouping and multi-step prompting to balance template fidelity against query volume. Beyond grouping, execution-layer optimizations target tail latency and inference overhead. EPAS [15] frames online parsing as an asynchronous scheduling problem, while other lines optimize prefix/KV-cache reuse or agentic decomposition (e.g., InferLog [177] and Parse-LLM [152]). The key trade-off is *error amplification*: once an incorrect template is cached, propagated, or promoted into memory, reuse layers can spread it widely. Hence, practical approaches typically require cache update policies, validation/refinement hooks, and sometimes variable-aware reuse strategies that explicitly track parameter slots to avoid template corruption at scale (e.g., SemanticLog [198]). A useful takeaway is that these approaches treat LLM reasoning as a high-quality but expensive primitive, and invest in coordination layers to maximize structured output per LLM token.

Overall, the method families address different parts of the challenge space. Supervised and log-specific model adaptation improves controllability and template-parameter boundary learning when labels or domain corpora are available. Prompt-based and adaptive ICL parsing supports rapid deployment when labeled data are limited and log patterns keep changing, but it depends on prompt and example quality. Retrieval/refinement-oriented parsing reduces manual labeling and helps cover less frequent patterns, but it depends on grouping and retrieval quality. System-level efficiency layers improve online throughput and output stability, but they can amplify cached or propagated errors if validation is weak.

Early log parsing work focused on feasibility and prompting sensitivity, while more recent work increasingly separates two concerns: the learning/adaptation paradigm used to infer templates, and the reuse or coordination mechanisms used to make LLM parsing practical at scale. Importantly, these families of approaches coexist. Supervised adaptation remains attractive when labeled logs are available, prompting dominates rapid deployment, retrieval/refinement reduces labeling needs, and system-level reuse is essential for online or mass scale. Across them, a unifying design principle is to combine LLM inference with lightweight parsing components, sampling, retrieval, grouping, caching, and validation that amortize LLM cost while preserving template stability under drift.

5.4 Performance Evaluation Metrics

Evaluation of log parsing has gradually shifted from *message-level* headline scores toward *template-level* metrics, because production logs are heavy-tailed and drift over time: frequent templates can dominate averages and hide failures on rare but diagnosis-critical events. As a result, recent LLM-based parsers increasingly report classic metrics together with template-level ones to better reflect robustness (e.g., LILAC [64], Le et al. [75], LogParser-LLM [213]).

Classic and still common: GA and PA (but frequency-biased). The most widely used metrics remain **Grouping Accuracy (GA)** and **Parsing Accuracy (PA)** because they are simple and comparable across studies [4,

20, 73]. GA evaluates whether each line is assigned to the correct event group, while PA evaluates whether the per-line template exactly matches the oracle [20, 73]. However, both are *message-level* averages: frequent templates dominate, so a parser can score high while failing on the long tail or under drift [64, 75]. They can also blur “grouping is correct” versus “placeholder boundaries are correct”, motivating template-level complements [57, 64].

Increasingly adopted for robustness: FGA and FTA. To mitigate frequency bias, many recent parsers report **FGA** (*F1 of Grouping Accuracy*) and **FTA** (*F1 of Template Accuracy*) as *template-level* complements to GA/PA [64, 75, 186, 213]. FGA scores correctness *per template group* (then aggregates via precision/recall-style F1), making rare-template fragmentation/merging visible. FTA is stricter because it also requires the *template text* (static tokens and placeholders) to match, exposing boundary errors that GA/PA can hide [57, 64]. In practice, reporting {GA,PA}+{FGA,FTA} gives both backward comparability and clearer evidence of template-inventory recovery [64, 75].

Diagnostics and niche variants: ED/LCS, GGD/PGD, PTA/RTA, F1, and RI. Several studies add **Edit Distance (ED)** and/or **Longest Common Subsequence (LCS)** between predicted and oracle templates to quantify *how far* near-misses are (useful for diagnosing systematic boundary mistakes), but these should be treated as secondary signals instead of standalone correctness measures [4, 20, 73]. Some papers further introduce **GGD/PGD** to diagnose *error modes*: whether mistakes stem mainly from over-merging/over-splitting (grouping granularity) or from static↔variable boundary mismatches in templates [213]. A few prompting-based parsers report **PTA/RTA** as template-level precision/recall variants that emphasize strict template recovery (often stricter than PA) [123, 188]. When parsing is formulated as token labeling, **F1** evaluates whether parameter boundaries are identified correctly [144]. And for clustering-style evaluation, **RI** (Rand Index) measures pairwise agreement of the predicted vs. oracle partition (together vs. apart) [42, 164]. Even within log parsing, direct cross-paper comparison remains limited because studies differ in datasets, online versus offline settings, oracle template definitions, grouping policies, and which metrics they report.

6 Downstream Log Analysis Tasks

Downstream log analysis aims to turn high-volume log evidence into actionable signals. Many pipelines operate on parsed templates and parameters, while some LLM-based approaches work directly with raw, lightly normalized, or embedded log windows. In this survey, we focus on four representative tasks: **anomaly detection** (flagging abnormal log sequences or windows), **failure prediction** (forecasting failures within a future time horizon), **root cause analysis** (identifying the most likely cause or component of an incident and justifying it with evidence), and **log summarization** (compressing large log windows into concise, operator-friendly summaries).

Challenges motivating the shift toward LLMs. Historically, operators often relied on keyword search and manual inspection, which is fragile under message vocabulary variation and scale [47]. Rule-based analytics can be efficient but are brittle under format drift and hard to adapt across systems, especially when log formats and templates evolve over time [21, 46]. Learning-based detectors and predictors improve automation, yet they often depend on careful feature engineering, stable parsing/representation choices, and sufficient labeled data, and can still be sensitive to log evolution [118]. They may also struggle to provide human-friendly explanations, motivating more interpretable and interaction-oriented log analysis workflows [68, 92]. In the LLM era, researchers increasingly leverage models’ semantic generalization to (i) better tolerate heterogeneous phrasing and evolution, and (ii) generate explanation-oriented outputs that help operators understand not only *what* happened but also *why* [20, 42].

A common workflow: partition → representation → task inference → action. In practice, downstream analysis is rarely performed on a monolithic log file. Instead, logs are first *partitioned* into sessions or windows to separate mixed sources (modules/threads/processes) into coherent log sequences. Partitioning can be driven

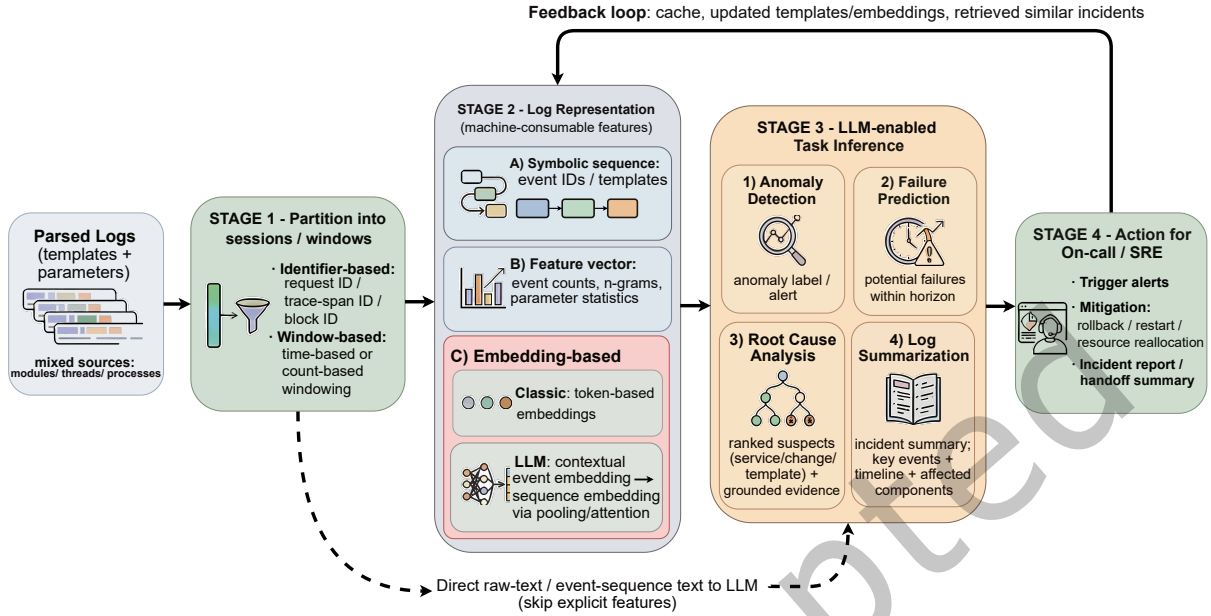


Fig. 5. A common workflow for LLM-enabled downstream log analysis.

by identifiers (e.g., request ID, trace/span ID, block ID) or by time- or count-based windowing when identifiers are unavailable. After sequences are formed, a pipeline may parse events, derive embeddings or other *log representations*, select salient context, or directly prompt an LLM, and then apply task-specific approaches to trigger alerts, rank suspected causes, predict upcoming failures, or summarize incidents for operators. Figure 5 summarizes this workflow and highlights where LLMs are commonly integrated to improve failure perception, diagnosis, and operator-facing actions.

We organize the rest of this section following the workflow in Figure 5. Section 6.1 discusses **log representation**, which largely determines what any downstream model can learn or infer. Sections 6.2, 6.3, 6.4, and 6.5 then cover four representative LLM-enabled tasks: **anomaly detection**, **failure prediction**, **root cause analysis**, and **log summarization**.

6.1 Log Representation

Given a sessionized (or windowed) log sequence, typically a sequence of parsed events/templates with optional parameters and timestamps, **log representation** maps it into machine-consumable features that preserve task-relevant signals while suppressing nuisance variability. Depending on the downstream task, the representation can be (i) a discrete symbolic sequence (event IDs/templates), (ii) a fixed-length feature vector (counts, n-grams, parameter statistics), or (iii) a continuous embedding sequence or sequence embedding produced by neural encoders. Logs contribute information that is difficult to replace with metrics, traces, or code alone: symptom wording, exception/error text, component-local execution narratives, chronological evidence trails, and sparse but informative runtime context. Representation choice is also a major source of cross-paper incomparability because studies differ in whether they use raw messages, parsed templates, event IDs, parameters, embeddings, or direct prompt inputs. Figure 6 illustrates a log window and how it can be mapped into different representation types.

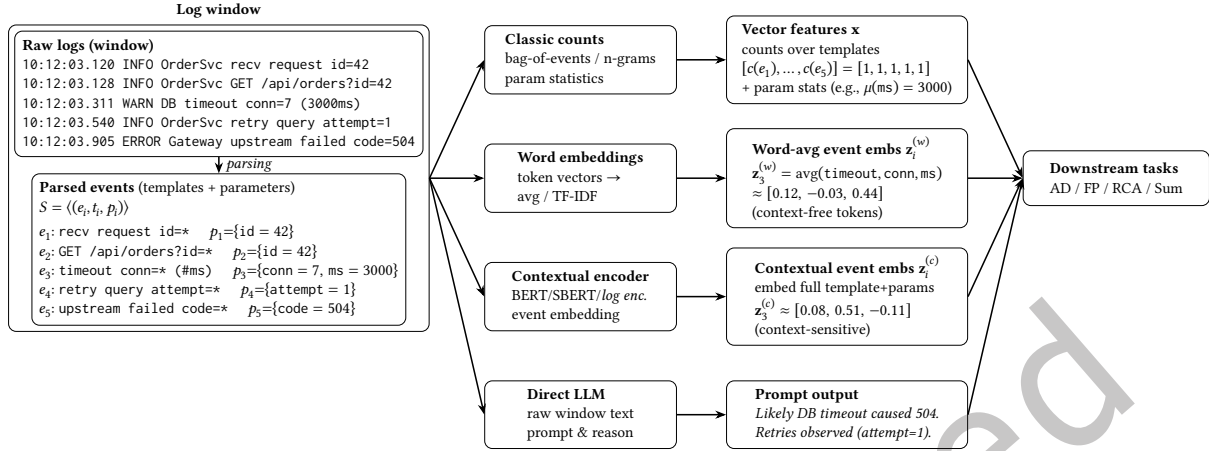


Fig. 6. An illustrative overview of log window representations: raw logs are parsed into templates and parameters, then mapped to (i) classic feature vectors, (ii) context-free word-averaged embeddings, (iii) contextual event embeddings, or (iv) direct prompt-based LLM outputs. Numeric vectors are schematic to highlight different representations.

Traditional representations: statistical, parameter-aware, and semantic sequence features. A common starting point is **statistics-based** representation, which converts each log sequence into a feature vector by counting event occurrences (bag-of-events) or short patterns such as n-grams. This view is compact and efficient, but it can lose semantic similarity between differently worded events and captures ordering only shallowly. A closely related line is **parameter-aware** representation, which augments event statistics with counts/patterns over extracted parameter values (IDs, paths, codes), useful when diagnosis depends on *which entities/values* appear, but potentially brittle under drift and high-cardinality parameters.

The most widely used **semantic** representations in traditional pipelines build embeddings from the message/template text. A typical workflow is to map each token in a log event to a word embedding (e.g., word2vec algorithm [112]), aggregate token vectors into an *event embedding* (often by weighted averaging), and then compose a *sequence embedding* by pooling or concatenating embeddings across events in the window. This family is attractive because it provides a continuous semantic space while remaining lightweight.

LLM-era representations: reuse classic features or switch to contextual encoders. Many recent LLM-based log analytics approaches still reuse classic representations (event-count vectors, parameter statistics, n-grams, or word-embedding-based features) as inputs to downstream models because they are cheap, stable, and compatible with existing monitoring pipelines. In these hybrid designs, LLMs may serve only as an auxiliary component (e.g., to improve parsing quality or provide explanations), while the anomaly detector/predictor still relies on traditional features and machine-learning models.

At the same time, many recent pipelines adopt *contextual event embeddings*. Instead of averaging context-free word vectors, they embed each entire log event with a pretrained encoder. The main advantage is robustness: contextual encoders model word order and compositional meaning within an event, and they can assign different representations to the same token across different contexts (which is common in fast-evolving software logs) [30, 125]. As a result, semantically equivalent events with heterogeneous phrasing tend to stay closer in the embedding space, while subtle boundary or wording changes are less likely to break downstream models, a property that is particularly useful under drift, long-tail patterns, and limited supervision.

In practice, the workflow is lightweight. Each parsed event (often the template text, optionally concatenated with selected parameters) is converted to a short input string and encoded with a contextual model such as BERT [24] and Sentence-BERT [135], or with log-oriented encoders such as PreLog [74] and OWL [42], to obtain a fixed-length *event embedding*. For sequence-level tasks, event embeddings within a session/window are then aggregated (e.g., pooling/attention or a lightweight sequential model) into a *window representation*. Downstream modules use these representations either for (i) similarity-based retrieval/matching (nearest-neighbor search for related events/windows) or (ii) training detectors/predictors for anomaly detection and failure prediction, often with improved tolerance to wording variation compared to word-embedding pipelines.

Unified log representations via large-scale pretraining. Beyond “using a general-purpose encoder,” BigLog [165] advocates *unsupervised large-scale pre-training* to learn a *unified* representation space for logs. The core idea is to pretrain on massive, heterogeneous log corpora so that a single encoder can produce transferable embeddings across systems and log styles, reducing the reliance on per-system feature engineering or task-specific representation tuning. This perspective is especially relevant in LLM-era pipelines where the bottleneck often shifts from model capacity to *representation compatibility*: when logs change frequently or when operators need cross-service retrieval and correlation, a unified embedding space enables (i) more reliable similarity search over historical windows, (ii) easier cross-system transfer/bootstrapping under limited labels, and (iii) a common interface that multiple downstream tasks (anomaly detection, failure diagnosis, summarization) can share. In practice, BigLog-style pretraining can be viewed as a middle ground between “classic features” and “prompt on raw text”: it keeps inference efficient (fixed-length embeddings) while improving semantic robustness and portability at scale.

Skipping explicit features: direct raw-text or event-sequence text as input. Finally, some LLM-based approaches partially or fully bypass handcrafted feature construction by feeding an LLM the *raw log window* (or a text form of the *event sequence*) and prompting it to perform the downstream task directly. This design can preserve richer context and enable explanation-oriented outputs, but it introduces new constraints: long contexts require careful windowing, summarization, or retrieval to control noise and cost, and online deployment may demand additional compression or caching mechanisms. Overall, representation choices trade off efficiency, robustness, and fidelity. Thus, many recent approaches combine compact classic features with contextual embeddings or selective raw-text reasoning, depending on task requirements and operational budgets.

6.2 Anomaly Detection

6.2.1 Task definition. Log anomaly detection identifies *unexpected* or *undesired* system behavior patterns from logs. Such anomalies typically indicate potential faults, misconfigurations, performance regressions, or other conditions that violate the expected normal execution, and thus serve as early signals for reliability triage.

For anomaly detection, results support log-specific findings in three situations: runtime logs are the main evaluated input, runtime logs are an explicit part of multimodal evidence, or the reported results explicitly involve logs in a setting where log use is optional.

Given a sessionized (or windowed) log sequence $S = \langle (e_i, t_i, p_i) \rangle_{i=1}^n$, where e_i is a parsed event/template (or the raw log message), t_i is the timestamp, and p_i are optional parameters, the task outputs either (i) an anomaly score $a(S)$ or a binary label $y \in \{0, 1\}$ indicating whether the sequence is abnormal, and optionally (ii) localized evidence such as suspicious positions $\mathcal{J} \subseteq \{1, \dots, n\}$ (or key templates/snippets) to support debugging.

In practice, anomaly detection is usually *reactive*: it flags abnormal states or behavior patterns in observed logs (often in unsupervised or weakly supervised settings), without necessarily predicting whether/when a concrete failure will occur.

Table 6. Summary of LLM-based anomaly detection studies organized by task category. (continued in Table 7)

Paper	Supervised?	Train?	Dataset(s) evaluated	Parsing?	Metric(s)	Base Model(s)
<i>Discriminative & Representation Learning (Encoder-based)</i>						
HitAnomaly [55]	Supervised	Yes	HDFS, BGL, OpenStack	Yes	F1	BERT
SwissLog [80]	Supervised	Yes	HDFS, BGL	Yes	F1	BERT
Ott et al. [118]	Semi-Sup	Yes	OpenStack	Yes	F1	BERT
LogBert [40]	Semi-Sup	Yes	HDFS, BGL, Thunderbird	Yes	F1	BERT
NeuralLog [71]	Supervised	Yes	HDFS, BGL, TB, Spirit	No	F1	BERT
BERT-Log [14]	Supervised	Yes	HDFS, BGL	Yes	F1	BERT-110M
LogST [210]	Semi-Sup	Yes	HDFS	Yes	F1	Sentence-BERT
Prog-BERT-LSTM [147]	Semi-Sup	Yes	HDFS, BGL, Thunderbird	Yes	F1	BERT
HilBERT [56]	Supervised	Yes	HDFS, BGL	Yes	F1	BERT
LogBP-LORA [48]	Supervised	Yes	BGL, Thunderbird	Yes	F1	BERT-base
LogDAPT [211]	Supervised	Yes	HDFS, BGL, Thunderbird	Yes	F1	BERT
LogADSBERT [52]	Semi-Sup	Yes	HDFS, Hadoop, Open-Stack	Yes	F1	Sentence-BERT
Karlsen et al. [67]	Unsup	No	ECML, CSIC, Access	Yes	F1	DistilRoBERTa
FastLogAD [90]	Semi-Sup	Yes	HDFS, BGL, Thunderbird	Yes	F1	ELECTRA
LAnoBERT [77]	Semi-Sup	Yes	HDFS, BGL, Thunderbird	No	F1, AUC	BERT
Corbelle et al. [18]	Supervised	Yes	HDFS	Yes	F1	BERT-base
Zhou et al. [218]	Supervised	Yes	HDFS, BGL	Yes	F1	BERT
LogELECTRA [191]	Semi-Sup	Yes	BGL, Thunderbird, Spirit	No	F1	ELECTRA
LogFiT [2]	Semi-Sup	Yes	HDFS, BGL, Thunderbird	No	F1	BERT
LogRoBERTa [159]	Supervised	Yes	HDFS, BGL	No	F1	RoBERTa
ADALog [127]	Unsup	Yes	BGL, Thunderbird, Spirit	No	F1	DistilBERT
Thali et al. [166]	Supervised	Yes	SWaT and Morris ICS telemetry	No	F1	BERT
APT-LLM [7]	Semi-Sup	Yes	DARPA TC host-audit events	Yes	AUC	BERT
Entezami et al. [29]	Supervised	Yes	KDD99, UNSW, CICIDS network traffic	No	F1	BERT
AnomalyExplainerBot [5]	Supervised	Yes	HDFS	Yes	F1	RoBERTa
Horváth et al. [51]	Supervised	Yes	BGL, Thunderbird	Yes	F1, Acc	LLaMA
<i>Generative Forecasting & Next-Token Prediction (Decoder-based)</i>						
LogGPT (BigData) [168]	Supervised	Yes	HDFS, BGL, Thunderbird	No	F1	GPT-2
LogSense [154]	Semi-Sup	Yes	-	Yes	-	GPT-2
PreLog [74]	Supervised	Yes	HDFS, BGL, Spirit	No	F1	FLAN-T5 (140M)
Bogdan et al. [9]	Supervised	Yes	ECU network packets	No	Top-k	Qwen2-0.4B
LogLM [94]	Supervised	Yes	BGL, Spirit	Yes	F1	LLaMA-2-7B
NLPLog [62]	Supervised	Yes	NLPLog	Yes	F1	LLaMA-2-7B

6.2.2 LLM-based Approaches for Anomaly Detection. Table 6–7 summarizes LLM-based anomaly detection studies. Across these studies, a common pipeline is: (i) partition logs into sessions/windows, (ii) encode each sequence into a representation (templates, embeddings, or raw text), (iii) compute an anomaly signal (score/label), and (iv) optionally return *evidence* (highlighted lines, retrieved neighbors, or explanations) to support triage. Existing approaches mainly differ along four design axes: **(1) representation interface** (parsed templates vs.

Table 7. Summary of LLM-based anomaly detection studies organized by task category. (continued from Table 6.)

Paper	Supervised?	Train?	Dataset(s) evaluated	Parsing?	Metric(s)	Base Model(s)
Prompt-Based & In-Context Learning (Zero/Few-shot)						
LogPrompt (IJCNN) [205]	Supervised	Yes	HDFS, BGL	No	F1, Acc	BERT-large
LogPrompt (ICPC) [92]	Unsup	No	HDFS, BGL	Yes	F1	ChatGPT
Setu et al. [106]	Supervised	Yes	Apache, BGL, HDFS	Yes	F1, AUC	GPT-3
LogGPT (HPCC) [129]	Supervised	No	BGL, Spirit	Yes	F1	GPT-3.5-turbo
Egersdoerfer et al. [27]	Supervised	No	Lustre	No	F1	GPT-3.5-turbo
Owl [42]	New Model	Yes	BGL, Spirit	Yes	F1	Owl (LLaMA)
Fariha et al. [32]	Unsup	Yes	HDFS	Yes	F1	GPT-3.5-turbo
VMFT-LAD [143]	Semi-Sup	Yes	VM/Server failures	Yes	AUC	GPT-3.5-turbo
FLEXLOG [44]	Supervised	Yes	ADFA, SynHDFS	Yes	F1	Mistral-22B
CloudAnoAgent [221]	Supervised	No	CloudAnoBench (logs + metrics)	No	Acc	Gemini-2.5
Retrieval-Augmented Generation (RAG)						
RAPID [115]	Semi-Sup	No	HDFS, BGL, Thunderbird	No	F1	BERT
LogRag [206]	Semi-Sup	Yes	BGL, Spirit	Yes	Prec, Rec, F1	GPT-3.5-turbo, Mistral-7B-Instruct
RAGLog [121]	Semi-Sup	No	BGL, Thunderbird	No	F1	RAG Model
XRAGLog [202]	Semi-Sup	No	HDFS, BGL, Thunderbird	Yes	F1	GPT-3.5
Hybrid & Collaborative (Small Model + Large Model)						
LLMeLog [45]	Supervised	Yes	HDFS, BGL, Thunderbird	Yes	F1	ChatGPT + BERT
LogFormer [41]	Supervised	Yes	HDFS, BGL, TB, GALA	No	F1	S-BERT + ChatGPT
CLogLLM [137]	Unsup	Yes	HDFS, BGL, Thunderbird	No	F1	Qwen + GPT-3.5
LogLLM [39]	Supervised	Yes	HDFS, BGL, TB, Liberty	No	F1	BERT + LLaMA-3
AdaptiveLog [101]	Supervised	Yes	BGL, Thunderbird	No	F1	BERT + ChatGPT
LogRules [57]	Supervised	Yes	BGL, Spirit	No	F1	GPT-4o + LLaMA-3
LogADRef [89]	Supervised	Yes	BGL, HDFS, Spirit, TB	Yes	F1	RoBERTa + GPT2
SemiRALD [158]	Supervised	Yes	HDFS, BGL	Yes	F1	ChatGPT + BERT
LogSynergy [155]	Supervised	Yes	Three production datasets, BGL, Thunderbird, Spirit	Yes	F1	ChatGPT-4o
CLSLog [183]	Supervised	Yes	BGL, Zookeeper	No	F1	BERT + Qwen
LEMAD [61]	Semi-Sup	Yes	State Grid logs + runtime metrics	Yes	F1	BERT + GPT-4o
CoLA [220]	Supervised	Yes	HDFS, BGL, Spirit	Yes	F1	LLaMA-3 + GPT-4o
LUK [98]	Supervised	Yes	BGL, Spirit, Thunderbird	Yes	F1	GPT-3.5 + Expert
SemiSMAC [157]	Supervised	Yes	BGL, HDFS, Spirit	Yes	F1	GPT-4-turbo
Agentic, Reasoning & Neuro-Symbolic Frameworks						
Audit-LLM [151]	Unsup	No	CERT r4.2/5.2	No	F1	GPT-3.5 (Agent)
WebNorm [88]	Semi-Sup	No	TrainTicket, NiceFish	Yes	F1	GPT-3.5 (Agent)
LogRESP-Agent [76]	Semi-Sup	Yes	EVTX-ATTACK	No	F1, TPR	Gemini-2.0-Flash
Song et al. [153]	Supervised	Yes	CERT v6.2	No	F1	ChatGLM-6B
SHIELD [156]	Supervised	Yes	DARPA-E3, ATLASv2	Yes	Prec	DeepSeek-R1
LogReasoner [100]	Supervised	Yes	BGL, Spirit	No	F1	Qwen2.5-1.5B
Data Generation, Benchmarks & Visions						
Karlsen et al. [68]	Supervised	Yes	CSIC, Thunderbird, BGL	Yes	F1	RoBERTa/GPT
LogEval [20]	Supervised	No	BGL, Thunderbird	No	F1	GPT-4
AD-LLM [193]	Semi-Sup	Yes	News, reviews, and spam	No	AUC	LLaMA-3/GPT-4o
AnomalyGen [81]	Supervised	Yes	Hadoop, HDFS	Yes	F1	GPT-4o

raw text), (2) **supervision setting** (fully supervised with both normal/anomalous labels vs. semi-supervised/one-class settings that train mostly or only on *normal* data, vs. unsupervised settings), (3) **adaptation/training strategy** (fine-tuning/prompt-tuning vs. keeping the LLM frozen and using it in a training-free manner), and (4) **inference form** (discriminative classification, likelihood/perplexity scoring, prompting/ICL, retrieval, or agentic reasoning). Notably, prompting/in-context learning (ICL) is largely orthogonal to supervision, and “training” is not synonymous with “supervision”: (i) many methods *train* models without using task labels (e.g., further pretraining or contrastive learning on related unlabeled logs/corpora), and (ii) some methods are *training-free* but still supervised at inference time by using labeled exemplars as demonstrations in ICL. Note that boundaries are not strict: many systems combine multiple mechanisms (e.g., encoder embeddings + prompting, or retrieval + generation).

Discriminative and representation-learning encoders. A large body of work uses pretrained encoders (typically BERT-like) to map log events or event sequences into embeddings, followed by a classifier or sequence model for anomaly prediction. Representative examples that operate on parsed templates and learn discriminative decision boundaries include HitAnomaly [55], SwissLog [80], and HilBERT [56]. Several studies adapt or optimize the encoder/sequence modeling pipeline, such as BERT-Log [14], Prog-BERT-LSTM [147], and the semantic hierarchical compaction + BERT-based detection by Corbelle et al. [18]. Other works emphasize robustness and transfer across systems and unstable log vocabularies. For instance, Ott et al. study robustness and transferability with pretrained language models [118], and APT-LLM applies embedding-based detection to host audit and provenance event logs [7]. Some papers explicitly investigate LLM-assisted *parsing + embedding* pipelines before detection (e.g., Zhou et al. [218]), while related comparisons by Thali et al. [166] and Entezami et al. [29] evaluate ICS telemetry and network traffic without explicit runtime logs. Finally, as LLM-era anomaly detection expands beyond classic system logs, recent large-scale studies also include LLaMA-family backbones in supervised pipelines (e.g., Horváth et al. [51]) and explanation-oriented add-ons (e.g., AnomalyExplainerBot [5]).

Masked-LM and self-/semi-supervised normality modeling. A closely related family models *normal* log patterns using self-supervised objectives and flags deviations as anomalies, which is well-suited to rare-anomaly regimes. LogBERT [40] and LAnoBERT [77] exemplify masked-language-model training on normal sequences, producing anomaly scores via reconstruction/prediction errors. Recent variants continue this direction with alternative backbones and training strategies, such as LogELECTRA [191], LogFiT [2], and ADALog [127]. FastLogAD [90] further augments semi-supervised learning by generating pseudo anomalies to strengthen discrimination. In practice, these methods often reduce label requirements but still rely on careful sessionization/windowing and calibration to control false positives across evolving software versions.

Reducing reliance on log parsing: from templates to raw text. Because parsing errors and format drift can directly harm downstream detectors, multiple works explicitly minimize or remove template parsing. Neural-Log [71] studies anomaly detection without conventional log parsing, and LogRoBERTa [159] further reduces parser reliance via hybrid language modeling. In addition, several prompt- or LLM-assisted pipelines still use lightweight normalization (e.g., regex-based variable masking) as a middle ground between full template parsing and raw text, especially when deploying at scale under frequent log evolution.

Decoder-style generative forecasting and likelihood-based scoring. Instead of classification, decoder LMs treat anomaly detection as a *language modeling* problem: a sequence is anomalous if it is hard to predict under a model trained (or adapted) on normal behavior. LogGPT [168] is a representative GPT-style next-token prediction approach, while PreLog [74] explores a pretrained model tailored for log analytics tasks (including anomaly detection) using a compact generative backbone. Instruction-centric systems also evaluate anomaly detection within a broader log analysis interface (e.g., LogLM [94] and knowledge-augmented adaptation in NLPLog [62]). For comparison, “Good Enough to Learn” investigates anomaly detection in ECU network packets under unreliable labels with small LMs [9].

Prompting and in-context learning for training-free detection and interpretation. With instruction-tuned LLMs, several works bypass explicit model training and instead prompt an LLM to classify sequences or explain suspicious patterns. LogPrompt-style prompt engineering emphasizes interpretability and online usage [92, 93], while feasibility studies explore ChatGPT-style models for anomaly detection under different settings and log sources [27, 106, 129]. Some lines treat LLMs as an *operations foundation model* that supports multiple tasks, including anomaly detection, under a unified interface (e.g., OWL [42]). At the benchmark level, large-context prompting is also evaluated in broad suites (e.g., LogEval [20]), clarifying how far prompting alone can go without task-specific training.

Retrieval-augmented detection: comparing against a memory of normality. Retrieval-based methods treat anomaly detection as *nearest-neighbor comparison* in an embedding space of normal sequences and often provide evidence by returning similar historical cases. RAPID [115] embeds normal windows using a pretrained encoder and flags anomalies by distance/neighborhood structure, while RAGLog [121] frames detection through retrieval-augmented generation by retrieving similar normal cases and judging the target’s consistency. XRAGLog [202] further targets resource efficiency and context awareness in RAG-style pipelines. This family is especially compatible with semi-supervised settings (normal-only data) and naturally supports operator-facing justifications through retrieved exemplars.

Hybrid and collaborative frameworks: small models for scoring, LLMs for enrichment, verification, or rules. Many practical designs combine a lightweight detector with an LLM component that improves robustness or interpretability. LLMeLog [45] enriches log events with LLM-derived semantics before training a smaller classifier. LogFormer [41] combines representation learning with LLM assistance, and LogLLM [39] explores joint use of encoders with larger LMs. AdaptiveLog [101] explicitly frames anomaly detection as collaboration between large and small models, while CoLA [220] studies model collaboration at scale with multiple LLMs. Knowledge and control signals are also injected via rules or domain constraints (e.g., LogRules [57]), and tuning procedures aim to stabilize performance (e.g., SemiSMAC [157] and SemiRALD [158]). Finally, cross-system generalization and “new system” adaptation are increasingly important, motivating transfer-focused designs such as LogSynergy [155].

Agentic and multi-step reasoning for complex security/incident contexts. A growing line of work frames anomaly detection as a multi-step investigation that iteratively retrieves context, applies tools, and produces both predictions and narratives. Examples include multi-agent collaboration for threat/anomaly scenarios (Audit-LLM [151]), consistency-based detection with evidence building (WebNorm [88]), and recursive/agentic pipelines for context-aware detection and response (LogRESP-Agent [76]). Related systems emphasize turning alerts into operator-facing intelligence (SHIELD [156]) or improving reasoning structure and controllability (LogReasoner [100]), while insider-threat anomaly detection appears as another domain where agentic reasoning is natural (Song et al. [153]). Agentic methods can improve interpretability and reduce analyst workload, but introduce additional latency/cost and require careful guardrails for reliable deployment.

Benchmarks and data generation. As the space expands, benchmarks and data-centric efforts have become important for fair comparison and stress-testing. Karlsen et al. benchmark LLMs for log analysis tasks including anomaly detection [68], and LogEval provides a broader benchmark suite for LLM-based log analysis [20]. For comparison, AD-LLM evaluates LLM-based anomaly detection across general-text domains [193], helping compare anomaly-detection capabilities outside runtime-log settings. Complementarily, AnomalyGen [81] explores LLM-driven generation of anomalous log sequences, reflecting an emerging direction where synthetic anomalies and augmentation are used to improve coverage of rare patterns and unstable logs.

Across LLM-based anomaly detection studies, the main shift is not just higher accuracy. It is also a move toward treating anomalies as *evidence-grounded inconsistencies* with expected behavior, such as deviations from retrieved “normal” cases, violations of explicit rules, or unusually high calibrated anomaly scores. The strongest systems therefore constrain LLMs with stable interfaces (templates, embeddings, or normalized text) and use them mainly to explain, verify, or enrich lightweight detectors. LLMs do not remove the ambiguity of what counts as an anomaly. Instead, they shift effort from feature design to normality curation, context selection (windowing or retrieval), and operational guardrails that control false alarms and cost. This makes hybrid, evidence-driven workflows a practical deployment pattern.

6.3 Failure Prediction

6.3.1 Task definition. Log-based failure prediction is a proactive early-warning task: it forecasts whether the system (or a component) will experience a *future failure* within a horizon, so operators can take preventive actions (e.g., mitigation, rollback, or resource reallocation) before the system enters an unrecoverable or user-visible failure state. Given logs observed up to the current time t (often represented as a session/window $S_{t-L:t}$ or a prefix of an ongoing session), the task outputs a probability $\Pr(\text{failure in } [t, t + H])$ (or a binary label) for a future horizon H , and sometimes additional signals such as predicted time-to-failure or early-warning decisions under different lead-time budgets.

Compared with anomaly detection, failure prediction is explicitly *forward-looking* and tied to an operational failure definition (e.g., crash, outage, severe SLO violation, incident ticket). Its difficulty depends on the prediction horizon: useful early signals in logs may be weak, sparse, and distributed across components or long time ranges, while short-horizon signals can be clearer but less actionable. Depending on the environment, prediction may focus on (i) failures of similar components in homogeneous systems (sequence-centric modeling), or (ii) failures driven by interactions among heterogeneous components (leveraging cross-component correlations).

6.3.2 LLM-based Approaches for Failure Prediction. Compared with anomaly detection, failure prediction must explicitly optimize lead time and horizon awareness: a useful predictor should raise early warnings sufficiently ahead of a failure event while controlling false alarms. Consequently, papers often report ranking/probabilistic metrics (e.g., AUC) and sometimes assess whether the model can provide actionable context (time-to-failure, likely causes).

Generative prediction of crash events and causes. CrashEventLLM casts failure prediction as a text generation problem: given preceding logs, the model predicts forthcoming crash-related information (e.g., time and cause) using an instruction-tuned LLaMA backbone, and evaluates generation quality with ROUGE-style metrics. [114] This formulation is attractive when operators want human-consumable explanations, but it also raises a methodological question: lexical-overlap metrics (e.g., ROUGE) may not faithfully reflect correctness of timestamps/causal factors, so future work may benefit from structured evaluation (e.g., exact match on failure type/time bins, and evidence grounding).

Discriminative early warning from language-model representations. FALL proposes a language-model-based detector that outputs an early-warning score (reported with AUC-style evaluation) to detect imminent failures in large-scale systems. [60] In this design, the PLM (e.g., ELECTRA-style encoders) provides a robust representation of evolving log patterns, and the predictor focuses on separating pre-failure prefixes from normal operation. Conceptually, this parallels semi-supervised anomaly detection, but with labels aligned to future failure windows (pre-failure vs. non-pre-failure), making horizon definition and data leakage control (e.g., avoiding post-failure artifacts) especially important.

Leveraging operational context beyond application logs. Combining LLMs and shell logs for backup-failure prediction highlights another practical pattern: operational failures are often best predicted using heterogeneous

telemetry, where shell commands, job traces, and system events provide context that is not present in application logs alone. [35] Here, an LLM can serve as a semantic integrator for noisy, free-form operational traces, potentially improving feature robustness and interpretability.

From anomaly detection to proactive fault tolerance. VMFT-LAD (Virtual Machine Proactive Fault Tolerance Using Log-Based Anomaly Detection) connects anomaly scoring with proactive mitigation for virtualized environments, treating detected abnormal patterns as early signals to trigger fault-tolerance actions (e.g., proactive migration or resource adjustment).[143] Although framed around anomaly detection, the operational goal aligns with failure prediction: catch the system early enough to intervene. This line suggests a promising integration point: LLM-based detectors could be paired with decision policies (rule-based or learned) that explicitly optimize lead time, cost, and risk.

The current failure prediction literature using LLMs is still sparse and heterogeneous. Most works either (i) reuse LLM representations for discriminative early warning, or (ii) exploit instruction-following to generate human-readable failure explanations. A key open challenge is establishing consistent evaluation that jointly measures predictive quality, timeliness, and actionability, under realistic constraints such as log evolution, partial observability, and cross-component dependency.

6.4 Root Cause Analysis / Failure Diagnosis

6.4.1 Task definition. Root cause analysis (RCA) / Failure diagnosis aims to identify the most likely underlying cause(s) of an observed incident and provide actionable, operator-facing evidence. The goal is not only to name a suspect (e.g., component/change/event type), but also to *justify* it with grounded signals so that time-to-mitigation can be reduced.

For RCA, results support log-specific findings in three situations: runtime logs are the main evaluated input, runtime logs are an explicit part of multimodal evidence, or the reported results explicitly involve logs in a setting where log use is optional.

Given an incident context (e.g., an anomalous time window or a set of correlated sessions), inputs commonly include (i) logs from multiple components/services within a time range, and optionally (ii) service topology/dependency information, deployment/change metadata, and alerts/metrics describing the incident. The output is typically a ranked list of candidate causes (top- k suspects), together with supporting evidence such as representative templates/snippets, affected components, and (when available) an explanatory chain linking symptoms to causes. Logs are especially useful in RCA because they preserve chronological symptom evidence, error/exception semantics, component-local context, and operator-facing evidence chains that connect observations to candidate causes. The “Dataset(s) type” column in Table 8 reports whether each RCA study uses logs alone or combines logs with other operational artifacts, making the scope of multimodal inclusion explicit.

6.4.2 LLM-based Approaches for Root Cause Analysis. Table 8 summarizes LLM-based root cause analysis (RCA) and closely related failure diagnosis studies. These studies share a common goal: given runtime logs, sometimes combined with metrics/traces, code, tickets, or knowledge bases, produce an actionable RCA output such as (i) a root-cause description or reason, (ii) localization to components/configurations, and sometimes (iii) remediation guidance and supporting evidence. Across the studies within the log-specific scope, the main design differences are **(1) how the LLM is used** (prompting/ICL vs. agentic tool use vs. trained task model), **(2) what external context is integrated** (repositories, databases, SOPs, graphs, knowledge bases), and **(3) what the output is optimized for** (free-form explanation vs. structured labels/locations vs. resolutions).

Prompting and in-context learning for incident RCA. A pragmatic line treats RCA as a text understanding and summarization problem and relies on prompting/ICL to map incident artifacts to root-cause statements. Chen et al. [16] perform automatic RCA for cloud incidents by consuming multiple incident signals (e.g., error

Table 8. LLM-based root cause analysis and failure diagnosis studies.

Paper	Train?	Dataset(s) type	Output	Metric(s)	Base Model(s)
Prompting & In-Context Learning (ICL) for RCA / Diagnosis					
Chen et al. [16]	Yes	Log, traces, metrics	Explanation; category	Micro and macro F1	GPT-4
Shan et al. [145]	No	Log	Root-cause configuration location	Accuracy; FP	GPT-4
Zhang et al. [209]	No	Incident title and summary	Root cause	ROUGE-L, ROUGE-1, METEOR, GLEU, BERTScore, Nubia	GPT-3.5-turbo, GPT-4
InsightAI [28]	No	Log	Flame graph; user Q&A chatbot	F1	GPT-4o
ScalaLog [201]	No	Log	Root-cause description	F1	GPT-3.5-turbo
Agentic & Tool-Augmented RCA (Multi-step Reasoning)					
RCAgent [179]	No	Log, codebase	Root causes, solutions, evidence, responsibilities	METEOR, NUBIA, Vicuna-13B-V1.5-16K, BLEURT, BARTScore, GTE-LARGE (embedding), EmbScore	
OPENRCA [189]	No	Log, traces, metrics	Root cause reasons and components	Accuracy	Claude 3.5
TAMO [208]	Yes	Log, traces, metrics	Root cause localization; fault type classification	Acc@k; MiPr, MaPr, MiRe, MaRe, MiF1, MaF1	GPT-4
Flow-of-Action [124]	No	Log, traces, metrics	Root cause localization; fault type classification	Root cause location accuracy (LA); root cause type accuracy (TA)	GPT-4-Turbo
RCLAgent [203]	No	Metrics and distributed traces	Root cause localization	Recall@k; MRR	Claude-3.5-sonnet
MicroRCA-Agent [163]	Yes	Log, traces, metrics	Root-cause	Top@K	deepseek-v3
Knowledge / Graph / Code-Augmented Diagnosis					
ForenSix [58]	No	Log	Context extraction, data dependency graph, diagnosis	Accuracy	GPT-4o
COCA [85]	No	Log, codebase, trace	Root-cause	BLEU-4, ROUGE-1, METEOR, Semantics, Usefulness, Exact Match, Top-K	GPT-4o
AetherLog [19]	Yes	Log	Root-cause category prediction	F1	GPT-4o
General Log Diagnosis: Fault Type / Resolution / Understanding					
LogSage [190]	No	Log	Root causes; solutions	Precision, Recall, F1	GPT-4o, Deepseek V3, Claude-3.7-Sonnet
Herrmann et al. [50]	Yes	Issue description, log files, engineer communications	Root cause description	MSS	MIXTRAL-8X7B
Huang et al. [53]	Yes	Log	Fault-indicating description; fault-indicating parameter	Precision, Recall, F1	UniXcoder
LogLM [94]	Yes	Log	Log-problem-resolution	BLEU, ROUGE-1, ROUGE-2, ROUGE-L	LLaMA2-7B
LogReasoner [100]	Yes	Log	Failure type	Accuracy; Weighted-F1	Qwen2.5-1.5B
Ji et al. [62]	Yes	Log	0/1 failure detection; fault type	F1	LLaMA-2-7B
LogExpert [172]	Yes	Log, StackOverflow issues	Executable resolution steps, command filling	BLEU-4, ROUGE-L	GPT-3.5-turbo, GPT-4
KnowLog [99]	Yes	Log	Cause ranking	Accuracy; Weighted F1	BERT
AdaptiveLog [101]	Yes	Log	Failure type	Recall@k	bert, chatgpt
Taheri et al. [161]	Yes	Log	Fault classification	F1	RoBERTa, BigBird, Flan-T5
Benchmarks & Evaluation Suites					
LogEval [20]	Yes	Log	Fault types	Accuracy; F1	GPT-4

logs, stack traces, socket metrics) and outputting both explanations and categories, evaluated with F1. Zhang et al. [209] study ICL-based root causing for cloud incidents using incident titles and summaries and report a suite of generation-oriented metrics. We discuss this study as a related comparison because its evaluated inputs do not expose runtime logs. In privacy- or token-limited settings, InsightAI [28] frames RCA as interactive diagnosis over large private logs, producing operator-facing artifacts (e.g., flame graphs and Q&A-style assistance) and evaluating quality with F1. These ICL-style systems are attractive due to low engineering overhead (no task-specific training required), but their performance is often bounded by context selection, prompt robustness, and the fidelity of the provided evidence.

Agentic and tool-augmented RCA for complex systems. When RCA requires multi-step investigation (e.g., correlating signals, fetching relevant traces, querying databases, or checking code ownership), many works move from single-pass prompting to *agentic* pipelines. RCAgent [179] is representative: it uses autonomous agents and tool augmentation over logs and external artifacts (e.g., databases and code repositories) to produce root causes, solutions, evidence, and responsibilities, and evaluates with multiple semantic similarity metrics. OPENRCA [189] explicitly benchmarks the ability of agentic LLM systems to locate root causes from observability data (logs/metrics/traces), emphasizing accuracy on components and reasons. Other methods specialize in structured outputs and fine-grained localization: TAMO [208] targets cloud-native RCA with tool-assisted agents over multi-modality observations and evaluates localization via Acc@k alongside fault-type classification metrics, while Flow-of-Action [124] incorporates SOP guidance to improve location/type accuracy. RCLAgent [203] explores multi-agent recursion-of-thought over metrics and distributed traces, reporting ranking-style metrics (Recall@k, MRR). It is discussed as a related comparison because runtime logs are not an explicit evaluated input. MicroRCA-Agent [163] further integrates an anomaly-identification stage (training an Isolation Forest) before agentic RCA, reflecting a common practical pattern: *small models for fast screening, agents for evidence-driven explanation and attribution*. Overall, agentic approaches tend to improve completeness and traceability of RCA, but introduce added latency/cost and require careful guardrails to avoid brittle tool use or overconfident narratives.

Knowledge-, graph-, and code-augmented RCA. Another prominent direction strengthens RCA by explicitly modeling dependencies and injecting structured knowledge. ForenSiX [58] combines context extraction with a data dependency graph for network diagnostics and reports accuracy, illustrating how graph structure can constrain reasoning and improve interpretability. COCA [85] augments RCA with code knowledge (logs + codebase + stack traces) to produce root-cause outputs and evaluates with both lexical and human-centric measures, reflecting the intuition that many failures are only disambiguated with code-level context. AetherLog [19] integrates knowledge graphs with LLMs for log-based RCA category prediction, evaluated with F1. These systems highlight that, beyond raw language ability, *external structure* (code, graphs, KBs) is a key ingredient for reliable RCA—especially when symptoms are ambiguous or multiple plausible causes exist.

RCA-adjacent diagnosis: fault types, resolutions, and log understanding. Several studies expand beyond pinpointing a cause and instead aim to complete the diagnosis loop: identifying fault types, extracting fault-indicating evidence, or generating resolutions. Huang et al. [53] extract fault-indicating descriptions and parameters from logs (precision/recall/F1), which can be viewed as a complementary capability that improves downstream RCA quality by distilling salient evidence. LogExpert [172] focuses on generating executable resolution steps grounded in external knowledge (StackOverflow), while LogLM [94] frames log analysis as instruction-following and outputs problem resolutions, evaluated with text generation metrics. For broader diagnostic labeling, LogReasoner [100] predicts failure types, and Ji et al. [62] adapt LLMs to log analysis with interpretable domain knowledge for failure detection and fault typing (F1). At the representation level, KnowLog [99] enhances log understanding and supports cause ranking, and AdaptiveLog [101] emphasizes collaboration between small and large models for failure-type identification (Recall@k). In telecom diagnostics, Taheri et al. [161] study domain-tailored models for log mask prediction used for fault classification (F1), reinforcing the importance of

domain adaptation in operational contexts. Finally, in less standardized environments, Herrmann et al. [50] show a case study for robotics support tickets (including logs and human communications), indicating RCA often spans heterogeneous, partially structured evidence beyond pure log streams.

Benchmarks and evaluation: what “good RCA” means. As methods diversify, benchmark efforts help clarify evaluation targets. LogEval [20] provides a broader suite for LLM-based log analysis (including fault types), using accuracy/F1, and complements RCA-focused studies by stressing comparability and coverage. Across the table, evaluation remains fragmented. Incident RCA often uses semantic similarity metrics. Localization prefers rank/Acc@k. Categorization uses accuracy/F1, and resolution generation uses ROUGE/BLEU. This metric diversity reflects different operational definitions of RCA (explanation vs. localization vs. remediation), and suggests that future work may benefit from task-specific, operator-aligned evaluation protocols.

Across RCA studies within the log-specific scope, LLMs matter less as standalone “reasoners” and more as components that unify heterogeneous evidence (logs, metrics, traces, code, and KBs) into a structured hypothesis with supporting traces. The most reliable designs constrain generation through retrieval, tools, SOPs, or graphs. This shifts the core challenge from modeling to *evidence selection, grounding, and verifiable outputs*, and makes hybrid, tool-augmented pipelines a practical pattern.

6.5 Log Summarization

6.5.1 Task definition. **Log summarization** compresses high-volume logs into a concise, human-consumable artifact that helps engineers quickly understand *what happened* and *what matters*. It is commonly used for on-call handoff, incident reports, and rapid situational awareness, where reading raw logs is too costly.

Given a set/sequence of logs associated with a session/window (often an incident window), the task outputs a natural-language summary and/or a compact structured summary (e.g., a short timeline of key events, prominent error templates, involved components, and extracted entities such as IDs/paths). Two common instantiations are: *extractive* summarization (select representative lines/templates) and *abstractive* summarization (generate a narrative description), often combined with grouping-by-time/component to keep summaries faithful and easy to scan.

6.5.2 LLM-based Approaches for Log Summarization. Compared with anomaly detection and RCA, log summarization is one of the most *LLM-native* log analysis tasks. Its primary deliverable is a human-readable artifact, and usefulness depends on semantic coherence, correct attribution, and actionable compression instead of only classification accuracy. Log summarization differs from generic text summarization because logs are noisy, temporally dense, uneven in informativeness, and useful only when the summary remains faithful to concrete evidence. Across the collected studies, LLM-based log summarization typically follows a “*structure-then-generate*” pattern: (i) pre-organize raw logs (by time, component, severity, or templates), (ii) select salient evidence (representative lines, errors, entities), and then (iii) generate an abstractive narrative or a structured synopsis (timeline, key events, involved modules), often with faithfulness constraints.

Benchmarks and evaluation: summarization as a first-class log task. LogEval [20] explicitly includes *log summarization* in a broader benchmark suite for LLM-based log analysis. It evaluates summary quality using both lexical overlap (e.g., ROUGE) and task-oriented correctness signals (e.g., accuracy/F1-style measures), and compares strong proprietary models against a wide range of open-source chat/instruction models. This benchmark perspective highlights two recurring issues in summarization: (i) *metric mismatch*—ROUGE-like scores only partially reflect incident usefulness, and (ii) *context limitation*—performance depends heavily on which log subset is provided to the model, motivating retrieval or guided selection before generation.

Guided enhancement assistants: making summaries faithful and operator-friendly. Practical summarization assistants often aim to reduce hallucinations and improve scanability by enforcing intermediate structure. logSage [96] (SAC’25) exemplifies this direction by treating summarization as an assistant workflow with guided enhancement: instead of asking an LLM to narrate raw logs end-to-end, the system emphasizes staged processing (e.g., evidence selection, grouping, and controlled rewriting) so that the final summary better reflects the underlying log evidence and aligns with on-call needs (handoff, incident notes, rapid situational awareness). Such designs typically trade some linguistic freedom for higher faithfulness and better operator trust.

Instruction-based log interpretation as summarization. Some works position summarization as part of a unified *instruction-following* interface for log analysis. LogLM [94] transforms multiple log tasks into instruction-based interactions. For summarization-like requests, it outputs natural-language interpretations and resolutions, and evaluates with standard generation metrics (BLEU/ROUGE variants). We discuss its summarization assignment as a related comparison because the evaluated output interprets or resolves individual log messages and does not compress a log set, sequence, session, or window. Similarly, Ji et al. [62] emphasize interpretable domain knowledge to adapt LLMs for log analysis. Their “interpretation” outputs overlap with summarization in practice by explaining what the logs indicate, and are assessed via manual scoring, reflecting that human utility and correctness are hard to capture with automatic metrics alone. This line suggests that in real deployments, summarization is rarely an isolated task. It often co-occurs with Q&A, diagnosis hints, and “what should I do next” guidance.

Tooling for scalable log processing and its relationship to summarization. While not a summarization method per se, LogLead [107] illustrates an important enabling trend: scalable pipelines that load, enhance, and process logs efficiently can serve as the *front-end* to LLM summarization by curating what the model sees (e.g., enhancing logs, selecting windows, and organizing content). In other words, summarization quality is frequently bounded by upstream log preparation and selection instead of purely by the decoder’s generation capability.

Across LLM-based log summarization studies, the key contribution of LLMs is turning noisy, heterogeneous log streams into coherent narratives and structured incident briefs. The most effective systems do not summarize “raw logs” directly. They first impose structure through grouping, salience selection, and evidence extraction, and then generate a constrained summary. This shifts the main challenge from generation to *faithful context curation and evaluation* under operator-centric criteria.

7 Cross-Cutting Insights and Future Directions

The preceding sections reviewed LLM-based work across the log-analysis pipeline, from logging statement generation to parsing and downstream operational tasks. Our findings suggest that LLM4Log should not be understood as a collection of isolated task-specific upgrades. Instead, the same challenges appear across tasks: semantic flexibility versus controllability, richer evidence integration versus higher cost and latency, and more operator-friendly outputs versus a greater need for grounding and verification. This section summarizes these recurring patterns and highlights the most important open directions for reliable real-world adoption.

7.1 Cross-cutting Insights and Trade-offs

Table 9 summarizes the comparison by linking recurring challenges to the main method families and their practical use. In practice, this means no single paradigm dominates across the whole workflow. Different tasks and constraints favor different combinations of supervision, retrieval, control, and system design.

The first cross-cutting observation is that *log-centric* LLM systems are distinct from broader LLM4AIOps settings because logs are both semantically rich and operationally noisy. Unlike metrics or traces, logs often contain the chronological symptom evidence that operators read directly during debugging and incident triage,

but they also mix free-form descriptions with identifiers, stack frames, configuration keys, and other code-like tokens. This makes log analysis not only a pattern-recognition problem, but also an evidence-integration and decision-support problem where the quality of the final operator-facing output matters as much as the underlying prediction [20, 42, 68].

The second recurring insight is that LLMs are most beneficial when the main challenge is semantic variation under drift, not stable repeated syntax. In logging-related tasks, this appears when developers need help generating or revising informative logging statements across evolving codebases and conventions [36, 87, 139, 219]. In parsing, it appears when template boundaries are ambiguous, delimiters are unstable, or previously unseen messages emerge under version and deployment drift [6, 64, 103, 188]. In downstream tasks, it appears when semantically related failures surface through heterogeneous wording across components, or when weak signals must be integrated across long sequences instead of being recognized from a single fixed pattern [20, 42, 68, 100]. Across these settings, the main value of LLMs is not that they eliminate preprocessing, but that they tolerate variation better than approaches tightly coupled to historically observed surface forms [68, 103, 188].

The third pattern is that successful LLM4Log systems rarely rely on unconstrained, end-to-end generation alone. Across tasks, the most practical designs are selective and layered: they first reduce the search space through preprocessing, retrieval, filtering, grouping, or lightweight scoring, and only then use a stronger LLM for semantic interpretation, explanation, or reporting [42, 98, 100, 103]. This pattern is visible in parsing researches that combine grouping and cache reuse with selective LLM calls, in anomaly detection pipelines that let smaller encoders or classical models narrow candidate windows before explanation, and in RCA or summarization systems that retrieve historical incidents, runbooks, or correlated artifacts before generating hypotheses or reports [20, 42, 98, 100, 103]. The broader lesson is that LLMs are most useful as a high-value reasoning and explanation component inside a larger pipeline. They are not a drop-in replacement for every stage [20, 42, 68].

These observations also clarify the main trade-offs across current LLM4Log paradigms. Fine-tuning is attractive when task boundaries are stable and enough supervision exists, but it is harder to refresh under rapid drift [43, 120]. Prompting and ICL support quick adaptation, yet they shift the burden toward prompt engineering and context management [11, 91, 113]. Retrieval-augmented approaches often provide a better balance for log-centric workflows because system-specific knowledge can be refreshed without retraining, but this depends on having a trustworthy retrieval layer [20, 42, 79, 98]. Agentic workflows extend this flexibility further, although the added coverage comes with higher latency, system complexity, and more opportunities for cascading errors [100, 111, 194]. Across the corpus, the most deployment-friendly compromise is often a hybrid design in which smaller models or structured filters handle routine narrowing, while a stronger LLM is invoked selectively for ambiguous cases, explanation, or cross-source analysis [68, 98, 100]. In comparative terms, fine-tuning offers the strongest task-specific stability when labels and task definitions are reliable, but it is the least convenient to refresh under rapid drift. Prompting and ICL are easier to adapt quickly, but they are more sensitive to prompt construction and context quality. Retrieval-augmented methods improve freshness and grounding by injecting system-specific evidence at inference time, but they depend on robust indexing and retrieval quality. Agentic or hybrid designs are the most capable of integrating multi-step evidence across tools and artifacts, but they also carry the highest latency, coordination overhead, and error-propagation risk.

The ratings in Table 9 are design-level comparisons, not measured cross-paper scores. We assess expected runtime cost from the number of large-model inference steps on the routine path: low indicates that a large model is invoked only for selected cases, medium indicates one main inference step per case, and high indicates repeated or multi-step model and tool calls. Training, index-maintenance, and other update costs are excluded from the runtime rating and noted qualitatively where relevant because the reviewed studies do not report them consistently. Drift adaptability is low when accommodating new log formats or system knowledge generally requires new labels or weight updates, medium when prompts, examples, or workflow settings can be changed without retraining, and high when system-specific evidence can be refreshed through an external index or routing

layer. Grounding support is low when outputs rely mainly on the prompt and model parameters, medium when structured context or intermediate checks are available, and high when outputs can be explicitly linked to retrieved records or logged tool evidence. Compound ratings indicate that the level depends on the implementation. The comparison assumes a comparable model scale and workload and does not rank every paper within a paradigm.

The final cross-task insight concerns operator alignment. Several downstream tasks, especially RCA and log summarization, require outputs that humans can inspect, challenge, and act on. This creates a stronger requirement than predictive accuracy alone. A detector that produces a strong F1 score but cannot explain which evidence matters may still have limited operational value, whereas an RCA assistant or summarizer that produces plausible but weakly augmented narratives can actively mislead users. Accordingly, many of the most promising LLM4Log directions couple generation with structured extraction, retrieved evidence, tool support, or schema-constrained outputs so that the resulting artifacts remain inspectable and verifiable instead of purely free-form [20, 68, 98, 100].

7.2 Open Challenges and Future Directions

Despite rapid progress, the current LLM4Log literature still exhibits substantial limitations that make direct cross-paper comparison and real-world translation difficult. A central issue is comparability. Many papers report the same headline metrics, but the underlying setups often differ in ways that materially affect results: datasets differ, sessionization or windowing strategies vary, some pipelines depend on parsing while others operate on raw or lightly normalized text, anomaly ratios are inconsistent, and prompt or model settings are frequently under-reported [6, 20, 68]. These differences make it difficult to compare performance numbers across studies. For example, two papers may both report F1, AUC, ROUGE, or Acc@k, but those scores may be based on different label definitions, windowing or grouping strategies, anomaly ratios, parsing choices, prompt templates, model versions, decoding settings, or evaluation targets. These differences also make performance claims harder to interpret. A label may define an anomaly, root cause, or correct answer differently across datasets. Windowing and grouping can change what evidence is available to the model and can also change the class balance. In highly imbalanced anomaly-detection datasets, a single aggregate score may hide important failure cases. Similarly, results based only on public benchmarks may overstate how ready a method is for industrial use. For this reason, simple pooled rankings or broad claims about the universally best LLM4Log method would be misleading. What the literature currently supports more reliably is a pattern-level summary of recurring design choices, strengths, evaluation gaps, and failure modes, not universal leaderboard conclusions [20, 68].

The second challenge is evaluation realism. A large fraction of the literature still depends on a relatively small set of public benchmarks, especially HDFS, BGL, and a few related datasets [20, 47, 68]. These benchmarks are useful for controlled comparison, but they only partially reflect the operational realities that motivate LLM-based log analysis in practice: long causal chains, mixed-source streams, privacy constraints, rapidly evolving software versions, and the need to correlate logs with tickets, runbooks, metrics, and code [42, 102, 192]. The corpus nevertheless provides limited and uneven evidence about operational realism. Most studies remain benchmark-oriented. Some studies use industrial or proprietary data, report deployment-style measures, or include human evaluation, but these forms of evidence answer different questions and should not be read as a single maturity scale. Industrial or proprietary data can improve operational realism, but they do not establish integration into an organizational workflow. Measures such as latency and token use provide evidence about technical feasibility, but they do not show whether operators benefit. Human evaluation may be limited to annotation or expert rating and need not reflect sustained use. The task distribution also suggests that readiness is task-specific. RCA contains the clearest examples of industrial and deployment-oriented evaluation, while parsing, anomaly detection, and summarization remain largely benchmark-driven. This distribution shows why progress in one task should not be generalized across the pipeline. Hence, we assess practical readiness through three separate questions: whether a

Table 9. Design-level trade-offs among major LLM adaptation paradigms in LLM4Log.

Paradigm	Strength	Limitation	Expected runtime cost / drift adaptability / grounding support	Best fit
Fine-tuning	Stable task-specific behavior	Needs labels and is harder to refresh	Medium, excluding training and update cost / Low / Low	Stable tasks with labeled data
Prompting / ICL	Fast adaptation without retraining	Sensitive to prompts, examples, and context selection	Medium, with context-length dependence / Medium / Low-medium	Low-label and fast-changing settings
Retrieval-augmented methods	Fresh system-specific evidence	Depends on retrieval quality and index freshness	Medium, including retrieval / High with a maintained index / High when retrieved evidence is exposed	RCA, summarization, and incident support
Agentic workflows	Multi-step evidence gathering and tool use	More complex, with higher latency and error-propagation risk	High / Medium-high when tools and knowledge can be updated / Medium-high when tool traces are exposed	Complex multi-source investigation workflows
Hybrid / selective escalation	Balances efficiency and capability	Requires routing, caching, and validation design	Low-medium on the routine path / High with updated routing and external knowledge / Medium-high when evidence or validation traces are exposed	Production pipelines with cost or latency constraints

method works on representative operational data, whether it functions within a real workflow, and whether it helps the people who use its output.

The third challenge concerns grounding, privacy, and deployment practicality. Operator-facing outputs in RCA and summarization can appear coherent even when they are only weakly supported by the logs or retrieved evidence, making grounding and evidence attribution essential [20, 68, 98, 100]. At the same time, logs may contain sensitive identifiers, internal topology, or security-relevant content, which complicates direct use of remote API-based LLMs and pushes many practical designs toward self-hosted, retrieval-bounded, or hybrid architectures [31, 34, 38, 70]. These constraints are not secondary deployment details. They directly shape what kinds of LLM4Log systems are feasible in practice [31, 34, 111].

Looking ahead, several research directions appear particularly important. First, benchmark design should better reflect drift, long-tail behavior, and cross-system variation instead of relying mainly on static splits [20, 103, 188].

Second, privacy-preserving and self-hosted LLM4Log deserves more attention, including parameter-efficient adaptation, on-prem retrieval, and redaction-aware pipelines [23, 31, 34]. Third, RCA and summarization should move toward more augmented and verifiable outputs, with explicit evidence attribution and stronger evaluation of faithfulness instead of plausibility alone [20, 68, 98, 100]. Fourth, long-context compression, memory, and adaptive evidence selection remain central challenges because incident-relevant signals are often sparse and distributed across long noisy histories [42, 111, 126]. Fifth, the field needs broader industrial-scale evaluation beyond the standard public benchmarks, including richer multimodal settings and operator-centered studies [20, 102, 192]. Finally, human-in-the-loop support is likely to be more practical than full autonomy in many settings: future systems should help engineers inspect evidence faster, compare likely causes, summarize timelines, and decide where to look next while remaining transparent about uncertainty and evidence provenance [98, 102, 192].

8 Conclusion

This survey presented a systematic review of *LLM-based log analysis* across the log-analysis pipeline, covering logging statement generation and maintenance, log parsing, and downstream tasks such as anomaly detection, failure prediction, root cause analysis, and log summarization. Across these areas, the literature shows that LLMs offer clear advantages in semantic generalization, evidence integration, and operator-facing assistance, while also introducing important challenges in evaluation, grounding, privacy, cost, and deployment practicality. As discussed in Section 7, progress in LLM4Log will depend not only on stronger models, but also on more rigorous cross-paper evaluation, better grounding and verification for operator-facing outputs, and system designs that remain reliable under real-world operational constraints.

References

- [1] Siraaj Akhtar, Saad Khan, and Simon Parkinson. Llm-based event log analysis techniques: A survey. *arXiv preprint arXiv:2502.00677*, 2025.
- [2] Crispin Almodovar, Fariza Sabrina, Sarvnaz Karimi, and Salahuddin Azad. Logfit: Log anomaly detection using fine-tuned language models. *IEEE Transactions on Network and Service Management*, 21(2):1715–1723, 2024.
- [3] Apache Software Foundation. Apache log4j 2. Online documentation, 2024. URL <https://logging.apache.org/log4j/2.x/>.
- [4] Merve Astekin, Max Hort, and Leon Moonen. A comparative study on large language models for log parsing. In *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM 2024)*, pages 36–47. ACM, 2024. doi: 10.1145/3674805.3686684. URL <https://doi.org/10.1145/3674805.3686684>.
- [5] Prasasthy Balasubramanian, Dumindu Kankanamge, Ekaterina Gilman, and Mourad Oussalah. AnomalyExplainerBot: Explainable AI for LLM-based anomaly detection using BERTViz & Captum, 2025.
- [6] Viktor Beck, Max Landauer, Markus Wurzenberger, Florian Skopik, and Andreas Rauber. System log parsing with large language models: A review. *arXiv preprint arXiv:2504.04877*, 2025.
- [7] Sidahmed Benabderrahmane, Petko Valtchev, James Cheney, and Talal Rahwan. APT-LLM: Embedding-based anomaly detection of cyber advanced persistent threats using large language models. In *Proceedings of the 13th International Symposium on Digital Forensics and Security (ISDFS 2025)*, pages 1–6, 2025. doi: 10.1109/ISDFS65363.2025.11011912.
- [8] Jasmin Bogatinovski and Odej Kao. Auto-logging: Ai-centred logging instrumentation. In *2023 IEEE/ACM 45th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, pages 95–100, 2023. doi: 10.1109/ICSE-NIER58687.2023.00023. URL <https://doi.org/10.1109/ICSE-NIER58687.2023.00023>.
- [9] Bogdan Bogdan, Arina Cazacu, and Laura Vasilie. Good enough to learn: LLM-based anomaly detection in ECU logs without reliable labels, 2025. URL <https://arxiv.org/abs/2507.01077>. Accepted to IEEE Intelligent Vehicles Symposium (IV) 2025.
- [10] Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, Erik Brynjolfsson, Shyamal Buch, Dallas Card, Rodrigo Castellon, Niladri Chatterji, Annie Chen, Kathleen Creel, Jared Quincy Davis, Jeff Dean, et al. On the opportunities and risks of foundation models, 2021. URL <https://arxiv.org/abs/2108.07258>.
- [11] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.

- [12] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2633–2650, 2021. URL <https://www.usenix.org/conference/usenixsecurity21/presentation/carlini-extracting>.
- [13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [14] Song Chen and Hai Liao. Bert-log: Anomaly detection for system logs based on pre-trained language model. *Applied Artificial Intelligence*, 36(1):2145642, 2022. doi: 10.1080/08839514.2022.2145642. URL <https://www.tandfonline.com/doi/full/10.1080/08839514.2022.2145642>.
- [15] Xiaolei Chen, Jie Shi, Jia Chen, Peng Wang, and Wei Wang. Epas: Efficient online log parsing via asynchronous scheduling of llm queries. In *Proceedings of the 41st IEEE International Conference on Data Engineering (ICDE 2025)*, pages 4025–4037. IEEE, 2025. doi: 10.1109/ICDE.2025.00318. URL <https://ieeexplore.ieee.org/abstract/document/11113127>.
- [16] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, et al. Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pages 674–688, 2024.
- [17] Kevin Clark, Minh-Thang Luong, Quoc V Le, and Christopher D Manning. Electra: Pre-training text encoders as discriminators rather than generators. arXiv preprint arXiv:2003.10555, 2020.
- [18] Clara Corbelle, Victor Carneiro, and Fidel Cacheda. Semantic hierarchical classification applied to anomaly detection using system logs with a bert model. *Applied Sciences*, 14(13), 2024. ISSN 2076-3417. doi: 10.3390/app14135388. URL <https://www.mdpi.com/2076-3417/14/13/5388>.
- [19] Tianyu Cui, Ruowei Fu, Changchang Liu, Yuhe Ji, Wenwei Gu, Shenglin Zhang, Yongqian Sun, and Dan Pei. Aetherlog: Log-based root cause analysis by integrating large language models with knowledge graphs. In *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, pages 49–60. IEEE, 2025.
- [20] Tianyu Cui, Shiyu Ma, Ziang Chen, Tong Xiao, Chenyu Zhao, Shimin Tao, Yilun Liu, Shenglin Zhang, Duoming Lin, Changchang Liu, Yuzhe Cai, Weibin Meng, Yongqian Sun, and Dan Pei. Logeval: A comprehensive benchmark suite for llms in log analysis. *Empirical Softw. Engg.*, 30(6), October 2025. ISSN 1382-3256. doi: 10.1007/s10664-025-10701-6. URL <https://doi.org/10.1007/s10664-025-10701-6>.
- [21] Hetong Dai, Heng Li, Che-Shao Chen, Weiyi Shang, and Tse-Hsun Chen. Logram: Efficient log parsing using n -gram dictionaries. *IEEE Transactions on Software Engineering*, 48(3):879–892, 2020.
- [22] Delgan and contributors. loguru: Python logging made simple. Online documentation, 2024. URL <https://github.com/Delgan/loguru>.
- [23] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms, 2023. URL <https://arxiv.org/abs/2305.14314>.
- [24] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [25] Shengcheng Duan, Yihua Xu, Sheng Zhang, Shen Wang, and Yue Duan. Pdlogger: Automated logging framework for practical software development, 2025. URL <https://arxiv.org/abs/2507.19951>.
- [26] Yiqi Duan, Jianliang Xu, Changyu Fan, and Zixin Liu. Unsupervised log parsing based on large language models and entropy. In *2025 11th International Symposium on System Security, Safety, and Reliability (ISSSR)*, pages 1–10. IEEE, 2025.
- [27] Chris Egersdoerfer, Di Zhang, and Dong Dai. Early exploration of using ChatGPT for log-based anomaly detection on parallel file systems logs. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC '23)*, pages 315–316. Association for Computing Machinery, 2023. doi: 10.1145/3588195.3595943. URL <https://doi.org/10.1145/3588195.3595943>.
- [28] Maryam Ekhlesi, Anurag Prakash, Maxime Lamothe, and Michel Dagenais. Insightai: Root cause analysis in large log files with private data using large language model. In *2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN)*, pages 31–41, 2025. doi: 10.1109/CAIN66642.2025.00012.
- [29] Mahmoudreza Entezami, Shahabeddin Rahimi Harsini, David Houshang, and Zahra Entezami. A novel framework for detecting anomalies in network security using LLM and deep learning. *Journal of Electrical Systems*, 21(1s):294–302, 2025. doi: 10.52783/jes.8791.
- [30] Kavin Ethayarajh. How contextual are contextualized word representations? comparing the geometry of BERT, ELMo, and GPT-2 embeddings. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 55–65, Hong Kong, China, November 2019. Association for Computational Linguistics. doi:

- 10.18653/v1/D19-1006.
- [31] European Parliament and Council of the European Union. Regulation (eu) 2016/679 of the european parliament and of the council (general data protection regulation). Official Journal of the European Union, L119, 2016. URL <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
 - [32] Asma Fariha, Vida Gharavian, Masoud Makrehchi, Shahryar Rahnamayan, Sanaa Alwidian, and Akramul Azim. Log anomaly detection by leveraging llm-based parsing and embedding with attention mechanism. In *2024 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE)*, pages 859–863. IEEE, 2024.
 - [33] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.findings-emnlp.139. URL <https://aclanthology.org/2020.findings-emnlp.139/>.
 - [34] Joint Task Force. Security and privacy controls for information systems and organizations (nist special publication 800-53 revision 5), 2020. URL <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>.
 - [35] Amy Foster and Selva Kumar. *COMBINING LLMS AND SHELL LOGS TO PREDICT BACKUP FAILURES*. 07 2025.
 - [36] Qiang Fu, Jieming Zhu, Wenlu Hu, Jian-Guang Lou, Rui Ding, Qingwei Lin, Dongmei Zhang, and Tao Xie. Where do developers log? an empirical study on logging practices in industry. In *Companion Proceedings of the 36th International Conference on Software Engineering*, pages 24–33. ACM, 2014. doi: 10.1145/2591062.2591175.
 - [37] Ying Fu, Meng Yan, Pinjia He, Chao Liu, Xiaohong Zhang, and Dan Yang. End-to-end log statement generation at block-level. *Journal of Systems and Software*, 216:112146, 2024. doi: 10.1016/j.jss.2024.112146.
 - [38] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christopher Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, 2023. URL <https://arxiv.org/abs/2302.12173>.
 - [39] Wei Guan, Jian Cao, Shiyu Qian, and Jianqi Gao. LogLLM: Log-based anomaly detection using large language models. *arXiv preprint*, 2024. URL <https://arxiv.org/abs/2411.08561>.
 - [40] H. Guo, S. Yuan, and X. Wu. Logbert: Log anomaly detection via bert. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2021.
 - [41] Hongcheng Guo, Jian Yang, Jiaheng Liu, Jiaqi Bai, Boyang Wang, Zhoujun Li, Tiejiao Zheng, Bo Zhang, Junran Peng, and Qi Tian. Logformer: A pre-train and tuning pipeline for log anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, pages 135–143, 2024.
 - [42] Hongcheng Guo, Jian Yang, Jiaheng Liu, Liqun Yang, Linzheng Chai, Jiaqi Bai, Junran Peng, Xiaorong Hu, Chao Chen, Dongfeng Zhang, Xu Shi, Tiejiao Zheng, Liangfan Zheng, Bo Zhang, Ke Xu, and Zhoujun Li. Owl: A large language model for it operations. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL <https://openreview.net/forum?id=SZOQ9RKJJu>.
 - [43] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8342–8360. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.740. URL <https://aclanthology.org/2020.acl-main.740/>.
 - [44] Fatemeh Hadadi, Qinghua Xu, Domenico Bianculli, and Lionel Briand. Llm meets ml: Data-efficient anomaly detection on unstable logs. *ACM Transactions on Software Engineering and Methodology*, 2025. doi: 10.1145/3771283. URL <https://doi.org/10.1145/3771283>. arXiv:2406.07467.
 - [45] Minghua He, Tong Jia, Chiming Duan, Huaqian Cai, Ying Li, and Gang Huang. Llmelog: An approach for anomaly detection based on llm-enriched log events. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, pages 132–143. IEEE, 2024.
 - [46] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R. Lyu. Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE International Conference on Web Services (ICWS)*, pages 33–40, 2017. doi: 10.1109/ICWS.2017.13.
 - [47] Shilin He, Pinjia He, Zhuangbin Chen, Tianyi Yang, Yuxin Su, and Michael R Lyu. A survey on automated log analysis for reliability engineering. *ACM computing surveys (CSUR)*, 54(6):1–37, 2021.
 - [48] Shiming He, Ying Lei, Ying Zhang, Kun Xie, and Pradip Kumar Sharma. Parameter-efficient log anomaly detection based on pre-training model and lora. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pages 207–217. IEEE, 2023.
 - [49] Yi Wen Heng, Zeyang Ma, Zhenhao Li, Dong Jae Kim, and Tse-Hsun Chen. Benchmarking open-source large language models for log level suggestion. In *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*, pages 314–325, 2025. doi: 10.1109/ICST62969.2025.10988921.
 - [50] Jordis Emilia Herrmann, Aswath Mandakath Gopinath, Mikael Norrlof, and Mark Niklas Mueller. Diagnosing robotics systems issues with large language models—a case study. In *ICLR 2025 Workshop on Foundation Models in the Wild*.
 - [51] András Horváth, András Oláh, Attila Pintér, Bálint Siklósi, Gergely Lukács, István Z. Regulý, Kálmán Tornai, Tamás Zsedrovits, and Zoltán Máthé. Anomaly detection algorithms for real-time log data analysis at scale. *IEEE Access*, 13:136288–136311, 2025. doi:

- 10.1109/ACCESS.2025.3565575.
- [52] Changze Hu, Yu Fang, Jinhua Wu, Haoyang Li, and Geng Wang. Research on log anomaly detection based on sentence-BERT. *Electronics*, 12(17):3580, 2023. doi: 10.3390/electronics12173580. URL <https://www.mdpi.com/2079-9292/12/17/3580>.
 - [53] Junjie Huang, Zhihan Jiang, Jinyang Liu, Yintong Huo, Jiazhen Gu, Zhuangbin Chen, Cong Feng, Hui Dong, Zengyin Yang, and Michael R Lyu. Demystifying and extracting fault-indicating information from logs for failure diagnosis. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, pages 511–522. IEEE, 2024.
 - [54] Junjie Huang, Zhihan Jiang, Zhuangbin Chen, and Michael R. Lyu. No more labelled examples? an unsupervised log parser with llms. In *Proceedings of the ACM on Software Engineering (FSE 2025)*, number FSE. ACM, 2025. doi: 10.1145/3729377. URL <https://dl.acm.org/doi/10.1145/3729377>.
 - [55] Shaohan Huang, Yi Liu, Carol J. Fung, Rong He, Yining Zhao, Hailong Yang, and Zhongzhi Luan. Hitanomaly: Hierarchical transformers for anomaly detection in system log. *IEEE Transactions on Network and Service Management*, 17(2):2064–2076, 2020. doi: 10.1109/TNSM.2020.3034647.
 - [56] Shaohan Huang, Yi Liu, Carol J. Fung, He Wang, Hailong Yang, and Zhongzhi Luan. Improving log-based anomaly detection by pre-training hierarchical transformers. *IEEE Transactions on Computers*, 72(9):2656–2667, 2023. doi: 10.1109/TC.2023.3257518.
 - [57] Xin Huang, Ting Zhang, and Wen Zhao. LogRules: Enhancing log analysis capability of large language models through rules. In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 452–470, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-195-7. doi: 10.18653/v1/2025.findings-naacl.28. URL <https://aclanthology.org/2025.findings-naacl.28/>.
 - [58] Xuanbo Huang, Kaiping Xue, Lutong Chen, Jiangping Han, Jian Li, and David S. L. Wei. Forensix: Automated network forensics and diagnostics for beyond-5g and 6g networks using large language models. *IEEE Network*, 39(5):74–80, 2025. doi: 10.1109/MNET.2025.3579925.
 - [59] Yintong Huo, Cheryl Lee, Yuxin Su, Shiwen Shan, Jinyang Liu, and Michael R. Lyu. Evlog: Evolving log analyzer for anomalous logs identification. arXiv preprint arXiv:2306.01509, 2023.
 - [60] Jaeyoon Jeong, Insung Baek, Byungwoo Bang, Junyeon Lee, Uiseok Song, and Seoung Bum Kim. Fall: Prior failure detection in large scale system based on language model. *IEEE Transactions on Dependable and Secure Computing*, 22(1):279–291, 2025. doi: 10.1109/TDSC.2024.3396166.
 - [61] Xin Ji, Le Zhang, Wenya Zhang, Fang Peng, Yifan Mao, Xingchuang Liao, and Kui Zhang. Lemad: LLM-empowered multi-agent system for anomaly detection in power grid services. *Electronics*, 14(15):3008, 2025. doi: 10.3390/electronics14153008. URL <https://doi.org/10.3390/electronics14153008>.
 - [62] Yuhe Ji, Yilun Liu, Feiyu Yao, Mingui He, Shimin Tao, Xiaofeng Zhao, Chang Su, Xinhua Yang, Weibin Meng, Yuming Xie, Boxing Chen, Shenglin Zhang, and Yongqian Sun. Adapting large language models to log analysis with interpretable domain knowledge. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management, CIKM '25*, page 1135–1144, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400720406. doi: 10.1145/3746252.3761189. URL <https://doi.org/10.1145/3746252.3761189>.
 - [63] Ziwei Ji, Nayeon Lee, Rita Frieske, Tianle Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation, 2023. URL <https://arxiv.org/abs/2304.04710>.
 - [64] Zhihan Jiang, Jinyang Liu, Zhuangbin Chen, Yichen Li, Junjie Huang, Yintong Huo, Pinjia He, Jiazhen Gu, and Michael R. Lyu. Lilac: Log parsing using llms with adaptive parsing cache. *Proceedings of the ACM on Software Engineering*, 1(FSE), 2024. doi: 10.1145/3643733. URL <https://doi.org/10.1145/3643733>.
 - [65] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, Dawn Drain, Nelson Chen, Yuntao Bai, Jared Kaplan, Sam McCandlish, Dario Amodei, Ethan Chen, and Catherine Olsson. Language models (mostly) know what they know, 2022. URL <https://arxiv.org/abs/2207.05221>.
 - [66] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. arXiv preprint arXiv:2001.08361, 2020. URL <https://arxiv.org/abs/2001.08361>.
 - [67] Crystal Karlsen, Denis Copstein, Yue Luo, Benjamin Schwartzentruber, Tim Niblett, and Olivier Rouyer. Exploring semantic vs. syntactic features for unsupervised learning on application log files. In *2023 International Conference on Cyber Security and Networks (CSNet)*, 2023. doi: 10.1109/CSNet59123.2023.10339765.
 - [68] Egil Karlsen, Xiao Luo, Nur Zincir-Heywood, and Malcolm Heywood. Benchmarking large language models for log analysis, security, and interpretation. *Journal of Network and Systems Management*, 32:59, 2024. doi: 10.1007/s10922-024-09831-x.
 - [69] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.550. URL <https://aclanthology.org/2020.emnlp-main.550/>.
 - [70] Karen Kent and Murugiah Souppaya. Nist special publication 800-92: Guide to computer security log management, 2006. URL <https://csrc.nist.gov/publications/detail/sp/800-92/final>.

- [71] Van-Hoang Le and Hongyu Zhang. Log-based anomaly detection without log parsing. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE/ACM, 2021.
- [72] Van-Hoang Le and Hongyu Zhang. Log parsing with prompt-based few-shot learning. In *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023)*, pages 2438–2449. IEEE, 2023. doi: 10.1109/ICSE48619.2023.00204. URL <https://conf.researchr.org/details/icse-2023/icse-2023-technical-track/165/Log-Parsing-with-Prompt-based-Few-shot-Learning>.
- [73] Van-Hoang Le and Hongyu Zhang. Log parsing: How far can chatgpt go? In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering, ASE '23*, page 1699–1704. IEEE Press, 2024. ISBN 9798350329964. doi: 10.1109/ASE56229.2023.00206. URL <https://doi.org/10.1109/ASE56229.2023.00206>.
- [74] Van-Hoang Le and Hongyu Zhang. Prelog: A pre-trained model for log analytics. *Proceedings of the ACM on Management of Data*, 2(3): 1–28, 2024. doi: 10.1145/3654966. URL <https://dl.acm.org/doi/10.1145/3654966>. Presented at SIGMOD 2024.
- [75] Van-Hoang Le, Yi Xiao, and Hongyu Zhang. Unleashing the true potential of semantic-based log parsing with pre-trained language models. In *Proceedings of the 47th International Conference on Software Engineering (ICSE 2025)*. IEEE/ACM, 2025. URL <https://conf.researchr.org/details/icse-2025/icse-2025-research-track/80/Unleashing-the-True-Potential-of-Semantic-based-Log-Parsing-with-Pre-trained-Language>. To appear.
- [76] Juyoung Lee, Yeonsu Jeong, Taehyun Han, and Taejin Lee. Logresp-agent: A recursive ai framework for context-aware log anomaly detection and ttp analysis. *Applied Sciences*, 15(13), 2025. ISSN 2076-3417. doi: 10.3390/app15137237. URL <https://www.mdpi.com/2076-3417/15/13/7237>.
- [77] Yukyung Lee, Jina Kim, and Pilsung Kang. Lanobert: System log anomaly detection based on bert masked language model, 2021. URL <https://arxiv.org/abs/2111.09564>.
- [78] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 7871–7880, 2020.
- [79] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2020. URL <https://arxiv.org/abs/2005.11401>.
- [80] Xiaoyun Li, Pengfei Chen, Linxiao Jing, Zilong He, and Guangba Yu. Swisslog: Robust and unified deep learning based log anomaly detection for diverse faults. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, pages 92–103. IEEE, 2020.
- [81] Xinyu Li, Yingtong Huo, Chenxi Mao, Shiwen Shan, Yuxin Su, Dan Li, and Zibin Zheng. AnomalyGen: An automated semantic log sequence generation framework with LLM for anomaly detection, 2025. URL <https://arxiv.org/abs/2504.12250>.
- [82] Yichen Li, Yintong Huo, Zhihan Jiang, Renyi Zhong, Pinjia He, Yuxin Su, Lionel C Briand, and Michael R Lyu. Exploring the effectiveness of llms in automated logging statement generation: An empirical study. *IEEE Transactions on Software Engineering*, 2024.
- [83] Yichen Li, Yintong Huo, Renyi Zhong, Zhihan Jiang, Jinyang Liu, Junjie Huang, Jiazhen Gu, Pinjia He, and Michael R. Lyu. Go static: Contextualized logging statement generation. *Proceedings of the ACM on Software Engineering*, 1(FSE):609–630, 2024. doi: 10.1145/3643754. URL <https://doi.org/10.1145/3643754>.
- [84] Yichen Li, Jinyang Liu, Junsong Pu, Zhihan Jiang, Zhuangbin Chen, Xiao He, Tieying Zhang, Jianjun Chen, Yi Li, Rui Shi, and Michael Lyu. Automated proactive logging quality improvement for large-scale codebases. In *Proceedings of the 2025 IEEE/ACM International Conference on Automated Software Engineering (ASE 2025) (Industry Showcase)*, 2025. URL <https://conf.researchr.org/details/ase-2025/ase-2025-industry-showcase/28/Automated-Proactive-Logging-Quality-Improvement-for-Large-Scale-Codebases>.
- [85] Yichen Li, Yulun Wu, Jinyang Liu, Zhihan Jiang, Zhuangbin Chen, Guangba Yu, and Michael R Lyu. Coca: Generative root cause analysis for distributed systems with code knowledge. *arXiv preprint arXiv:2503.23051*, 2025.
- [86] Zhenhao Li, Heng Li, Tse-Hsun Chen, and Weiyi Shang. Deeply: Suggesting log levels using ordinal based neural networks. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 1461–1472. IEEE, 2021.
- [87] Zhenhao Li, An Ran Chen, Xing Hu, Xin Xia, Tse-Hsun Chen, and Weiyi Shang. Are they all good? studying practitioners’ expectations on the readability of log messages. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE 2023)*, 2023. doi: 10.48550/arXiv.2308.08836. URL <https://arxiv.org/abs/2308.08836>.
- [88] Yifan Liao, Ming Xu, Yun Lin, Xiwen Teoh, Xiaofei Xie, Ruitao Feng, Frank Liaw, Hongyu Zhang, and Jin Song Dong. Detecting and explaining anomalies caused by web tamper attacks via building consistency-based normality. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pages 531–543, 2024.
- [89] Ying Fu Lim, Jiawen Zhu, and Guansong Pang. Adapting large language models for parameter-efficient log anomaly detection. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2503.08045>. arXiv:2503.08045.
- [90] Yifei Lin, H. Deng, and X. Li. Fastlogad: Log anomaly detection with mask-guided pseudo anomaly generation and discrimination, 2024. URL <https://arxiv.org/abs/2404.08750>.
- [91] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing, 2021. URL <https://arxiv.org/abs/2107.13586>.

- [92] Yilun Liu, Shimin Tao, Weibin Meng, Jingyu Wang, Wenbing Ma, Yuhang Chen, Yanqing Zhao, Hao Yang, and Yanfei Jiang. Interpretable online log analysis using large language models with prompt strategies. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension (ICPC '24)*, pages 35–46. Association for Computing Machinery, 2024. doi: 10.1145/3643916.3644408. URL <https://doi.org/10.1145/3643916.3644408>.
- [93] Yilun Liu, Shimin Tao, Weibin Meng, Feiyu Yao, Xingyu Zhao, and Hao Yang. LogPrompt: Prompt engineering towards zero-shot and interpretable log analysis. In *Companion Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE Companion '24)*. Association for Computing Machinery, 2024. doi: 10.1145/3639478.3643108. URL <https://doi.org/10.1145/3639478.3643108>.
- [94] Yilun Liu, Yuhe Ji, Shimin Tao, Mingguai He, Weibin Meng, Shenglin Zhang, Yongqian Sun, Yuming Xie, Boxing Chen, and Hao Yang. Loglm: From task-based to instruction-based automated log analysis. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 401–412. IEEE, 2025.
- [95] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [96] Yang Lu. *LogSage: Log Summarization Assistant with Guided Enhancement*, page 1979–1981. Association for Computing Machinery, New York, NY, USA, 2025. ISBN 9798400706295. URL <https://doi.org/10.1145/3672608.3707990>.
- [97] Wengang Lyu, Yilun Wang, Wei Wei, Runkun Xiao, Youwei Li, Gen Li, Laiyuan Yang, and Zibin Zheng. Exploring variable potential for llm-based log parsing efficiency and reduced costs. In *Proceedings of the FSE 2025 Companion*, 2025. doi: 10.1145/3696630.3728506. URL <https://doi.org/10.1145/3696630.3728506>.
- [98] Lipeng Ma, Weidong Yang, Sihang Jiang, Ben Fei, Mingjie Zhou, Shuhao Li, Mingyu Zhao, Bo Xu, and Yanghua Xiao. LUK: Empowering log understanding with expert knowledge from large language models, 2024.
- [99] Lipeng Ma, Weidong Yang, Bo Xu, Sihang Jiang, Ben Fei, Jiaqing Liang, Mingjie Zhou, and Yanghua Xiao. Knowlog: Knowledge enhanced pre-trained language model for log understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24*, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400702174. doi: 10.1145/3597503.3623304. URL <https://doi.org/10.1145/3597503.3623304>.
- [100] Lipeng Ma, Yixuan Li, Weidong Yang, Mingjie Zhou, Xinyi Liu, Ben Fei, Shuhao Li, Xiaoyan Sun, Sihang Jiang, and Yanghua Xiao. LogReasoner: Empowering LLMs with expert-like coarse-to-fine reasoning for log analysis tasks, 2025.
- [101] Lipeng Ma, Weidong Yang, Yixuan Li, Ben Fei, Mingjie Zhou, Shuhao Li, Sihang Jiang, Bo Xu, and Yanghua Xiao. Adaptivelog: An adaptive log analysis framework with the collaboration of large and small language model. *ACM Transactions on Software Engineering and Methodology*, 2025. doi: 10.1145/3749840. URL <https://doi.org/10.1145/3749840>. arXiv:2501.11031.
- [102] X. Ma, Y. Li, J. Keung, X. Yu, H. Zou, Z. Yang, F. Sarro, and E. T. Barr. Practitioners’ expectations on log anomaly detection. *IEEE Transactions on Software Engineering*, 51(9):2455–2471, 2025. doi: 10.1109/TSE.2025.3586700.
- [103] Zeyang Ma, An Ran Chen, Dong Jae Kim, Tse-Hsun Chen, and Shaowei Wang. Lmparser: An exploratory study on using large language models for log parsing. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE 2024)*, pages 99:1–99:13, 2024. doi: 10.1145/3597503.3639150. URL <https://doi.org/10.1145/3597503.3639150>.
- [104] Zeyang Ma, Dong Jae Kim, and Tse-Hsun Chen. Librelog: Accurate and efficient unsupervised log parsing using open-source large language models. In *Proceedings of the 47th International Conference on Software Engineering (ICSE 2025)*. IEEE/ACM, 2025. URL <https://conf.researchr.org/details/icse-2025/icse-2025-research-track/75/LibreLog-Accurate-and-Efficient-Unsupervised-Log-Parsing-Using-Open-Source-Lan>. To appear.
- [105] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models, 2023. URL <https://arxiv.org/abs/2303.08896>.
- [106] Praveen Kumar Mannam. Optimizing software release management with gpt-enabled log anomaly detection. In *2023 26th International Conference on Computer and Information Technology (ICCIT)*, pages 1–6. IEEE, 2023.
- [107] Mika V Mäntylä, Yuqing Wang, and Jesse Nyssölä. Loglead-fast and integrated log loader, enhancer, and anomaly detector. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 395–399. IEEE, 2024.
- [108] Antonio Mastropaolo, Luca Pascarella, and Gabriele Bavota. Using deep learning to generate complete log statements. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*, pages 2279–2290, 2022. doi: 10.1145/3510003.3511561. URL <https://doi.org/10.1145/3510003.3511561>.
- [109] Antonio Mastropaolo, Valentina Ferrari, Luca Pascarella, and Gabriele Bavota. Log statements generation via deep learning: Widening the support provided to developers. *Journal of Systems and Software*, 210:111947, 2024. doi: 10.1016/j.jss.2023.111947. URL <https://doi.org/10.1016/j.jss.2023.111947>.
- [110] Maryam Mehrabi, Abdelwahab Hamou-Lhadj, and Hossein Moosavi. The effectiveness of compact fine-tuned llms in log parsing. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 438–448. IEEE, 2024.
- [111] Grégoire Mialon, Roberto Dessi, Maria Lomeli, Christos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Damien Sileo, Lewis Tunstall, et al. Augmented language models: a survey, 2023. URL <https://arxiv.org/abs/2302.07842>.

- [112] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems*, 26, 2013.
- [113] Sewon Min, Mike Lewis, Luke Zettlemoyer, and Hannaneh Hajishirzi. Rethinking the role of demonstrations: What makes in-context learning work?, 2022. URL <https://arxiv.org/abs/2202.12837>.
- [114] Priyanka Mudgal, Bijan Arbab, and Swaathi Sampath Kumar. Crasheventllm: Predicting system crashes with large language models. In *2024 International Conference on Information Technology and Computing (ICITCOM)*, pages 72–76. IEEE, 2024.
- [115] Gunho No, Yukyung Lee, Hyeongwon Kang, and Pilsung Kang. RAPID: Training-free retrieval-based log anomaly detection with pre-trained language model considering token-level information. *Engineering Applications of Artificial Intelligence*, 133:108613, 2024. doi: 10.1016/j.engappai.2024.108613.
- [116] OpenAI. Gpt-4 technical report, 2023. URL <https://arxiv.org/abs/2303.08774>.
- [117] Oracle. Java platform, standard edition java.util.logging. Online documentation, 2024. URL <https://docs.oracle.com/en/java/javase/>.
- [118] Harold Ott, Jasmin Bogatinovski, Alexander Acker, Sasho Nedelkoski, and Odej Kao. Robust and transferable anomaly detection in log data using pre-trained language models, 2021. URL <https://arxiv.org/abs/2102.11570>.
- [119] Youssef Esseddiq Ouattiti, Mohammed Sayagh, Bram Adams, and Ahmed E. Hassan. Omnillp: Enhancing llm-based log level prediction with context-aware retrieval, 2025. URL <https://arxiv.org/abs/2508.08545>.
- [120] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. URL <https://arxiv.org/abs/2203.02155>.
- [121] Jonathan Pan, Swee Liang Wong, and Yidi Yuan. Raglog: Log anomaly detection using retrieval augmented generation, 2023. URL <https://arxiv.org/abs/2311.05261>.
- [122] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318, 2002.
- [123] Changhua Pei, Zihan Liu, Jianhui Li, Erhan Zhang, Le Zhang, Haiming Zhang, Wei Chen, Dan Pei, and Gaogang Xie. Self-evolutionary group-wise log parsing based on large language model. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, pages 49–60. IEEE, 2024.
- [124] Changhua Pei, Zexin Wang, Fengrui Liu, Zeyan Li, Yang Liu, Xiao He, Rong Kang, Tieying Zhang, Jianjun Chen, Jianhui Li, et al. Flow-of-action: Sop enhanced llm-based multi-agent system for root cause analysis. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 422–431, 2025.
- [125] Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-1202.
- [126] Reiner Pope, Yash Jain, Rohan Peri, Jeff Hankins, Baolin Lin, and Ion Stoica. Efficiently scaling transformer inference with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.
- [127] Przemek Pospieszny, Wojciech Mormul, Karolina Szyndler, and Sanjeev Kumar. ADALog: Adaptive unsupervised anomaly detection in logs with self-attention masked language model. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2505.13496>.
- [128] Python Software Foundation. Python logging — logging facility for python. Online documentation, 2024. URL <https://docs.python.org/3/library/logging.html>.
- [129] Jiaxing Qi, Shaohan Huang, Zhongzhi Luan, Shu Yang, Carol J. Fung, Hailong Yang, Depei Qian, Jing Shang, Zhiwen Xiao, and Zhihui Wu. LogGPT: Exploring ChatGPT for log-based anomaly detection. In *2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*, pages 273–280, 2023. doi: 10.1109/HPCC-DSS-SMARTCITY-DEPENDSYS60770.2023.00045. URL <https://doi.org/10.1109/HPCC-DSS-SmartCity-DependSys60770.2023.00045>.
- [130] Xipeng Qiu, Tianxiang Sun, Yige Xu, Yunfan Shao, Ning Dai, and Xuanjing Huang. Pre-trained models for natural language processing: A survey. *Science China Technological Sciences*, 63(10):1872–1897, 2020. doi: 10.1007/s11431-020-1647-3. URL <https://arxiv.org/abs/2003.08271>.
- [131] QOS.ch. Logback. Online documentation, 2024. URL <https://logback.qos.ch/>.
- [132] QOS.ch. Slf4j: Simple logging facade for java. Online documentation, 2024. URL <https://www.slf4j.org/>.
- [133] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2023. URL <https://arxiv.org/abs/2305.18290>.
- [134] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

- [135] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.
- [136] Youcef Remil, Anes Bendimerad, Romain Mathonat, and Mehdi Kaytoue. Aiops solutions for incident management: Technical guidelines and a comprehensive literature review. *arXiv preprint arXiv:2404.01363*, 2024.
- [137] Hengyi Ren, Kun Lan, Zhi Sun, and Shan Liao. Clogllm: A large language model enabled approach to cybersecurity log anomaly analysis. In *2024 4th International Conference on Electronic Information Engineering and Computer Communication (EIECC)*, pages 963–970. IEEE, 2024.
- [138] Mayra Sofia Ruiz Rodriguez, SayedHassan Khatoonabadi, and Emad Shihab. Automated file-level logging generation for machine learning applications using llms: A case study using gpt-4o mini, 2025. URL <https://arxiv.org/abs/2508.04820>.
- [139] Guoping Rong, Shenghui Gu, Haifeng Shen, He Zhang, and Hongyu Kuang. How do developers’ profiles and experiences influence their logging practices? an empirical study of industrial practitioners. In *Proceedings of the 45th International Conference on Software Engineering*, 2023.
- [140] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL <https://arxiv.org/abs/2302.04761>.
- [141] Hynek Schlawack and contributors. structlog: Structured logging for python. Online documentation, 2024. URL <https://www.structlog.org/>.
- [142] Abigail See, Peter J. Liu, and Christopher D. Manning. Get to the point: Summarization with pointer-generator networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, volume 1, pages 1073–1083, 2017. doi: 10.18653/v1/P17-1099.
- [143] Pratheek Senevirathne, Samindu Cooray, Jerome Dinal Herath, and Dinuni Fernando. Virtual machine proactive fault tolerance using log-based anomaly detection. *IEEE Access*, 2024.
- [144] Febrian Setianto, Erion Tsani, Fatima Sadiq, Georgios Domalis, Dimitris Tsakalidis, and Panos Kostakos. Gpt-2c: A parser for honeypot logs using large pre-trained language models. In *Proceedings of the 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM 2021)*, pages 649–653. ACM, 2021. doi: 10.1145/3487351.3492723. URL <https://dl.acm.org/doi/10.1145/3487351.3492723>.
- [145] Shiwen Shan, Yintong Huo, Yuxin Su, Yichen Li, Dan Li, and Zibin Zheng. Face it yourselves: An llm-based two-stage strategy to localize configuration errors via logs. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 13–25, 2024.
- [146] Shiwen Shan, Yintong Huo, Yuxin Su, Zhining Wang, Dan Li, and Zibin Zheng. Conflogger: Enhance systems’ configuration diagnosability through configuration logging. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE ’26)*. ACM, 2026. doi: 10.1145/3744916.3764570. URL <https://arxiv.org/abs/2508.20977>.
- [147] Yangyi Shao, Wenbin Zhang, Peishun Liu, Ren Huyue, Ruichun Tang, Qilin Yin, and Qi Li. Log anomaly detection method based on bert model optimization. In *2022 7th International conference on cloud computing and big data analytics (ICCCBDA)*, pages 161–166. IEEE, 2022.
- [148] Noah Shinn, Beck Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [149] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2019. URL <https://arxiv.org/abs/1909.08053>.
- [150] Honglin Shu, Dong Wang, Antonio Mastropaolo, Gabriele Bavota, and Yasutaka Kamei. An empirical study on language models for generating log statements in test code. *ACM Transactions on Software Engineering and Methodology*, 2025. doi: 10.1145/3759915.
- [151] Chengyu Song, Linru Ma, Jianming Zheng, Jinzhi Liao, Hongyu Kuang, and Lin Yang. Audit-LLM: Multi-agent collaboration for log-based insider threat detection. *arXiv preprint*, 2024. URL <https://arxiv.org/abs/2408.08902>.
- [152] Chengyu Song, Lin Yang, Jianming Zheng, and Fei Cai. Parse-llm: A prior-free llm parser for unknown system logs. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM 2025)*. ACM, 2025. doi: 10.1145/3746252.3761363. URL <https://dl.acm.org/doi/10.1145/3746252.3761363>.
- [153] Shuang Song, Yifei Zhang, and Neng Gao. Confront insider threat: Precise anomaly detection in behavior logs based on LLM fine-tuning. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert, editors, *Proceedings of the 31st International Conference on Computational Linguistics*, pages 8589–8601, Abu Dhabi, UAE, January 2025. Association for Computational Linguistics. URL <https://aclanthology.org/2025.coling-main.574/>.
- [154] Abhay Srivatsa and Venkatasai Gudisa. Logsense: Scalable real-time log anomaly detection architecture.
- [155] Yicheng Sui, Xiaotian Wang, Tianyu Cui, Tong Xiao, Chenghao He, Shenglin Zhang, Yuzhi Zhang, Xiao Yang, Yongqian Sun, and Dan Pei. Bridging the gap: LLM-powered transfer learning for log anomaly detection in new software systems. In *41st IEEE International Conference on Data Engineering, ICDE 2025, Hong Kong, May 19-23, 2025*, pages 4414–4427. IEEE, 2025. doi: 10.1109/ICDE65448.2025.00331. URL <https://doi.org/10.1109/ICDE65448.2025.00331>.

- [156] Danyu Sun, Jinghuai Zhang, Jiachen Xu, Yu Zheng, Yuan Tian, and Zhou Li. From alerts to intelligence: A novel LLM-aided framework for host-based intrusion detection, 2025.
- [157] Yicheng Sun, Jacky Wai Keung, Zhen Yang, Shuo Liu, and Yihan Liao. SemiSMAC: A semi-supervised framework for log anomaly detection with automated hyperparameter tuning. *Information and Software Technology*, 187:107869, 2025. doi: 10.1016/j.infsof.2025.107869.
- [158] Yicheng Sun, Jacky Wai Keung, Zhen Yang, Shuo Liu, and Hi Kuen Yu. Semirald: A semi-supervised hybrid language model for robust anomalous log detection. *Information and Software Technology*, 183:107743, July 2025. doi: 10.1016/j.infsof.2025.107743. URL <https://doi.org/10.1016/j.infsof.2025.107743>.
- [159] Yicheng Sun, Jacky Keung, Zhen Yang, Shuo Liu, and Hi Kuen Yu. Improving anomaly detection in software logs through hybrid language modeling and reduced reliance on parser. *Automated Software Engineering*, 33(1):12, 2026.
- [160] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 27, 2014.
- [161] Sayed Taheri, Achintha Ihalage, Prateek Mishra, Sean Coaker, Faris Muhammad, and Hamed Al-Raweshidy. Domain tailored large language models for log mask prediction in cellular network diagnostics. *IEEE Transactions on Network and Service Management*, 22(3): 2370–2381, 2025. doi: 10.1109/TNSM.2025.3541384.
- [162] Boyin Tan, Junjielong Xu, Zhouruixing Zhu, and Pinjia He. Al-bench: A benchmark for automatic logging, 2025. URL <https://arxiv.org/abs/2502.03160>.
- [163] Pan Tang, Shixiang Tang, Huanqi Pu, Zhiqing Miao, and Zhixing Wang. Microrca-agent: Microservice root cause analysis method based on large language model agents. *arXiv preprint arXiv:2509.15635*, 2025.
- [164] Shimin Tao, Weibin Meng, Yimeng Chen, Yichen Zhu, Ying Liu, Chunling Du, Tao Han, Yongpeng Zhao, Xiangguang Wang, and Hao Yang. Logstamp: Automatic online log parsing based on sequence labelling. *ACM SIGMETRICS Performance Evaluation Review*, 49(4): 93–98, 2022. doi: 10.1145/3543146.3543168. URL <https://dl.acm.org/doi/10.1145/3543146.3543168>.
- [165] Shimin Tao, Yilun Liu, Weibin Meng, Zuomin Ren, Hao Yang, Xun Chen, Liang Zhang, Yuming Xie, Chang Su, Xiaosong Oiao, Weinan Tian, Yichen Zhu, Tao Han, Ying Qin, and Yun Li. Biglog: Unsupervised large-scale pre-training for a unified log representation. In *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*, pages 1–11, 2023. doi: 10.1109/IWQoS57198.2023.10188759.
- [166] Subin Thali, R. Chethan, G. Supreeth, M. Sangamesh, and S. Swami. LLM-based detection of cyber anomalies in industrial control systems. *Zhuza/Foundry*, 28(6):237–241, 2025. doi: 10.29014/FJ-2025-1001-4977.2073.
- [167] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [168] Mohamed Trabelsi, Zenghui Yuan, Jiuyong Han, and Fatima Zohra Ros. LogGPT: Log anomaly detection via GPT. *arXiv preprint*, 2023. URL <https://arxiv.org/abs/2309.14482>.
- [169] Risto Vaarandi and Hayretdin Bahsi. Using large language models for template detection from security event logs. *International Journal of Information Security*, 24, 2025. doi: 10.1007/s10207-025-01018-y. URL <https://doi.org/10.1007/s10207-025-01018-y>.
- [170] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [171] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th International Conference on Machine Learning (ICML)*, pages 1096–1103, 2008. doi: 10.1145/1390156.1390294.
- [172] Jiabo Wang, Guojun Chu, Jingyu Wang, Haifeng Sun, Qi Qi, Yuanyi Wang, Ji Qi, and Jianxin Liao. Logexpert: Log-based recommended resolutions generation using large language model. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, ICSE-NIER'24*, page 42–46, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400705007. doi: 10.1145/3639476.3639773. URL <https://doi.org/10.1145/3639476.3639773>.
- [173] Kehan Wang, Siqin Zhang, Haijing Nan, Xueyu Hou, Jiaqi Zou, and Zicong Miao. *Empirical Analysis of LLM-DPP: Advancing Log Parsing in the LLM Era*, page 753–758. Association for Computing Machinery, New York, NY, USA, 2025. ISBN 9798400714535. URL <https://doi.org/10.1145/3711875.3736683>.
- [174] Tingting Wang and Guilin Qi. A comprehensive survey on root cause analysis in (micro) services: Methodologies, challenges, and trends. *arXiv preprint arXiv:2408.00803*, 2024.
- [175] Xin Wang, Zhenhao Li, and Zishuo Ding. Defects4log: Benchmarking llms for logging code defect detection and reasoning, 2025. URL <https://arxiv.org/abs/2508.11305>.
- [176] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2022. URL <https://arxiv.org/abs/2203.11171>.
- [177] Yilun Wang, Pengfei Chen, Haiyu Huang, Zilong He, Gou Tan, Chuanfu Zhang, Jingkai He, and Zibin Zheng. Inferlog: Accelerating llm inference for online log parsing via icl-oriented prefix caching. In *Proceedings of the 48th International Conference on Software Engineering (ICSE 2026)*, 2026. URL <https://arxiv.org/abs/2507.08523>. Accepted.
- [178] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*

- (EMNLP), pages 8696–8708. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.emnlp-main.685. URL <https://aclanthology.org/2021.emnlp-main.685/>.
- [179] Zefan Wang, Zichuan Liu, Yingying Zhang, Aoxiao Zhong, Jihong Wang, Fengbin Yin, Lunting Fan, Lingfei Wu, and Qingsong Wen. Ragent: Cloud root cause analysis by autonomous agents with tool-augmented large language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 4966–4974, 2024.
 - [180] Jason Wei, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Jeff Dean, Yuhuai Li, Adam Roberts, Anna Rumshisky, Noam Shazeer, et al. Emergent abilities of large language models. arXiv preprint arXiv:2206.07682, 2022. URL <https://arxiv.org/abs/2206.07682>.
 - [181] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2022. URL <https://arxiv.org/abs/2201.11903>.
 - [182] Yifan Wu, Siyu Yu, and Ying Li. Log parsing using llms with self-generated in-context learning and self-correction. In *Proceedings of the 33rd IEEE/ACM International Conference on Program Comprehension (ICPC 2025)*. IEEE/ACM, 2025. URL <https://arxiv.org/abs/2406.03376>. Method name: AdaParser.
 - [183] Pei Xiao, Tong Jia, Chiming Duan, Minghua He, Weijie Hong, Xixuan Yang, Yihan Wu, Ying Li, and Gang Huang. Csllog: Collaborating large and small models for log-based anomaly detection. In *Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25)*, pages 686–690, Trondheim, Norway, June 2025. ACM. doi: 10.1145/3696630.3728524. URL <https://doi.org/10.1145/3696630.3728524>.
 - [184] Yi Xiao, Van-Hoang Le, and Hongyu Zhang. Demonstration-free: Towards more practical log parsing with large language models. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE 2024)*, pages 1243–1255. ACM, 2024. doi: 10.1145/3691620.3694994. URL <https://dl.acm.org/doi/10.1145/3691620.3694994>. Method name: LogBatcher.
 - [185] Xiaoyuan Xie, Zhipeng Cai, Songqiang Chen, and Jifeng Xuan. Fastlog: An end-to-end method to efficiently generate and insert logging statements. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24)*, pages 26–37. ACM, 2024. doi: 10.1145/3650212.3652107. URL <https://arxiv.org/abs/2311.02862>.
 - [186] Andy Xu and Arno Gau. Help: Hierarchical embeddings-based log parsing. arXiv preprint arXiv:2408.08300, 2024. URL <https://arxiv.org/abs/2408.08300>.
 - [187] Junjielong Xu, Ziang Cui, Yuan Zhao, Xu Zhang, Shilin He, Pinjia He, Liqun Li, Yu Kang, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. Unilog: Automatic logging via LLM and in-context learning. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24)*. ACM, 2024. doi: 10.1145/3597503.3623326.
 - [188] Junjielong Xu, Ruichun Yang, Yintong Huo, Chengyu Zhang, and Pinjia He. Divlog: Log parsing with prompt enhanced in-context learning. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24)*, pages 199:1–199:12, 2024. doi: 10.1145/3597503.3639155. URL <https://doi.org/10.1145/3597503.3639155>.
 - [189] Junjielong Xu, Qinan Zhang, Zhiqing Zhong, Shilin He, Chaoyun Zhang, Qingwei Lin, Dan Pei, Pinjia He, Dongmei Zhang, and Qi Zhang. Openrca: Can large language models locate the root cause of software failures? In *The Thirteenth International Conference on Learning Representations*, 2025.
 - [190] Weiyuan Xu, Juntao Luo, Tao Huang, Kaixin Sui, Jie Geng, Qijun Ma, Isami Akasaka, Xiaoxue Shi, Jing Tang, and Peng Cai. A two-staged llm-based framework for ci/cd failure detection and remediation with industrial validation. arXiv preprint arXiv:2506.03691, 2025.
 - [191] Yuuki Yamanaka, Tomokatsu Takahashi, Takuya Minami, and Yoshiaki Nakajima. LogELECTRA: Self-supervised anomaly detection for unstructured logs. arXiv preprint, 2024. URL <https://arxiv.org/abs/2402.10397>.
 - [192] N. Yang, P. J. L. Cuijpers, D. Hendriks, R. Schiffelers, J. Lukkien, and A. Serebrenik. An interview study about the use of logs in embedded software engineering. *Empirical Software Engineering*, 28(2):43, 2023. doi: 10.1007/s10664-022-10258-8.
 - [193] Tiankai Yang, Yi Nian, Li Li, Ruiyao Xu, Yuangang Li, Jiaqi Li, Zhuo Xiao, Xiyang Hu, Ryan A. Rossi, Kaize Ding, Xia Hu, and Yue Zhao. AD-LLM: Benchmarking large language models for anomaly detection. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 1524–1547, Vienna, Austria, July 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.findings-acl.79. URL <https://aclanthology.org/2025.findings-acl.79/>.
 - [194] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2022. URL <https://arxiv.org/abs/2210.03629>.
 - [195] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023. URL <https://arxiv.org/abs/2305.10601>.
 - [196] Xian Yu, Shengxi Nong, Dongbiao He, Weijie Zheng, Teng Ma, Ning Liu, Jianhui Li, and Gaogang Xie. Loggenius: An unsupervised log parsing framework with zero-shot prompt engineering. In *2024 IEEE International Conference on Web Services (ICWS)*, pages 1321–1328, 2024. doi: 10.1109/ICWS62655.2024.00159. URL <https://doi.org/10.1109/ICWS62655.2024.00159>.
 - [197] Chenbo Zhang, Wenying Xu, Jinbu Liu, Lu Zhang, Guiyang Liu, Jihong Guan, Qi Zhou, and Shuigeng Zhou. Logbase: A large-scale benchmark for semantic log parsing. In *Proceedings of the 34th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2025)*. ACM, 2025. doi: 10.1145/3728969. URL <https://dl.acm.org/doi/10.1145/3728969>.

- [198] Chenbo Zhang, Wenying Xu, Jinbu Liu, Lu Zhang, Guiyang Liu, Jihong Guan, Qi Zhou, and Shuigeng Zhou. Semanticlog: Towards effective and efficient large-scale semantic log parsing. *IEEE Transactions on Software Engineering*, 2025. doi: 10.1109/TSE.2025.3524891. URL <https://ieeexplore.ieee.org/abstract/document/11216353>.
- [199] Hao Zhang, Dongjun Yu, Lei Zhang, Guoping Rong, Yongda Yu, Haifeng Shen, He Zhang, Dong Shao, and Hongyu Kuang. Aucad: Automated construction of alignment dataset from log-related issues for enhancing llm-based log generation. In *Proceedings of the 16th International Conference on Internetware*, Internetware '25, page 413–425, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400719264. doi: 10.1145/3755881.3755889. URL <https://doi.org/10.1145/3755881.3755889>.
- [200] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip Yu, and Ying Li. A survey of aiops in the era of large language models. *ACM Computing Surveys*, 2025.
- [201] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Hongyi Liu, and Ying Li. Scalalog: Scalable log-based failure diagnosis using llm. In *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5, 2025. doi: 10.1109/ICASSP49660.2025.10888670.
- [202] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Hongyi Liu, and Ying Li. Xraglog: A resource-efficient and context-aware log-based anomaly detection method using retrieval-augmented generation. In *AAAI 2025 Workshop on Preventing and Detecting LLM Misinformation (PDLMI)*, 2025.
- [203] Lingzhe Zhang, Tong Jia, Kangjin Wang, Weijie Hong, Chiming Duan, Minghua He, and Ying Li. Adaptive root cause localization for microservice systems with multi-agent recursion-of-thought. *arXiv preprint arXiv:2508.20370*, 2025.
- [204] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675*, 2019.
- [205] Ting Zhang, Xin Huang, Wen Zhao, Shaohuang Bian, and Peng Du. Logprompt: A log-based anomaly detection framework using prompts. In *2023 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2023. doi: 10.1109/IJCNN54540.2023.10191948.
- [206] Wanhao Zhang, Qianli Zhang, Enyu Yu, Yuxiang Ren, Yeqing Meng, Mingxi Qiu, and Jilong Wang. Leveraging rag-enhanced large language model for semi-supervised log anomaly detection. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, pages 168–179, 2024. doi: 10.1109/ISSRE62328.2024.00026.
- [207] Wei Zhang, Xiangyuan Guan, Lu Yunhong, Jie Zhang, Shuangyong Song, Xianfu Cheng, Zhenhe Wu, and Zhoujun Li. Lemur: Log parsing with entropy sampling and chain-of-thought merging, 2025. URL <https://arxiv.org/abs/2402.18205>.
- [208] Xiao Zhang, Qi Wang, Mingyi Li, Yuan Yuan, Mengbai Xiao, Fuzhen Zhuang, and Dongxiao Yu. Tamo: Fine-grained root cause analysis via tool-assisted llm agent with multi-modality observation data in cloud-native systems. *IEEE Transactions on Services Computing*, 18(6):4221–4233, 2025.
- [209] Xuchao Zhang, Supriyo Ghosh, Chetan Bansal, Rujia Wang, Minghua Ma, Yu Kang, and Saravan Rajmohan. Automated root causing of cloud incidents using in-context learning with gpt-4. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, pages 266–277, 2024.
- [210] Zhaohui Zhang, Shu Chen, and Dandan Shi. LogST: Log semi-supervised anomaly detection based on sentence-BERT. In *2022 6th International Conference on System Innovation (ICSIP)*, pages 356–361, 2022. doi: 10.1109/ICSIP55141.2022.9886069.
- [211] Haoyu Zheng, Guojun Chu, Haifeng Sun, Jingyu Wang, Shimin Tao, and Hao Yang. LogDAPT: Log data anomaly detection with domain-adaptive pretraining (industry track). In *Proceedings of the 24th International Middleware Conference: Industrial Track (Middleware '23 Industry)*, pages 15–21. Association for Computing Machinery, 2023. doi: 10.1145/3626562.3626830. URL <https://doi.org/10.1145/3626562.3626830>.
- [212] Chen Zhi, Liye Cheng, Meilin Liu, Xinkui Zhao, Yueshen Xu, and Shuiguang Deng. Llm-powered zero-shot online log parsing. In *2024 IEEE International Conference on Web Services (ICWS)*, pages 877–887. IEEE, 2024.
- [213] Aoxiao Zhong, Dengyao Mo, Guiyang Liu, Jinbu Liu, Qingda Lu, Qi Zhou, Jiesheng Wu, Quanzheng Li, and Qingsong Wen. Logparser-llm: Advancing efficient log parsing with large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 4559–4570, 2024.
- [214] Renyi Zhong, Yichen Li, Jinxi Kuang, Wenwei Gu, Yintong Huo, and Michael R. Lyu. Logupdater: Automated detection and repair of specific defects in logging statements. *ACM Transactions on Software Engineering and Methodology*, 2025. doi: 10.1145/3731754.
- [215] Renyi Zhong, Yichen Li, Guangba Yu, Wenwei Gu, Jinxi Kuang, Yintong Huo, and Michael R. Lyu. Larger is not always better: Exploring small open-source language models in logging statement generation. *ACM Trans. Softw. Eng. Methodol.*, October 2025. ISSN 1049-331X. doi: 10.1145/3773287. URL <https://doi.org/10.1145/3773287>. Just Accepted.
- [216] Dahlia Ziqi Zhou and Marios Fokaefs. Ai assistants for incident lifecycle in a microservice environment: A systematic literature review. *arXiv preprint arXiv:2410.04334*, 2024.
- [217] Denny Zhou, Nathanael Sch"arli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models, 2022. URL <https://arxiv.org/abs/2205.10625>.
- [218] Yihan Zhou, Yan Chen, Xuanming Rao, Yukang Zhou, Yuxin Li, and Chao Hu. Leveraging large language models and bert for log parsing and anomaly detection. *Mathematics*, 12(17), 2024. ISSN 2227-7390. doi: 10.3390/math12172758. URL <https://www.mdpi.com/2227->

- 7390/12/17/2758.
- [219] Jieming Zhu, Pinjia He, Qiang Fu, Hongyu Zhang, Michael R. Lyu, and Dongmei Zhang. Learning to log: Helping developers make informed logging decisions. In *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering*. IEEE/ACM, 2015. doi: 10.1109/ICSE.2015.60.
 - [220] Xuhang Zhu, Xiu Tang, Sai Wu, Gang Chen, Haobo Wang, Chang Yao, Quanqing Xu, and Jichen Li. CoLA: Model collaboration for log-based anomaly detection. *Proceedings of the VLDB Endowment*, 18(11):3979–3987, 2025. doi: 10.14778/3749646.3749668.
 - [221] Xinkai Zou, Xuan Jiang, Ruikai Huang, Haoze He, Parv Kapoor, Hongrui Wu, Yibo Wang, Jian Sha, Xiongbo Shi, Zixun Huang, and Jinhua Zhao. Towards generalizable context-aware anomaly detection: A large-scale benchmark in cloud environments, 2025. URL <https://arxiv.org/abs/2508.01844>.

Just Accepted